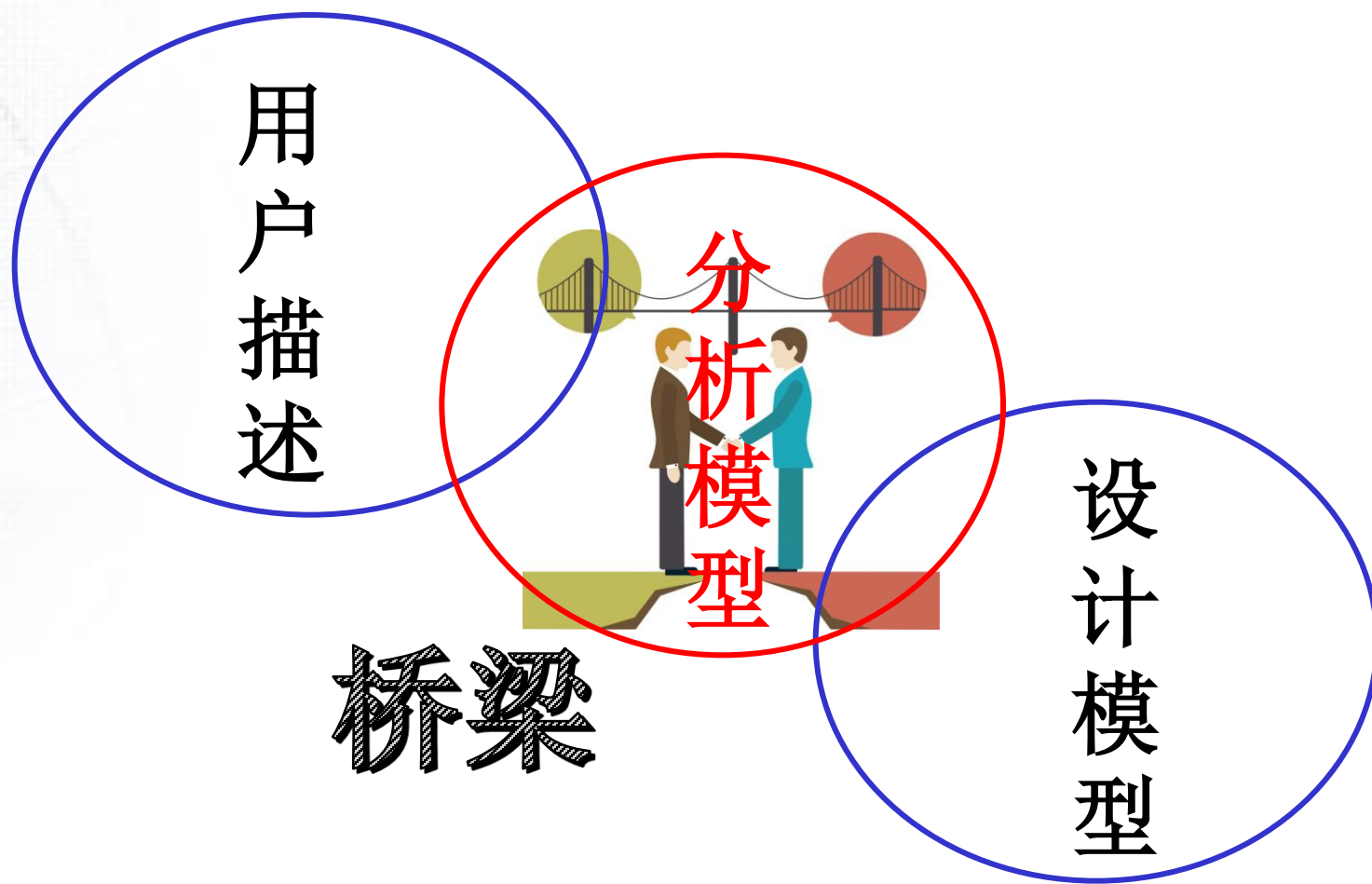




回顾需求分析活动





Chapter 5 需求分析方法

- 5.1 需求分析的原则
- 5.2 需求收集方法
- 5.3 传统需求分析建模方法
- 5.4 面向对象的需求分析建模方法
- 5.5 相关文档规范





5.1 需求分析的原则(1/22)

- 1)循序渐进

- 理由

- 系统的规模越大且复杂->难以一下子理解完整
 - 急于求成->“边建边改” or 错误 or 功能不完善

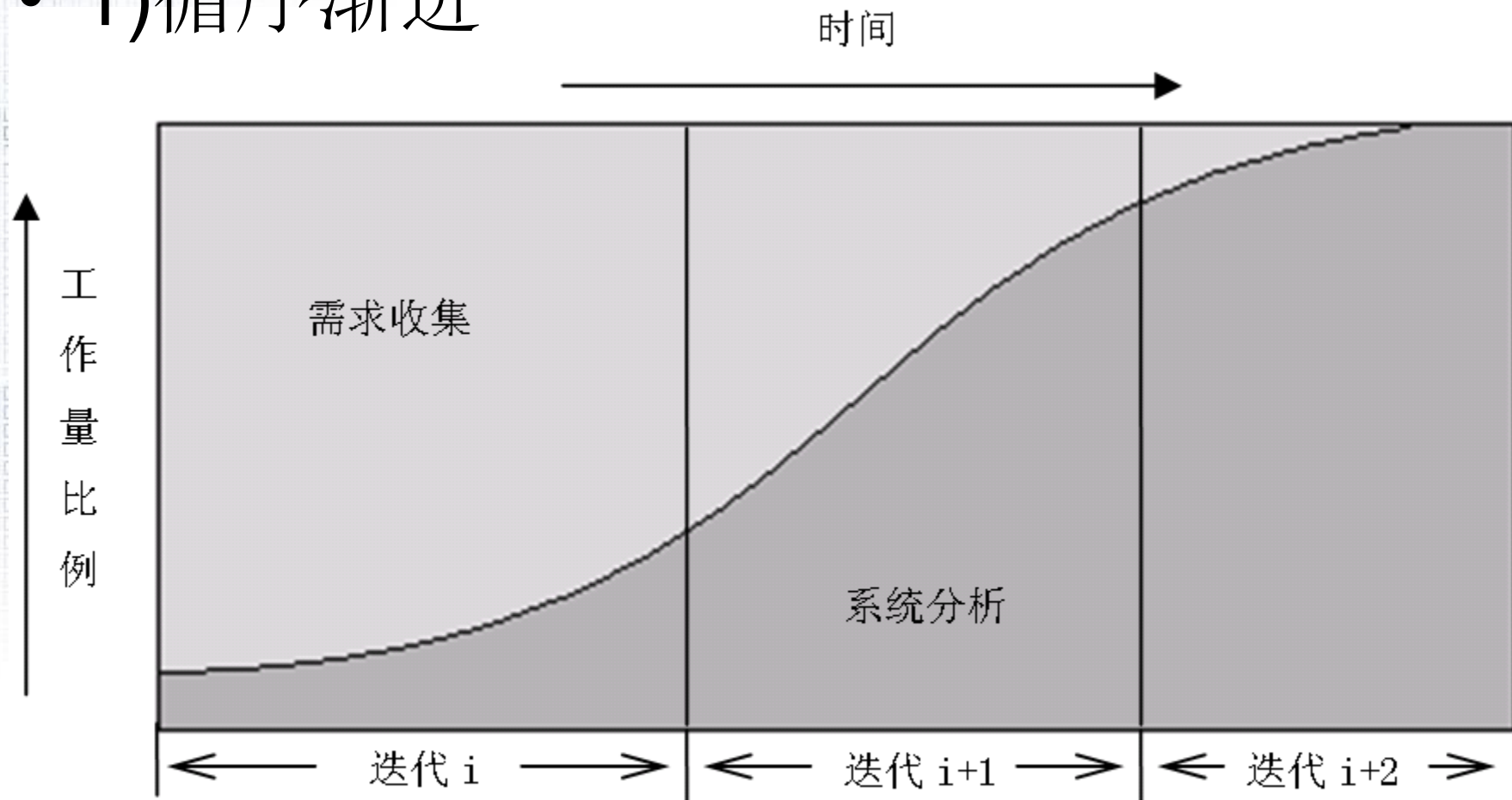
- 步骤

- 采集原始需求
 - 整理需求
 - 建立需求分析模型
 - 编写需求规约文档
 - 复审



5.1 需求分析的原则(2/22)

- 1) 循序渐进



需求收集和系统分析在不同时期的工作量比较



5.1 需求分析的原则(3/22)

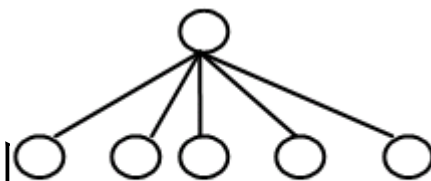
- 2) 自顶向下，逐层分解

- 理由

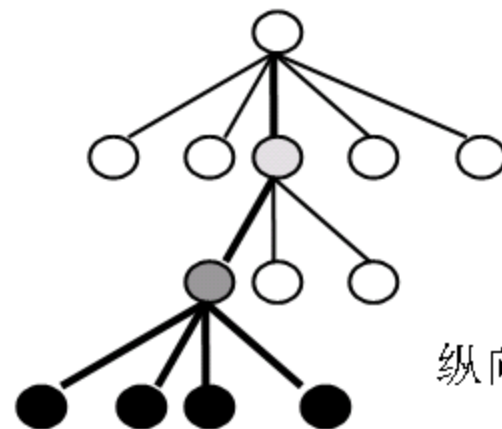
- 庞大的、复杂的系统很难一下子被完全理解
 - 小鸟视角
 - 对一个个小任务分别把握和控制

- 分解

- 分解还是分割？
 - 怎样分解？
 - 分解多少/多深为宜？



横向分解

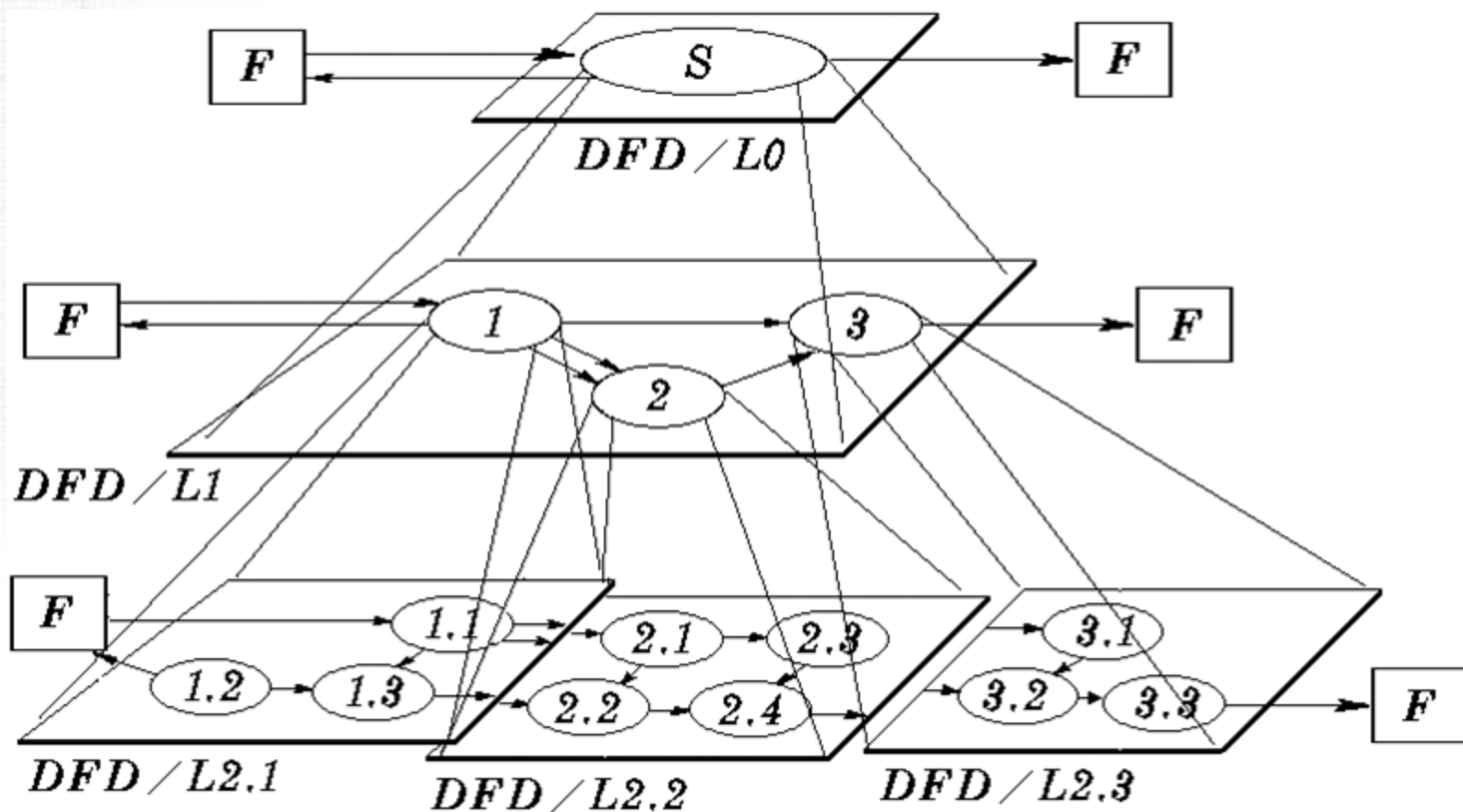


纵向分解



5.1 需求分析的原则(4/22)

- 2) 自顶向下，逐层分解



逐层分解的数据流程图



5.1 需求分析的原则(5/22)

• 3)与实现分离

— 理由

- 避免记录一些因为当前的技术才存在的需求，或者使用一些可能不适合新产品的技术
 - 例5.1 一个地铁售票产品的需求被描述为“乘客通过键盘输入目的地车站”。
 - 修改：“本产品接受乘客输入的目的地”。
- 避免对实现的方式做出束缚。除非已经严格的做出要求，否则一般不应使用属于实现的描述
 - 例5.2 在表达宠物商店的清空购物车功能时，将其描述为“用户按下‘确定’按钮，系统则删除当前数据记录”。



5.1 需求分析的原则(6/22)

- 3)与实现分离

- 理由（续）

- 各尽其责

- 解决办法

- 职业化、专业化的需求分析人员
 - 复审



5.1 需求分析的原则(7/22)

- 4)定义需求属性

- 理由

- *每个需求仅仅是几行描述语句吗?*
 - 评估测试故障对需求的影响
 - 评估用户对系统的满意程度
 - 估算开发成本
 - 管理项目的规模
 - 分配开发资源
 - 安排调度计划
 - 管理需求变更
 - 管理项目风险



5.1 需求分析的原则(8/22)

- 4)定义需求属性

- 属性包括

- 每个需求并不仅仅是一行描述语句。每个需求都有类型、原因、开发优先级、风险、客户满意度、客户不满意度、依赖关系、冲突关系，以及来源等属性
 - 随着开发的深入，还可以继续丰富需求的属性，如开发难度、开发工作量等



5.1 需求分析的原则(9/22)

- 5)可验证性

- 理由

- 证明所开发的系统符客户和用户的要求的依据
 - 不可验证的需求，仅仅是对需求的一种主观愿望，对于设计和测试等活动而言都是缺乏意义的
 - 度量出系统实现的质量



5.1 需求分析的原则(10/22)

- 5)可验证性

- 功能性需求的可验证性

- 例：宠物商店有一个功能需求如下：

- 系统的服务器为用户提供数据下载服务

- 验收标准

- 服务器端下载的数据和客户端下载的数据完全一致



5.1 需求分析的原则(11/22)

- 5)可验证性

- 非功能性需求的可验证性

- 易用性

- 描述：系统应该用户友好。
- 验收标准1：在所有连锁店，经理第一次使用该产品时，他们能够在1小时内掌握下载数据、上传财务报表、查询销售额等权限内的功能。
- 验收标准2：经过开发方培训服务，管理员能够在3个培训个工作日内掌握系统的所有功能，并能独立工作。在开始的连续30个工作日内，因为不熟悉功能而需要开发方上门服务的次数最多有1次。
- 验收标准3：在使用该网站的前3个月，至少有95%的宠物商店的会员对该网站表示满意。



5.1 需求分析的原则(12/22)

- 5)可验证性

- 非功能性需求的可验证性

- 性能

- 描述：查询宠物信息时，系统响应速度应该足够快。

- 验收标准：在90%的情况下，响应时间将不超过3秒钟；在其他情况下，不超过10秒钟。

- 可靠性

- 描述：系统应该具体很高的可靠性。

- 验收标准1：在使用该产品的前3个月，系统从未发生过崩溃（数据不可恢复）或者死机现象。

- 验收标准2：在3个随机确定的工作日，系统的 $Q_{reliability}$ 指标都应大于0.95。



5.1 需求分析的原则(13/22)

- 5)可验证性

- 非功能性需求的可验证性

- 安全性

- 描述1：只有连锁的经理具有才能使用下载数据、上传财务报表、查询销售额等权限范围内的功能。
- 验收标准：在1个月内系统应用中，下载数据、上传财务报表、查询销售额等功能都是连锁店经理完成的。
- 描述2：系统应该能够自动备份数据。
- 验收标准：系统能够在用户设定的时间周期内（如每隔1小时），定时备份数据，所备份的数据与当前数据库的数据应完全一致。



5.1 需求分析的原则(14/22)

- 5)可验证性

- 非功能性需求的可验证性

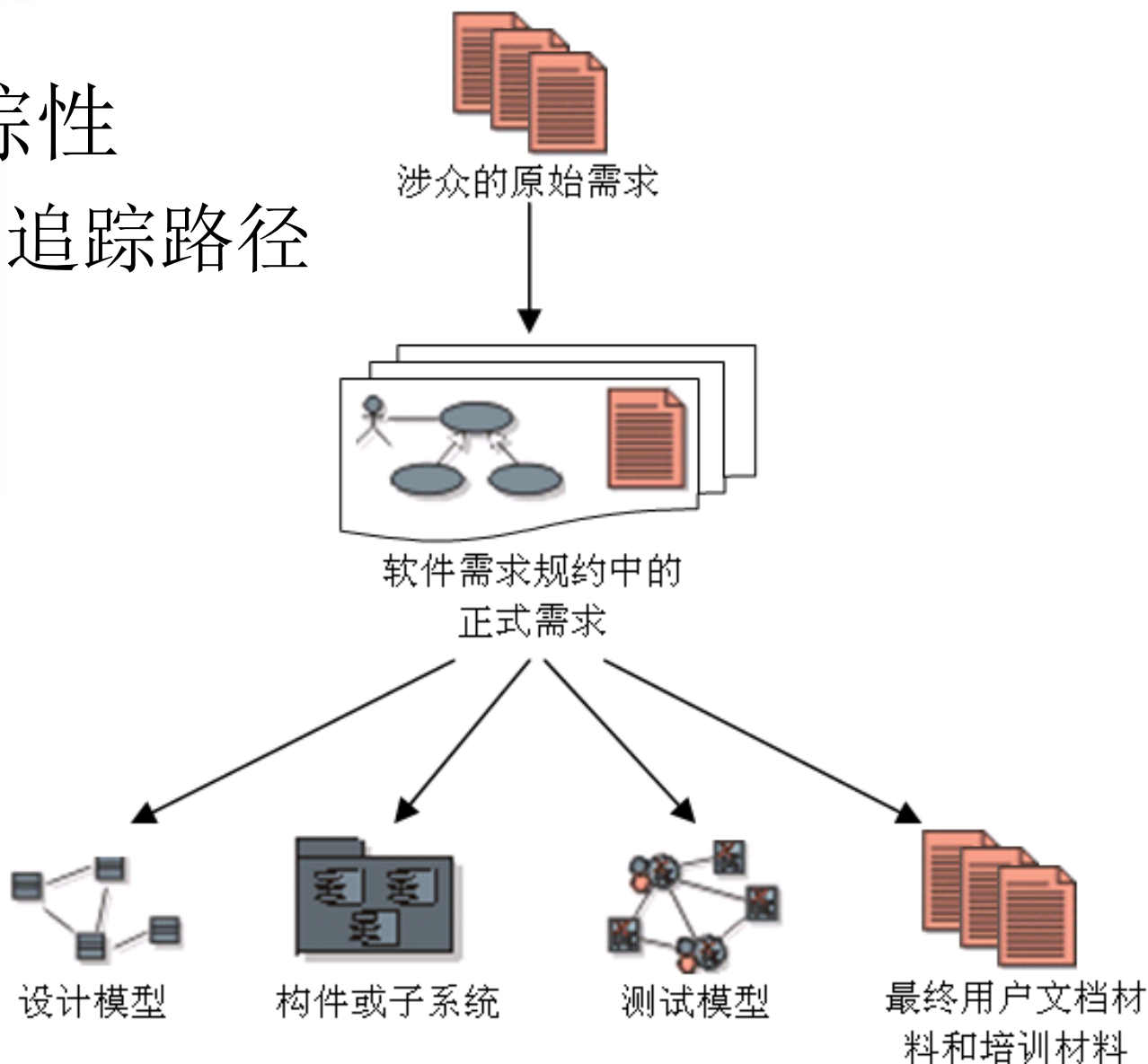
- 架构可扩展性

- 随着业务的发展，当访问量和数据量增加时，可横向添加数据库设备，满足存储及数据分析的需求；可横向添加服务器，满足并发连接数的需求；可横向添加内存，满足整个系统对内存的使用。以上拓展均不需要重构代码，且服务中断不得超过1个小时。



5.1 需求分析的原则(15/22)

- 6)可追踪性
— 需求的追踪路径





5.1 需求分析的原则(16/22)

- 6)可追踪性

- 理由

- 跟踪问题：在工件的转换过程中，可能会出现很多问题，而问题存在传递性
 - 开发成员误解需求
 - 错误地使用需求
 - 变更需求
 - ...
 - 分析潜在变更的影响
 - 核实通过实施系统所有的需求都已经被实现



5.1 需求分析的原则(17/22)

- 7)其它原则
 - 使用术语
 - 尊重客户的意见
 - 重视复用需求
 - 对变更进行管理
 - 要求“确认需求”



5.1 需求分析的原则(18/22)

- 对需求描述的要求

- 1) 清晰性：即需求的描述必须明确，不能模糊不清。

- 例如：“每小时比目前处理更多的工作指令”

- 避免使用的词：

- 比较型——更快、更好、更多以及更亮？“更快”是多快？“更好”是如何定义？“更多”是多少？这些需要被量化。

- 不精确的形容词——快的、健壮的、用户友好的、有效的、灵活的。在商业案例、营销材料中看似很好，但如果在需求报告中使用时，显得不精确。

- 含糊不清——最小、最大、优化



5.1 需求分析的原则(19/22)

- 对需求描述的要求
 - 2) 简洁性：即针对需求的描述要简明扼要，一般采用各种图表并辅以文字说明加以表示。
 - 3) 一致性：各种需求之间不应存在不一致、相互矛盾的现象，并且应被所有相关人员认可。如
 - 预约时间减少到两小时之内
 - 满足90%的预约计划



5.1 需求分析的原则(20/22)

- 对需求描述的要求

- 4) 无歧义性：需求的描述应该准确，不能产生歧义。

- 不能有模糊词

- 大概、可能、尽量、很快、方便、安全、合适

- 可量化

- 长度、时间、次数、格式、范围、条件

- 只有一种理解方式

- 张三、李四、王五读了，理解必须完全一样

- 例如：

- 找到从起点到终点的“最佳”路径

- 密码要设置得安全一点



5.1 需求分析的原则(20/22)

- 对需求描述的要求
 - 5) 有意义性：需求是各个有意义方面的陈述的一个集合，所有与新系统开发无关的方面都不应涉及（即无意义的），描述的需求应使用陈述句的形式，不应存在疑问



5.1 需求分析的原则(21/22)

- 对需求描述的要求
 - 6) 可验证性：所有需求必须可以通过某种方法进行验证，否则就没有意义
 - 每小时至少处理100个工作指令
 - 在一个正常的工作日内，平均每小时至少处理100个工作指令
 - 7) 唯一性：所有需求必须具有唯一性，避免描述重复的需求。
 - 8) 完整性：描述需求必须是完整的不应有遗漏。
 - 功能、性能、限制条件、操作环境



5.1 需求分析的原则(22/22)

- 对需求描述的要求
 - 9) 可追踪性：每个需求必须描述其来源，即该需求的提出者是谁，以便需求变更、设计和测试等工作能有针对性
 - 10) 有界性：对做什么、不做什么的需求描述要有明确的界限划分；用户对各项需求的要求程度也是不同的，有的是必须完成的，因此需要对这些不同程度的需求加以区分（优先级别**MSCW**）



需求分析原则小结

需求分析的原则

- 循序渐进
- 自顶向下，逐层分解
- 与实现分离
- 定义需求属性
- 可验证性
- 可追踪性
- 其它原则

需求描述的要求

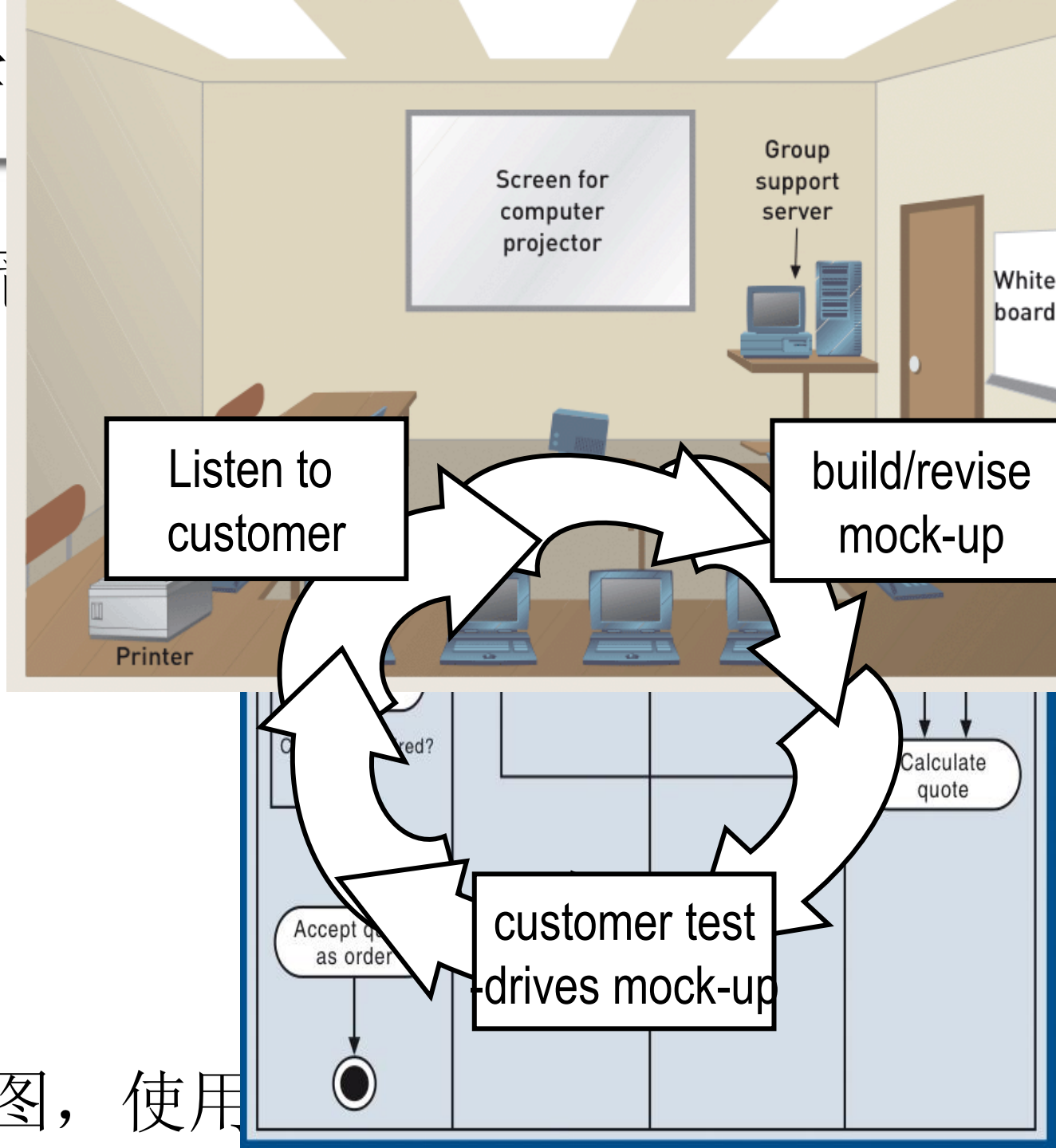
- 清晰
- 简洁、唯一
- 一致
- 无歧义、有意义
- 可追踪性
- 可验证性
- 有界性

理解&小练习



5.2 需

- 与用户交流
 - 用户访谈
 - 做学徒
 - 问卷调查
 - 讨论会
 - 头脑风暴
 - 原型法
 - ...
- 交流手段
 - 使用思维图, 使用





5.2 需求

- 交流手段
—原型法

学号

姓名

返校日期

手机号码

短信验证码

确认

信息查询

学号	姓名	专业	班级	宿舍	寝室号	返校日期时间
20210213	张三	软件工程	软2101	东3	东301 2022.02.26	09:30
20210214	李四	软件工程	软2101	东3	东301 2022.02.26	12:50
20210215	王五	软件工程	软2101	东3	东301 2022.02.26	12:50
20210216	赵六	软件工程	软2101	东3	东301 2022.02.26	16:10
20210217	张三	软件工程	软2101	东3	东301 2022.02.26	09:30
20210218	李四	软件工程	软2101	东3	东301 2022.02.26	12:50
20210219	王五	软件工程	软2101	东3	东301 2022.02.26	12:50
20210230	赵六	软件工程	软2101	东3	东301 2022.02.26	16:10
20210132	李四	软件工程	软2101	东3	东301 2022.02.26	11:30
20210133	王五	软件工程	软2101	东3	东301 2022.02.26	12:50
20210134	赵六	软件工程	软2101	东3	东301 2022.02.26	16:10

信息查询

学号查询: 姓名: 班级: 返校日期: 清空 查询

学号	姓名	专业	班级	宿舍	寝室号
20210213	张三	软件工程	软2101	东3	东301 2022.02.26
20210214	李四	软件工程	软2101	东3	东301 2022.02.26
20210216	王五	软件工程	软2101	东3	东301 2022.02.26
20210218	张六	软件工程	软2101	东3	东301 2022.02.26
20210219	张三	软件工程	软2101	东3	东301 2022.02.26
20210230	赵六	软件工程	软2101	东3	东301 2022.02.26
20210132	李四	软件工程	软2101	东3	东301 2022.02.26
20210133	王五	软件工程	软2101	东3	东301 2022.02.26
20210134	赵六	软件工程	软2101	东3	东301 2022.02.26

信息查询

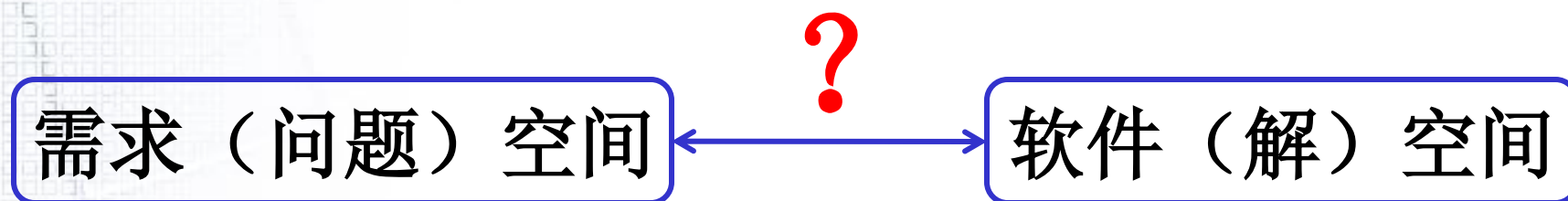
学号查询: 姓名: 班级: 返校日期: 清空 查询 打印

学号	姓名	专业	班级	宿舍	寝室号	返校日期时间
20210213	张三	软件工程	软2101	东3	东301 2022.02.26	09:30
20210214	李四	软件工程	软2101	东3	东301 2022.02.26	12:50
20210216	王五	软件工程	软2101	东3	东301 2022.02.26	16:10
20210218	张六	软件工程	软2101	东3	东301 2022.02.26	19:30
20210219	张三	软件工程	软2101	东3	东301 2022.02.26	12:50
20210230	赵六	软件工程	软2101	东3	东301 2022.02.26	16:10
20210132	李四	软件工程	软2101	东3	东301 2022.02.26	11:30
20210133	王五	软件工程	软2101	东3	东301 2022.02.26	12:50
20210134	赵六	软件工程	软2101	东3	东301 2022.02.26	16:10



5.3 需求分析建模

- WHY?



- 表达手段：建模



5.3 传统需求分析建模方法

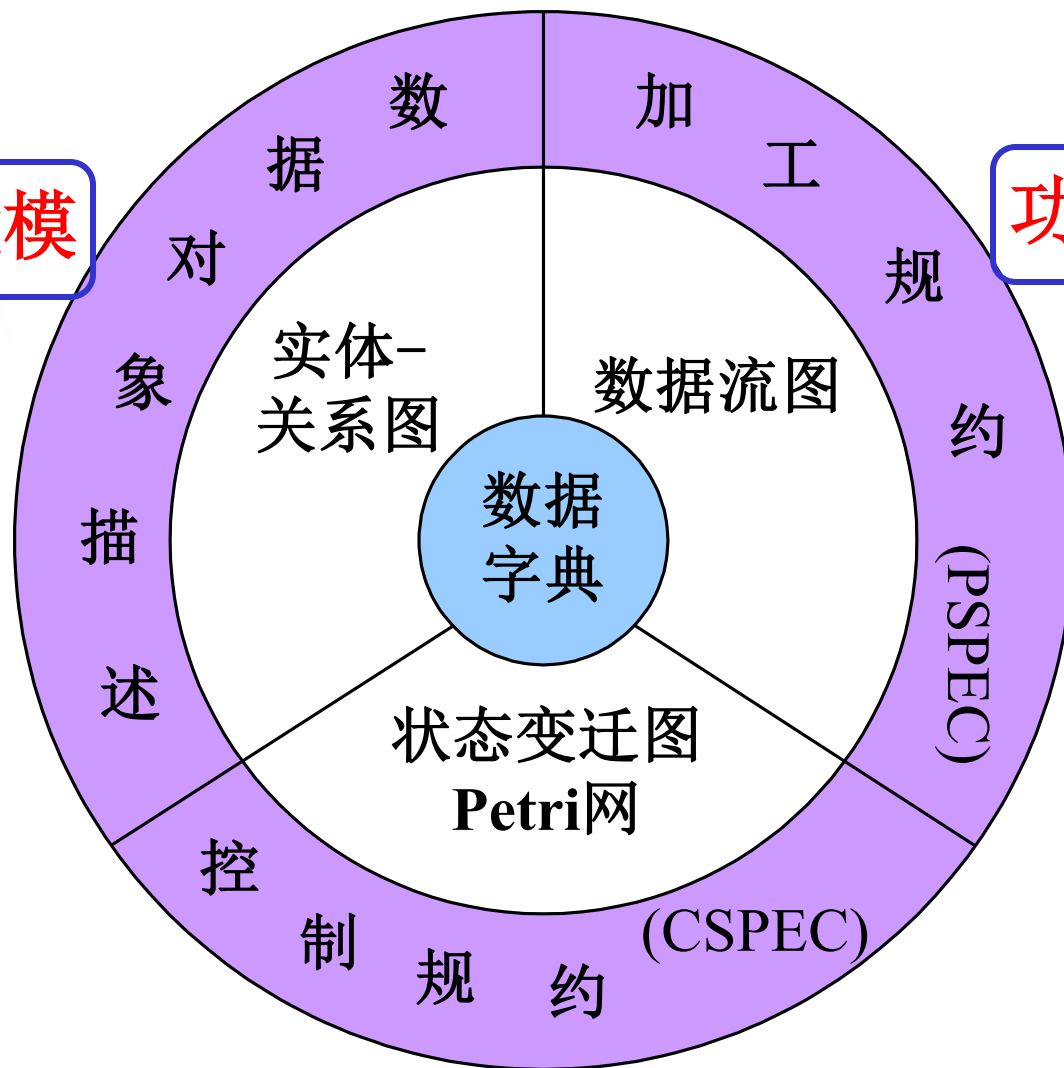
- 发展历史
 - 提出：20世纪60年代末到70年代初
 - 成熟：20世纪70年代末到80年代中期
- 主要思想：抽象与自顶向下的逐层分解
(控制复杂性的两个基本手段)



5.3 传统需求分析建模方法

- 分析模型

数据建模



功能建模

行为建模



5.3 传统需求分析建模方法

- 数据建模
- 功能建模
 - 数据流程图(Data Flow Diagram, DFD)
- 行为建模
 - 状态变迁图(State-Transition Diagram, STD)
 - Petri网
- 数据字典
- 判定表和判定树



5.3 传统——数据流程图(1/12)

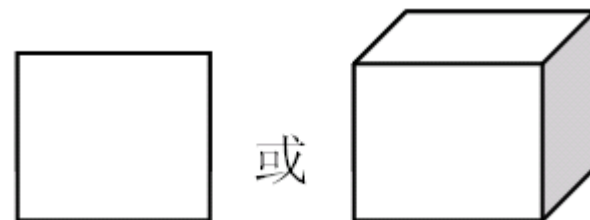
- 数据流程图Data Flow Diagram(简称DFD)
 - 从数据传递和加工角度，描述系统中数据流从输入到输出的移动和变换（加工）的过程
 - 一个分层的概念模型。3个层次：总体图，零级图，细节图；分别描述系统的不同特征
 - 4种符号：方，圆，开口矩（平行线），箭头
- 优点
 - 容易理解，容易在开发方和用户方之间进行交流，以及在开发组织内部交流



5.3 传统——数据流程图(2/12)

- 表示符号

- 外部实体



- 数据处理



- 数据存储



- 数据流



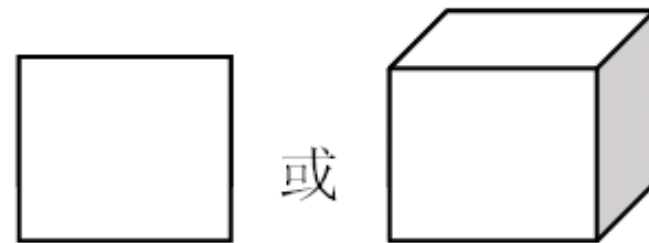


5.3 传统——数据流程图(3/12)

• 基本图形元素

- 软件系统之外的人员或组织
- 外部实体（宿或源），表示软件系统输入数据的来源和输出数据去向
 - 例如，一个考务处理系统
 - 考生：考生向系统提供报名表(输入数据流)
 - 考试中心：系统将考试成绩的统计分析表(输出数据流)传给考试中心
- 外部实体用相同的图形符号表示

- 当数据流从该符号流出时表示是源
- 当数据流流向该符号时表示是宿
- 当两者皆有时表示既是源又是宿





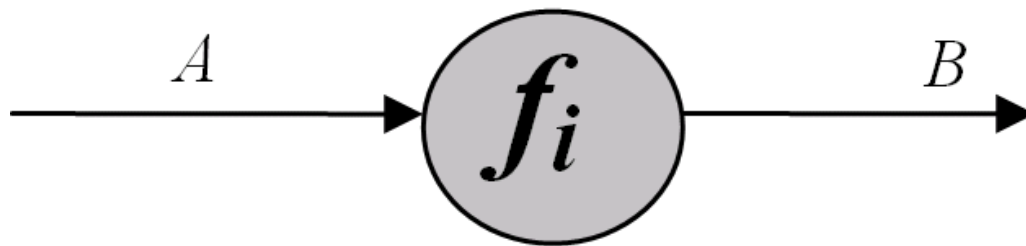
5.3 传统——数据流程图(4/12)

- 基本图形元素

- 数据处理（加工）：描述输入数据流到输出数据流的变换



- 每个处理用一个明确的名字标识
- 至少有一个输入数据流和一个输出流
- 可以有多个输入数据流和多个输出数据流

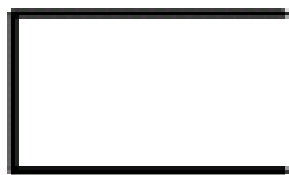


一个简单的数据变换

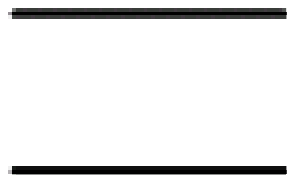


5.3 传统——数据流程图(5/12)

- 数据存储（文件）：保存数据信息的外部单元
 - 每个文件有明确的名字标识
 - 由数据处理进行读写
 - **DFD**称文件，（文件系统或数据库系统实现）



或





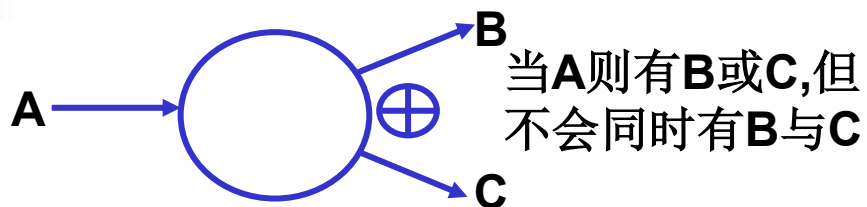
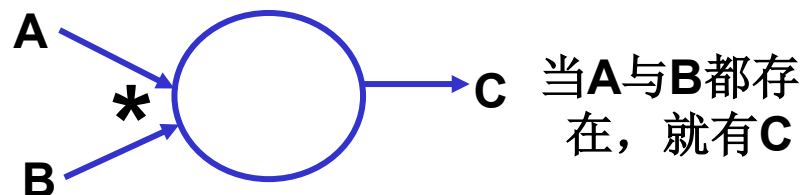
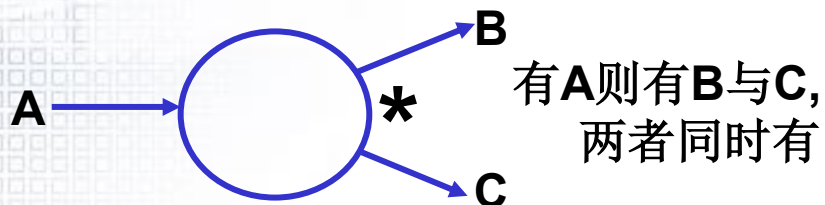
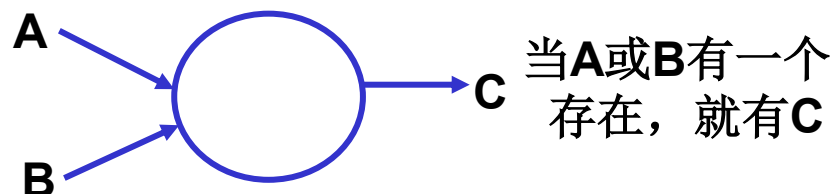
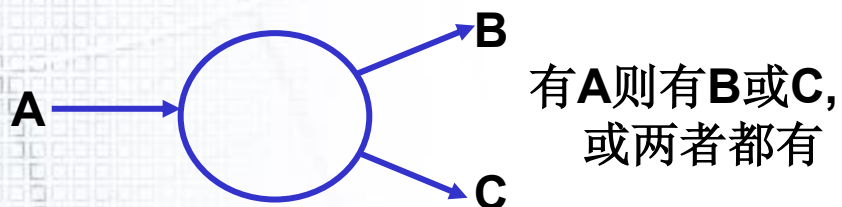
5.3 传统——数据流程图(6/12)

- 数据流：每个数据流用一组固定成分的数据组成并拥有一个定义明确的名字标识
 - 如：运动会管理系统中，报名单(数据流)由队名、姓名、性别、参赛项目等数据组成
- 数据流的流向 
 - 一个数据处理向另一个数据处理
 - 数据处理流向文件(写文件)
 - 文件流向数据处理(读文件)
 - 外部实体流向数据处理
 - 数据处理流向外部实体



5.3 传统——数据流程图(7/12)

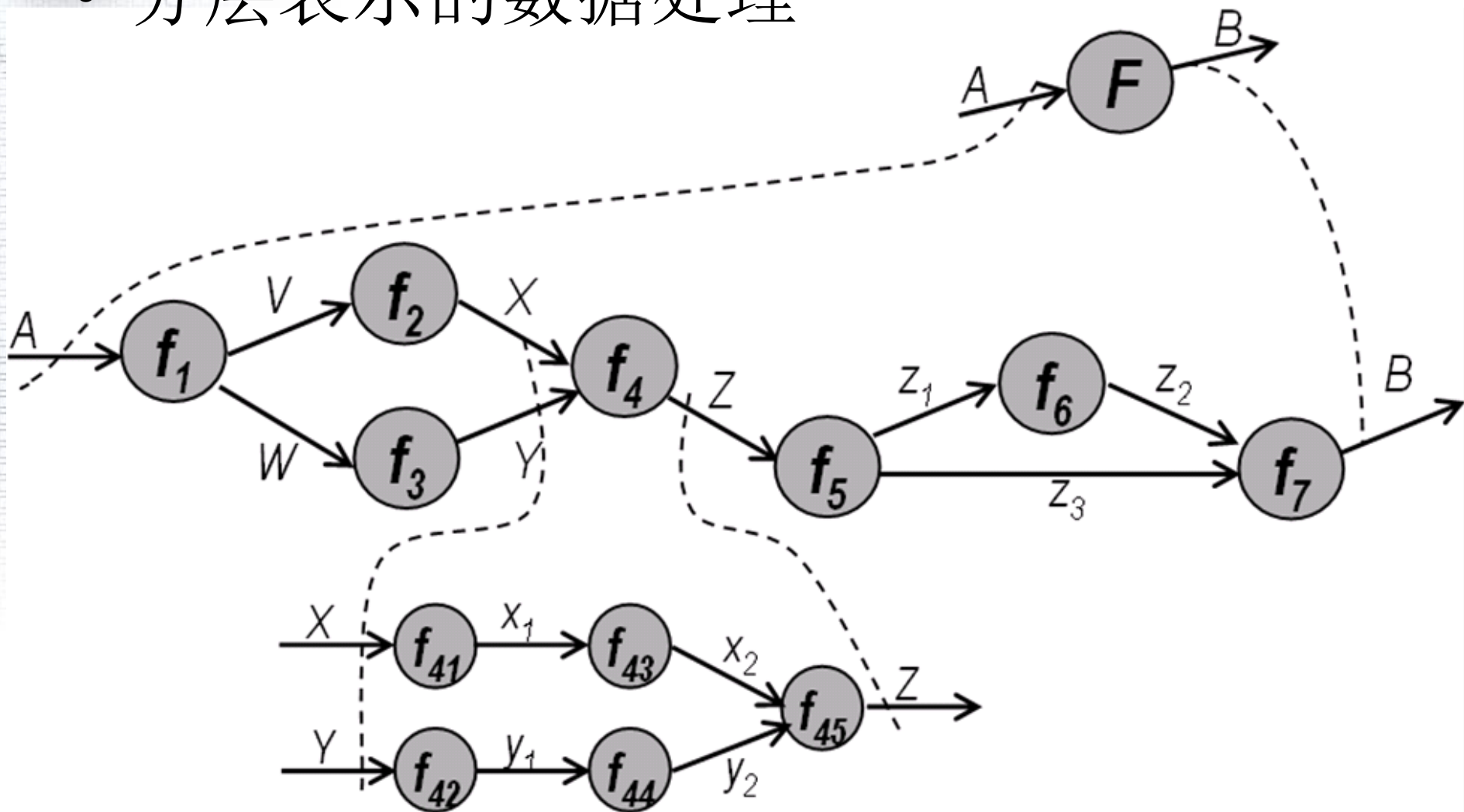
- 一个数据处理的多个数据流之间的关系





5.3 传统——数据流程图(8/12)

- 分层表示的数据处理





5.3 传统——数据流程图(9/12)

- 分层表示的数据处理

- 总体图:

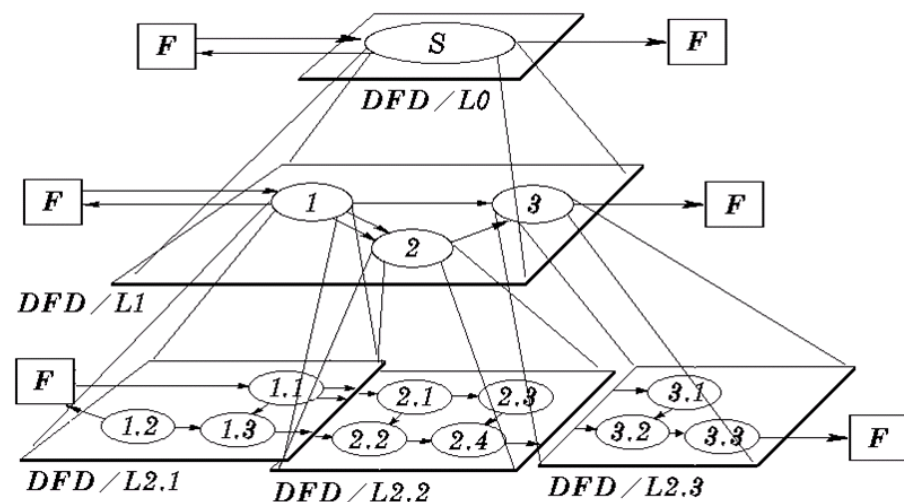
- 描述了系统和周围环境的关系

- 零级图:

- 表示一个系统的主要功能或者是一个大型系统的主要组成子系统

- 细节图:

- 表示一个复杂的处理的详细内部表示

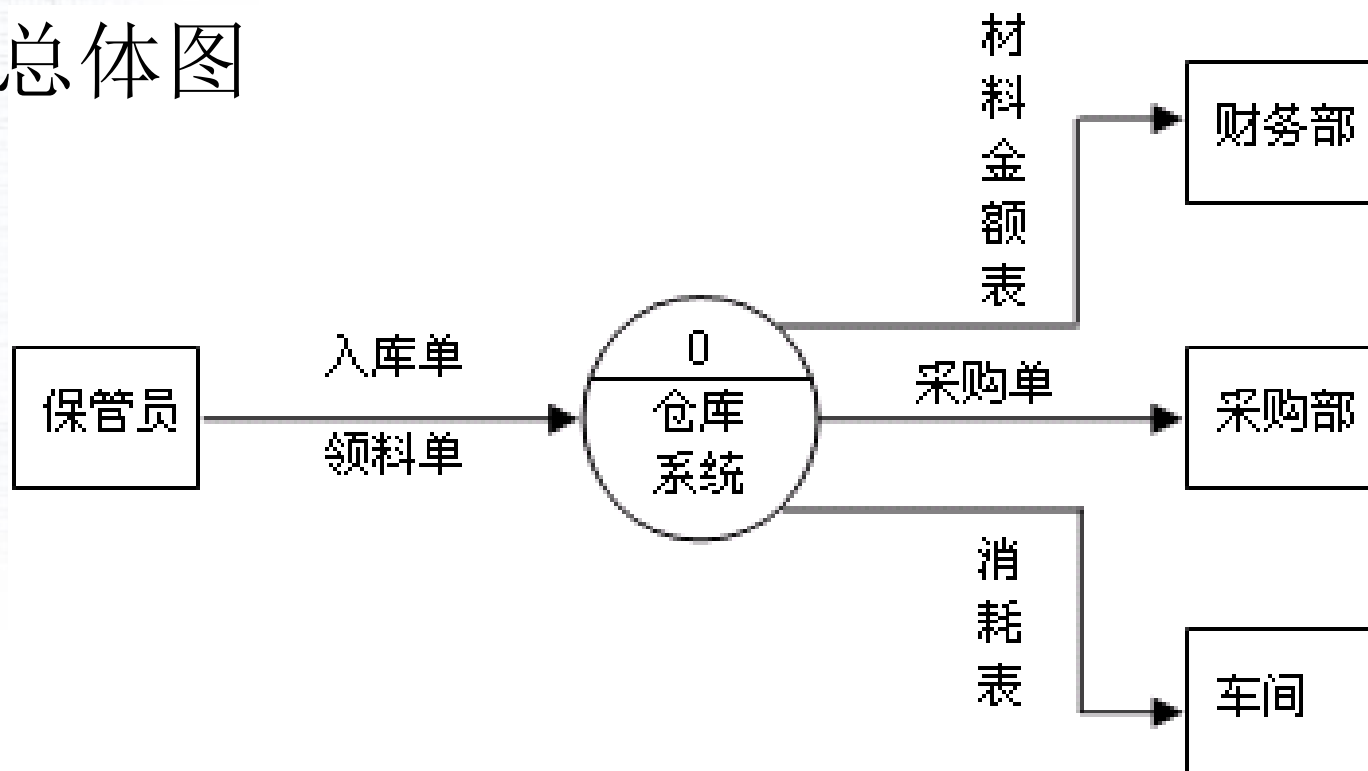


逐层分解的数据流程图



5.3 传统——数据流程图(10/12)

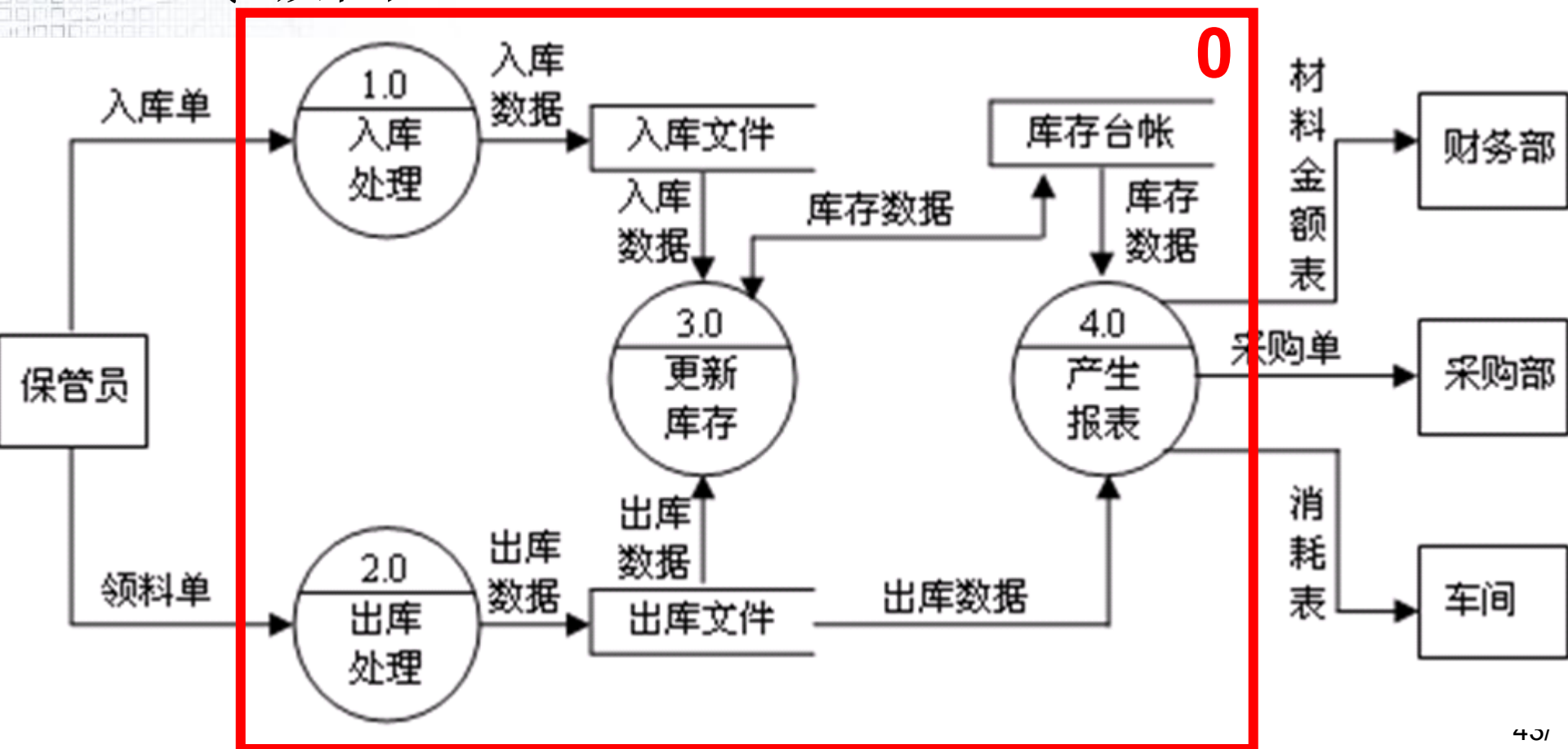
- 分层表示的数据处理
— 总体图





5.3 传统——数据流程图(11/12)

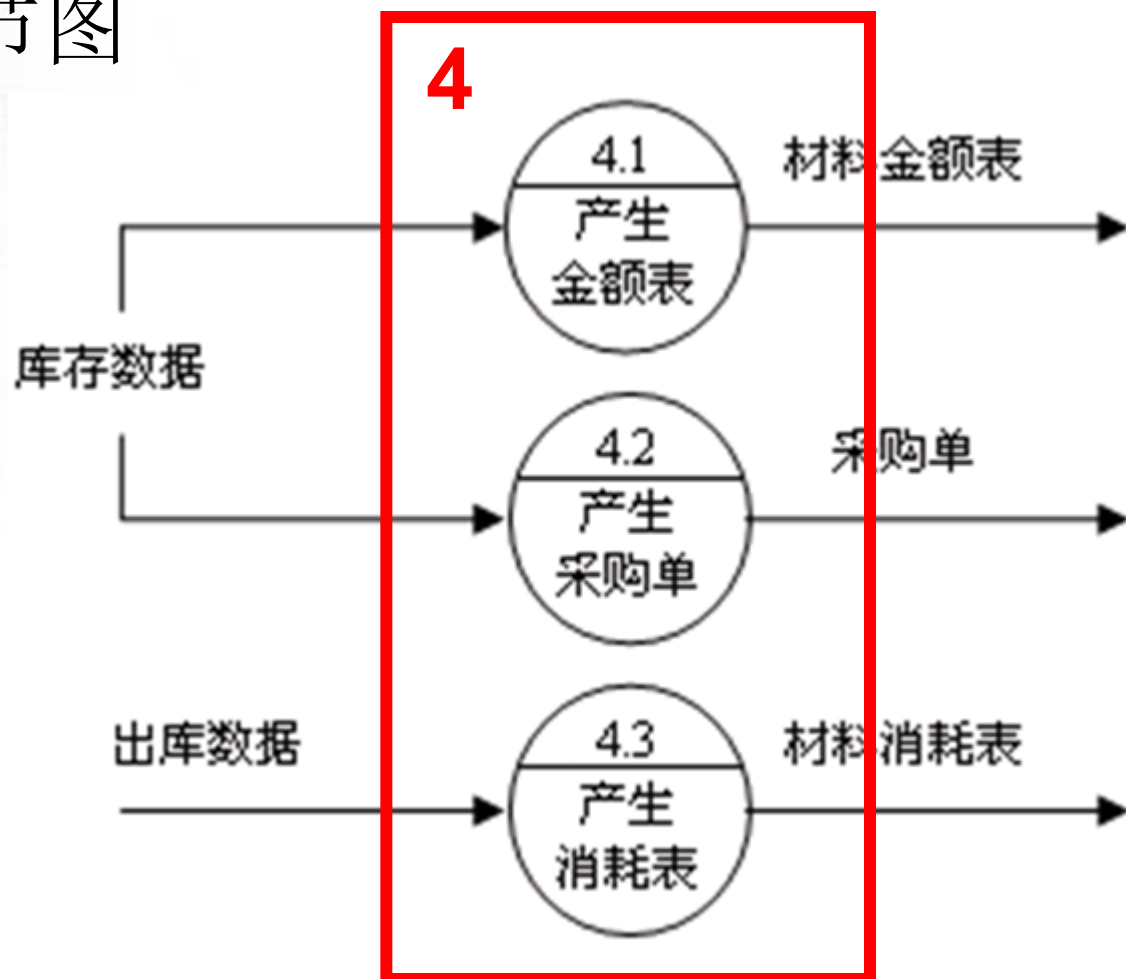
- 分层表示的数据处理
— 零级图





5.3 传统——数据流程图(12/12)

- 分层表示的数据处理
— 细节图





5.3 传统——数据流程图绘制规定(1/22)

- 数据流程图绘制的有关规定(1/2)
 - ① 外部实体只能出现在总体图和零级图中
 - ② 数据存储只能出现在零级图和细节图中
 - ③ 同一数据存储是分层的DFD图中只能出现一次
 - ④ 通常，数据存储必须既有读操作，也有写操作
 - ⑤ 数据流要有名字
 - ⑥ 数据流必须开始或结束在处理圆圈上，即数据不能直接在外实体与外实体间、外实体与数据存储间、数据存储与数据存储间流动
 - ⑦ 数据流不表示有关的控制逻辑



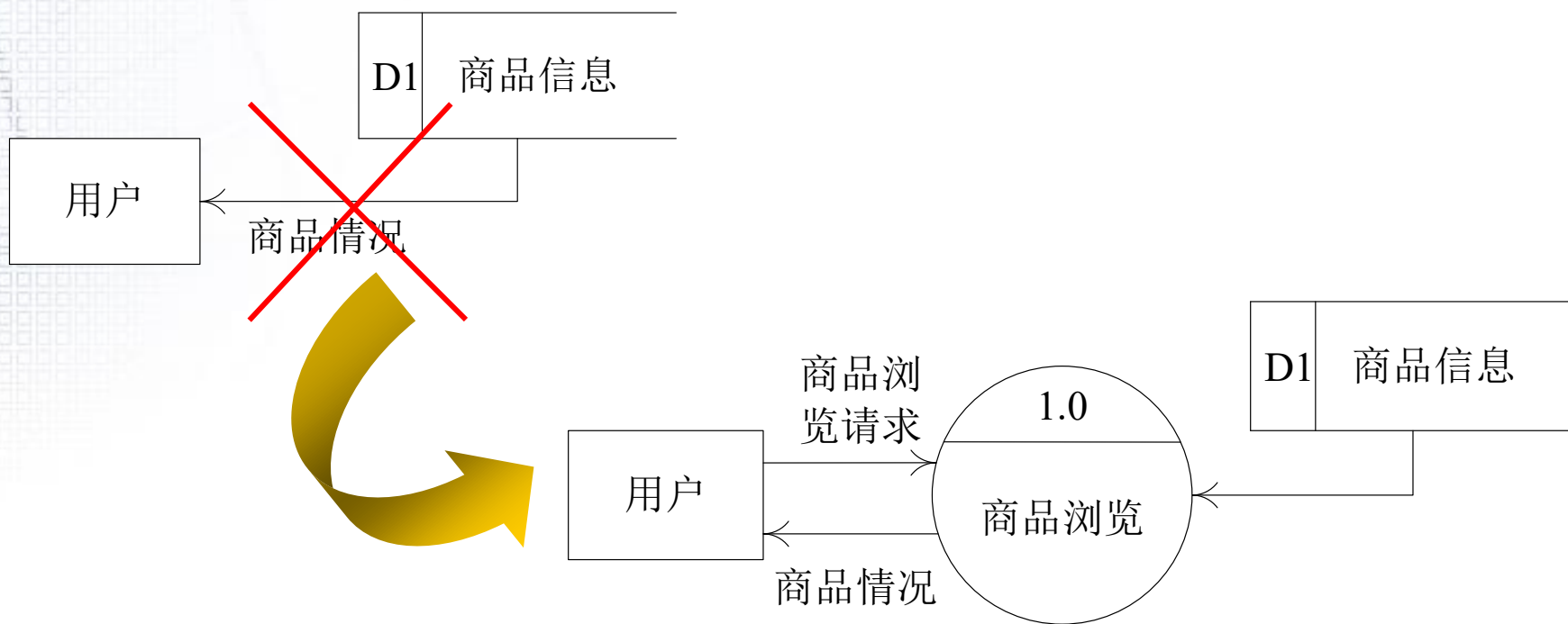
5.3 传统—数据流程图绘制规定(2/22)

- 数据流程图绘制的有关规定(2/2)
 - ⑧ 每个处理要有编号，但不表示先后顺序
 - ⑨ 每个图中处理的数通常不应超过9个
 - ⑩ 每个处理应该既有输入的数据流，也有输出的数据流
 - ⑪ 子图与父图中对应的处理必须执行相同的
功能，并且子图与对应的处理流入和流出的
数据流相同
 - ⑫ 输入/输出命令不能作为DFD图中的处理



5.3 传统—数据流程图绘制规定(3/22)

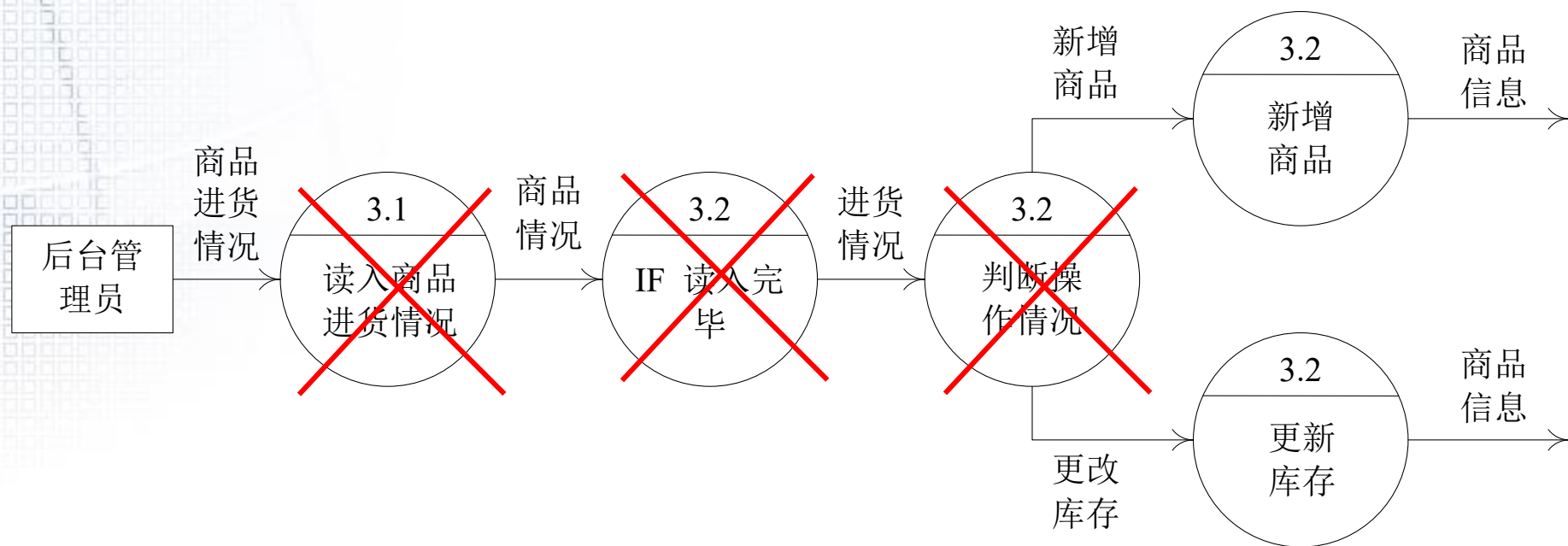
- 绘制规定——正确与否的规定
 - 示例1(每个数据流必须与处理关联)





5.3 传统—数据流程图绘制规定(4/22)

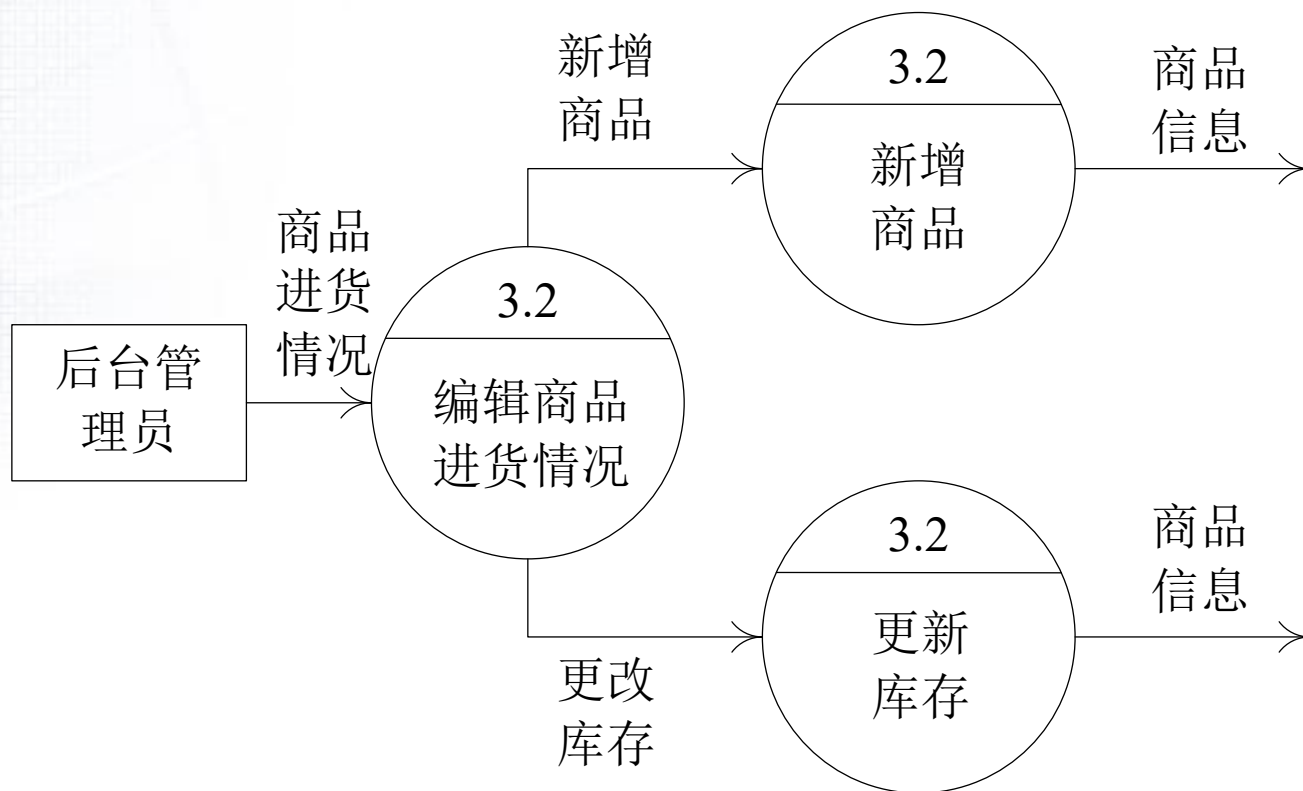
- 绘制规定——正确与否的规定
—示例2(控制逻辑和输入/输出)





5.3 传统—数据流程图绘制规定(5/22)

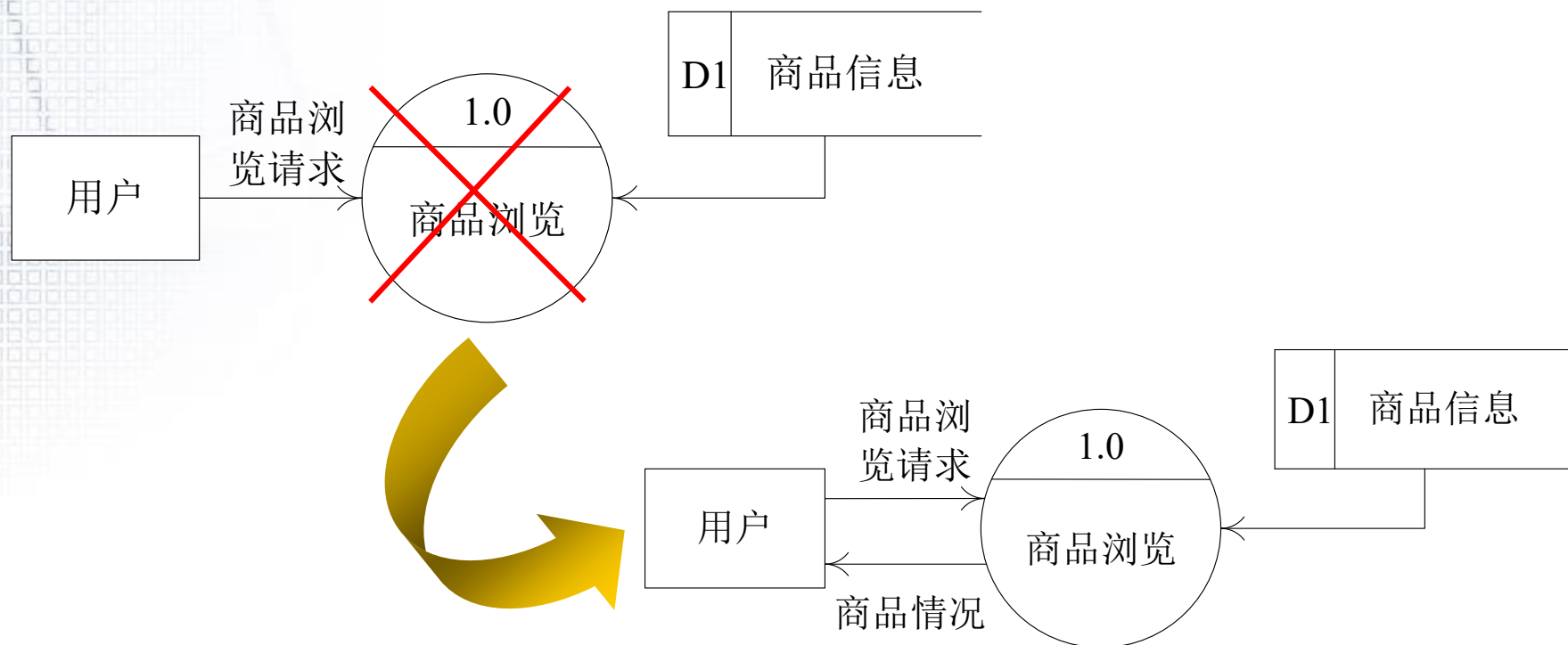
- 绘制规定——正确与否的规定
— 示例2(控制逻辑和输入/输出)





5.3 传统—数据流程图绘制规定(6/22)

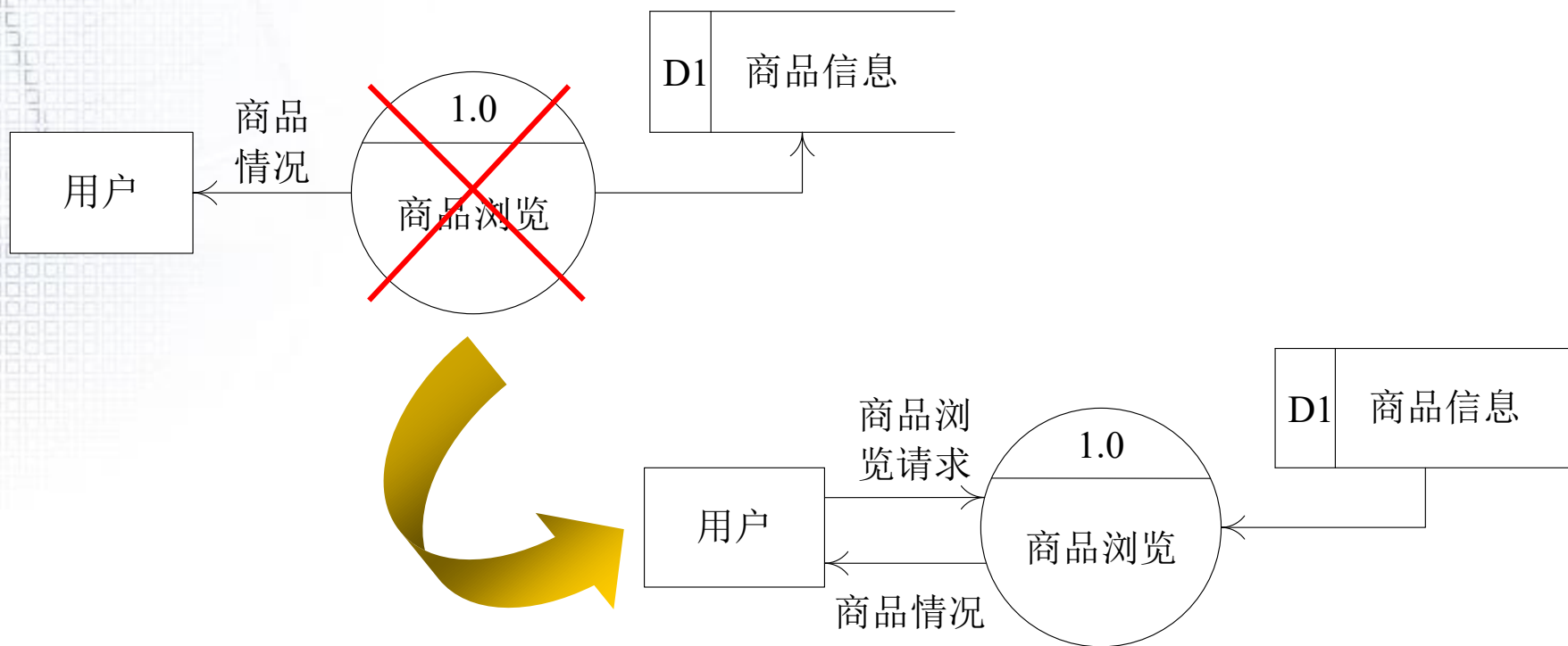
- 绘制规定——正确与否的规定
 - 示例3(黑洞：仅有数据流入而没有数据流出)





5.3 传统—数据流程图绘制规定(7/22)

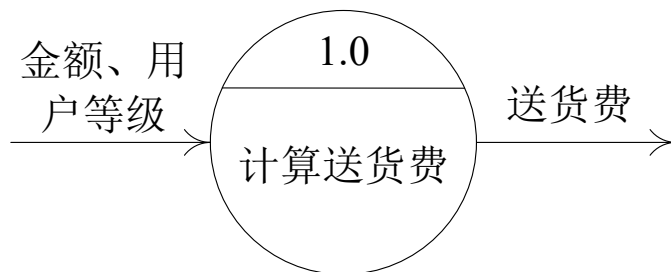
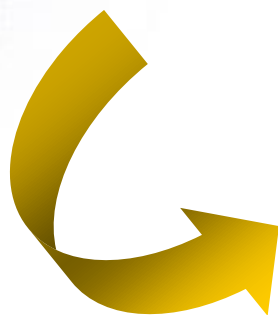
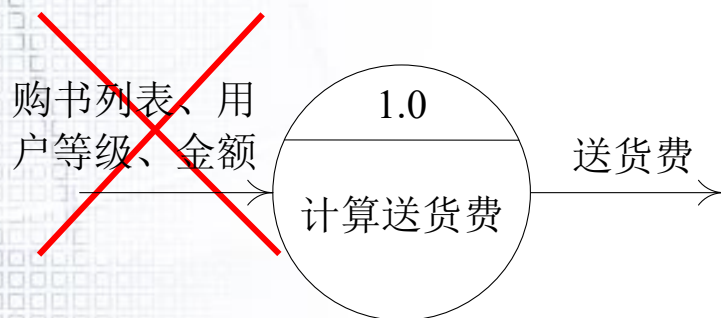
- 绘制规定——正确与否的规定
 - 示例4(奇迹：仅有数据流出而没有数据流入)





5.3 传统——数据流程图绘制规定(8/22)

- 绘制规定——正确与否的规定
 - 示例5(不必要的输入)



加工说明描述为:

If 金额 $\geq$ 80 then

 送货费=0

Else

 If 用户等级= “普通”

 送货费=5

 else

 送货费=0

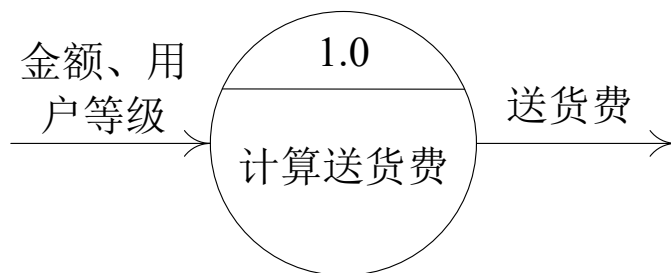
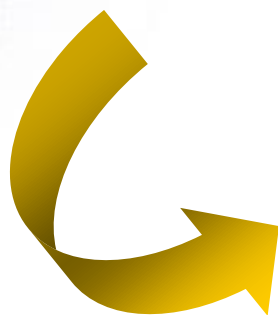
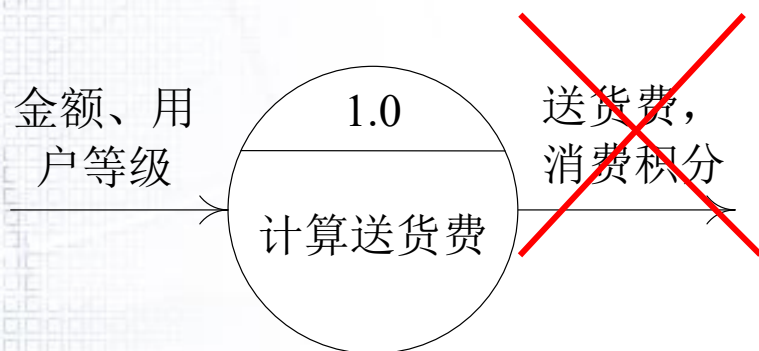
 endif

Endif



5.3 传统—数据流程图绘制规定(9/22)

- 绘制规定——正确与否的规定
 - 示例6(不可能的输出)



加工说明描述为:

If 金额 $\geq$ 80 then
 送货费=0

Else

 If 用户等级=“普通”
 送货费=5

 else

 送货费=0

 endif

Endif



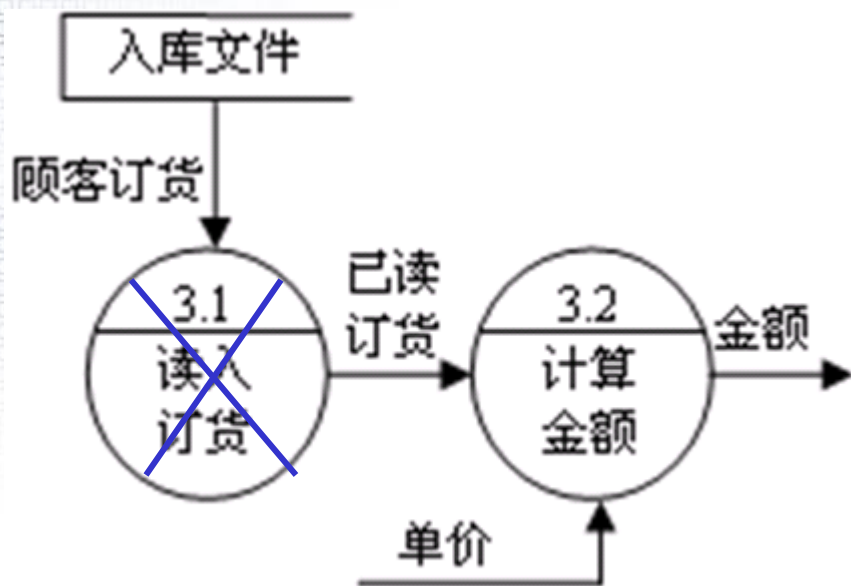
5.3 传统——数据流程图绘制规定(10/22)

- 绘制规定——正确与否的规定
 - 每个数据处理或数据存储都不应该存在不必要的输入、输入缺失、不可能的输出。
 - 数据流程图只是使用图形方式描述的模型，因此需要针对图中相关元素辅以必要的文字说明，即数据字典，此类问题仅通过数据流程图无法发现，只有辅以数据字典才能检查出来。

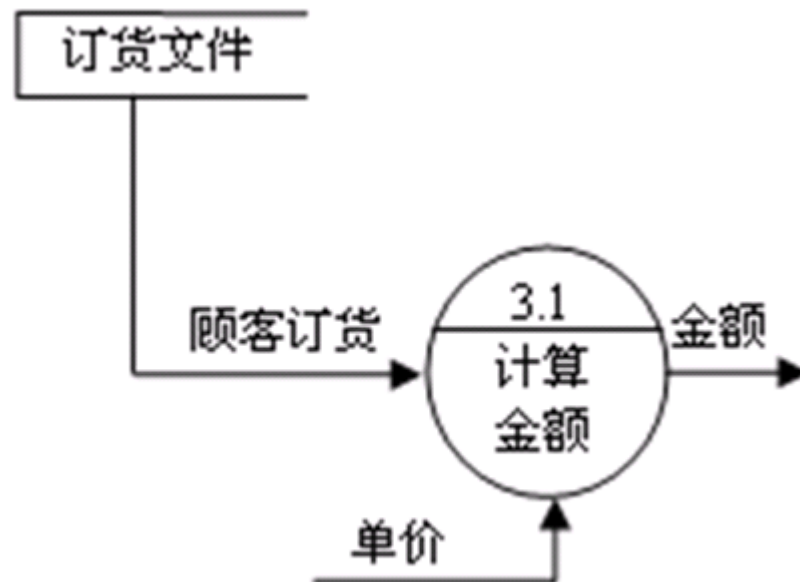


5.3 传统—数据流程图绘制规定(11/22)

- 规定举例1



(a) 错误的形式

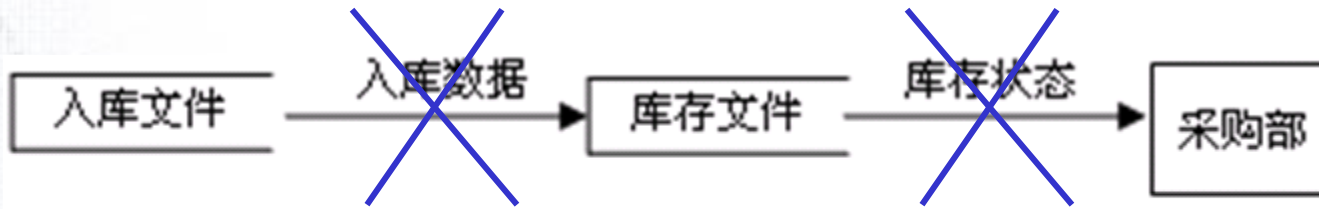


(b) 正确的形式

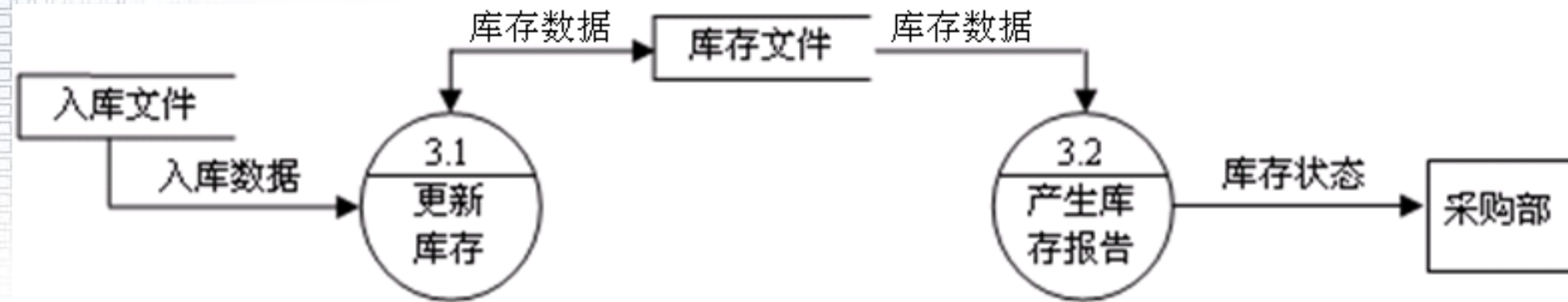


5.3 传统—数据流程图绘制规定(12/22)

• 规定举例2

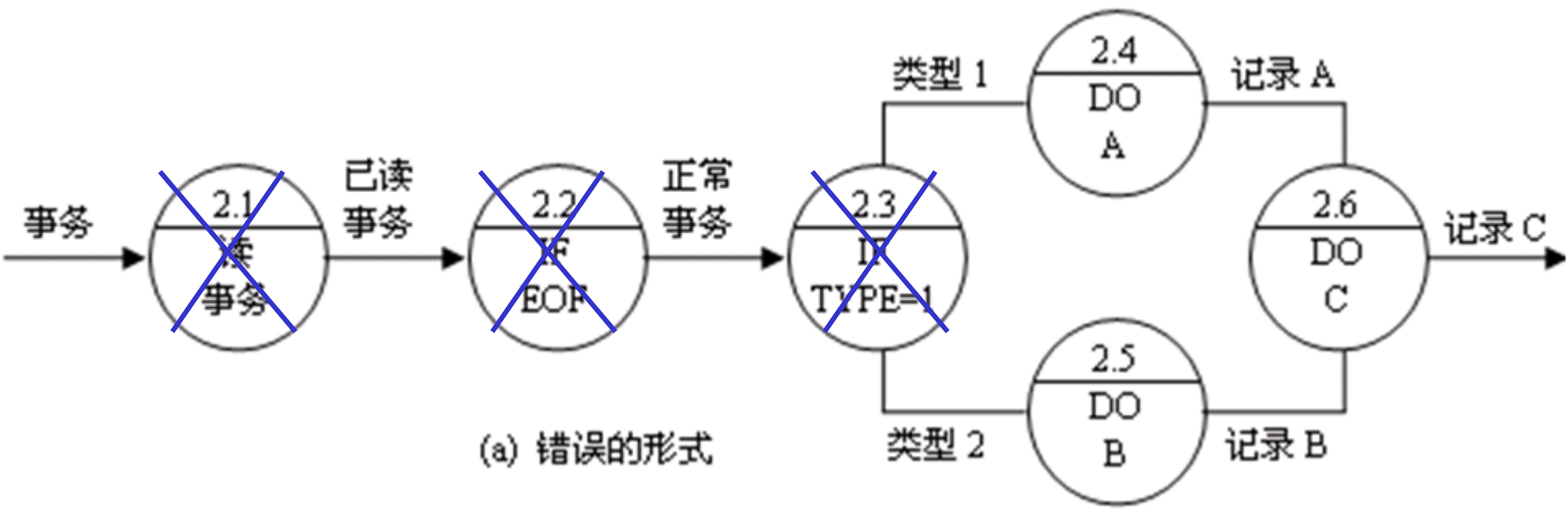


(a) 错误的形式

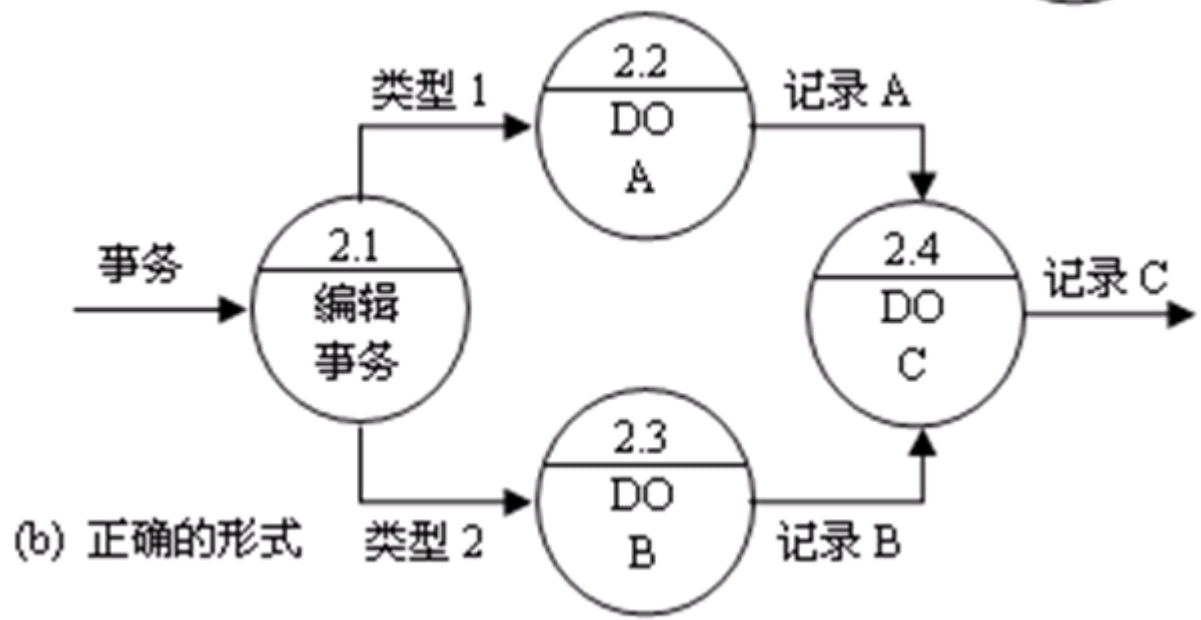


(b) 正确的形式

• 规定举例3



(a) 错误的形式

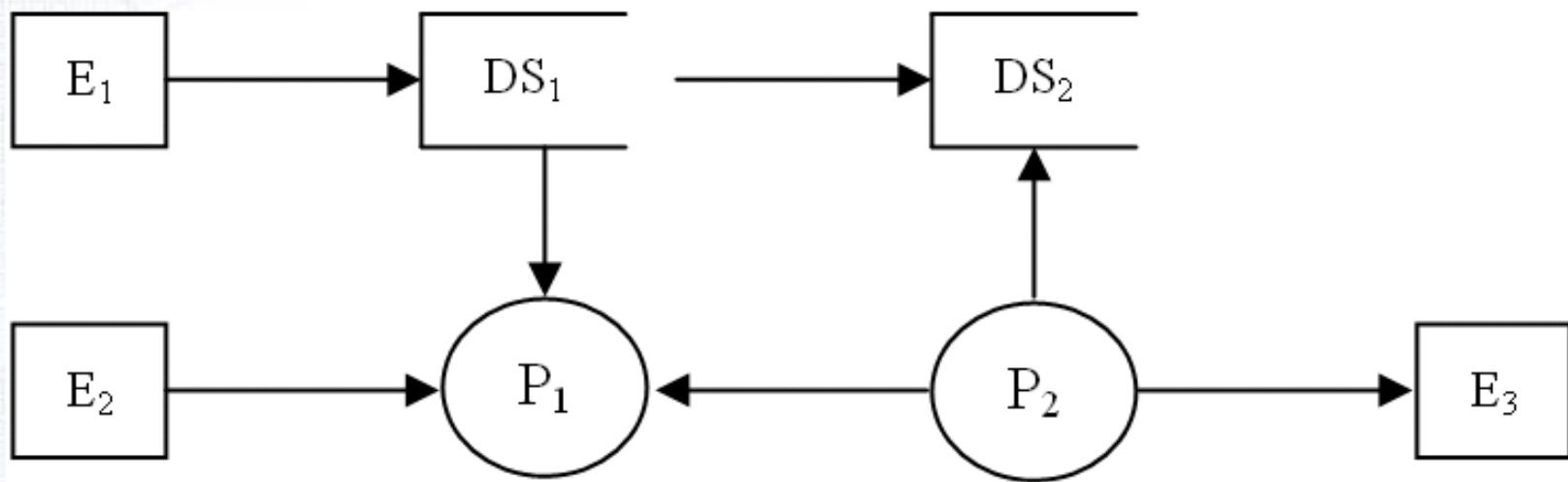


(b) 正确的形式



5.3 传统—数据流程图绘制规定(14/22)

• 规定举例4



① **E1->DS1**数据流未与处理相连

② **DS1->DS2**数据流无处理

③ 处理**P1**无输出数据流

④ 处理**P2**无输入数据流

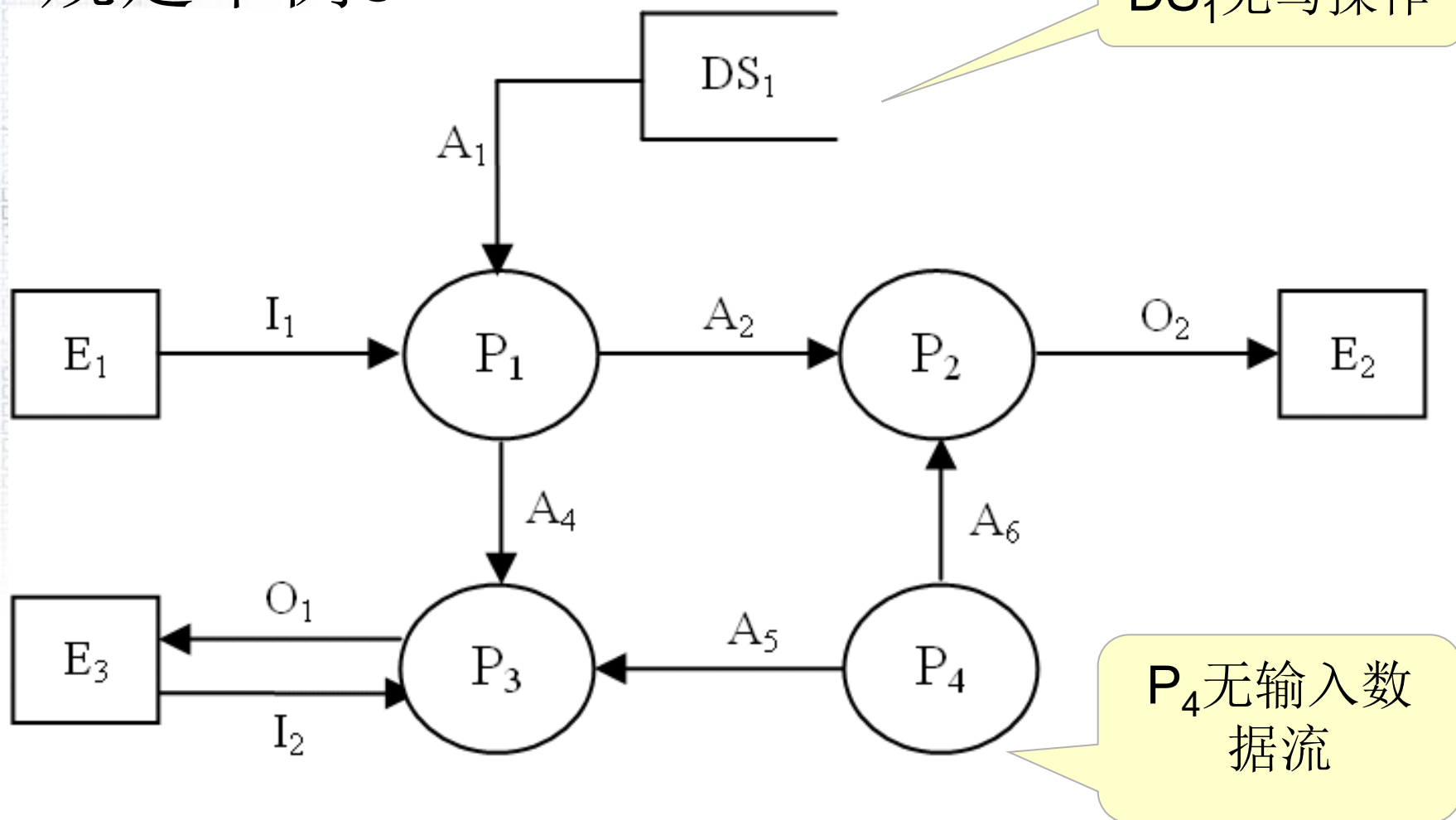
⑤ 数据存储**DS2**无读操作

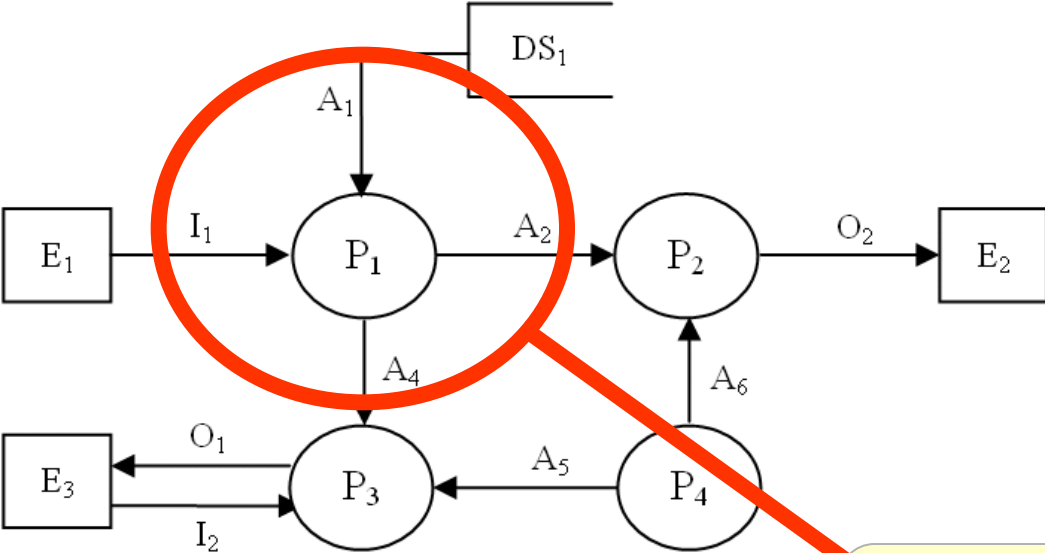
⑥ 所有数据流没有名字



5.3 传统—数据流程图绘制规定(15/22)

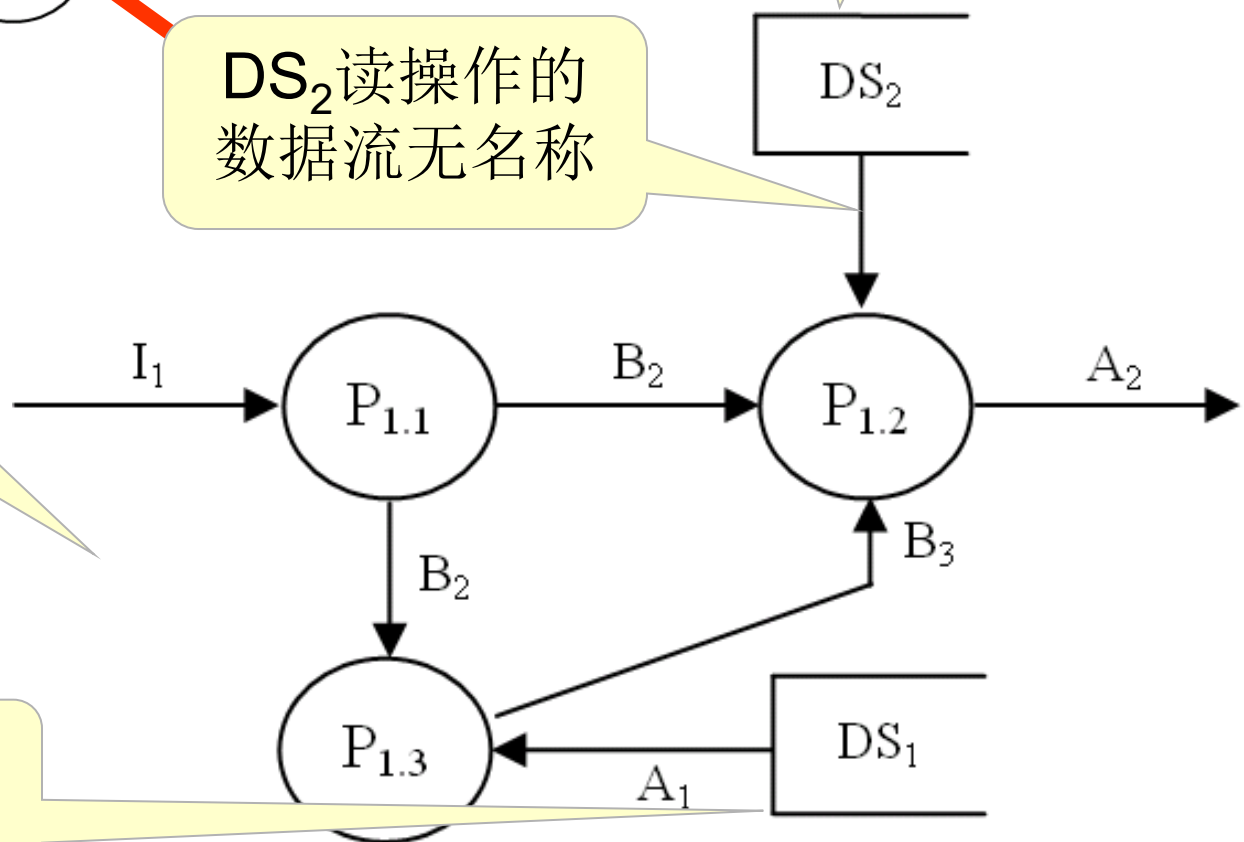
• 规定举例5





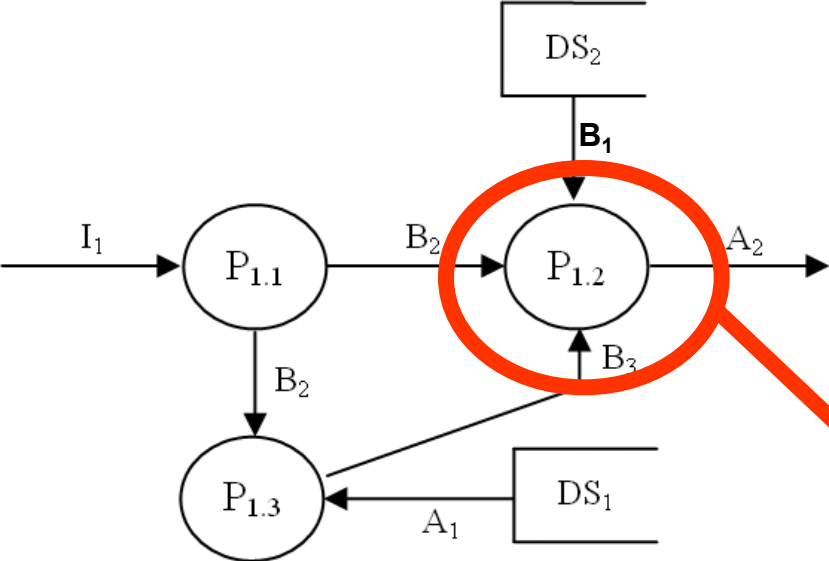
DS₂无写操作

DS₂读操作的数据流无名称

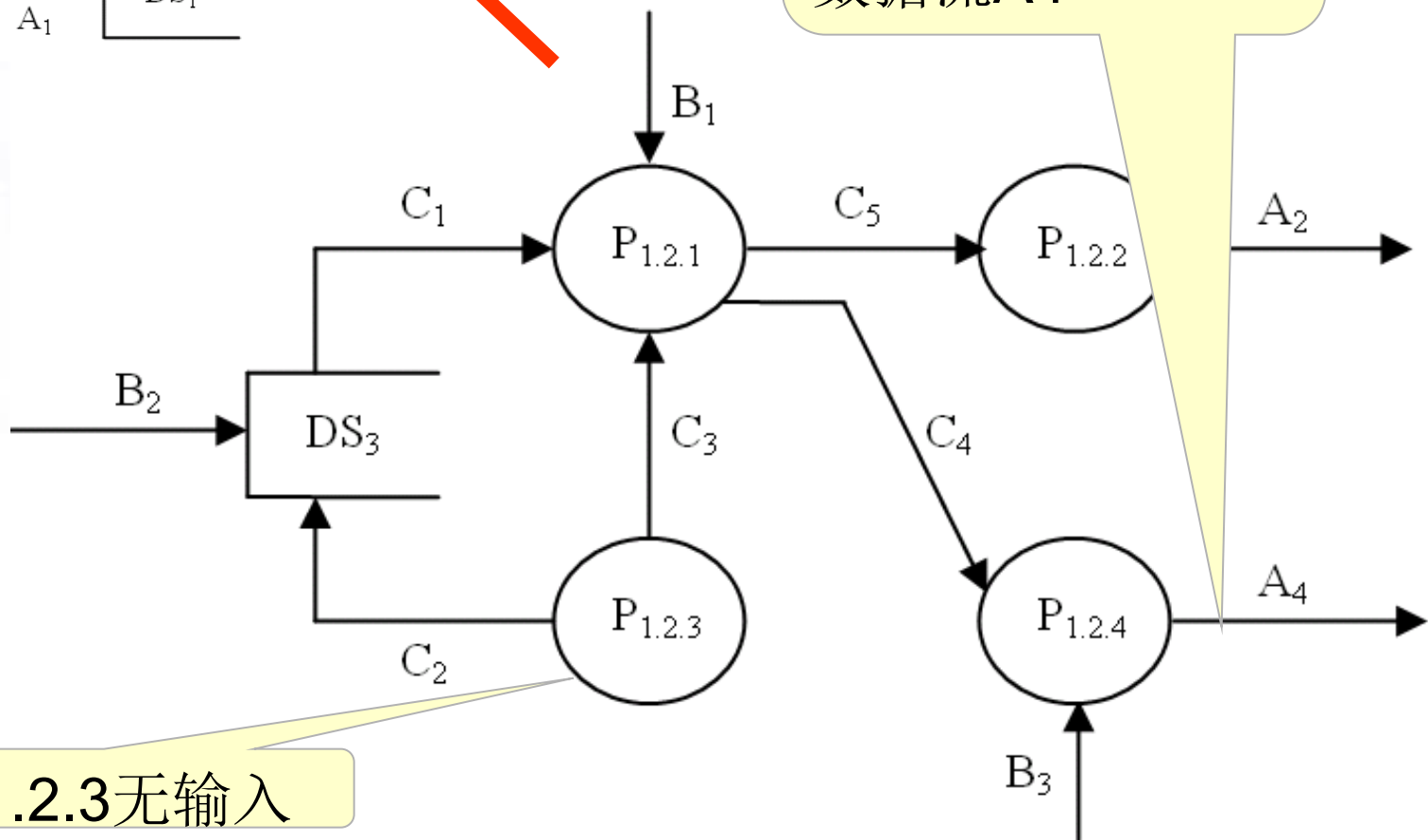


相对于零级图中的过程P₁，少数数据流A₄

DS₁与零级图中的数据存储重复



相对于其父图中过程P1.2,此处多了数据流A4



过程P1.2.3无输入



5.3 传统——数据流程图绘制规定(18/22)

- 绘制规定——正确与否的规定
 - 表示数据流的箭头上要标有数据流的名字，但流入或流出数据存储的数据流除外。
 - 数据流入数据存储表示写操作，是对数据的存储或更新（删除或修改）
 - 数据流出数据存储表示读操作，是对数据的检索或使用



5.3 传统——数据流程图绘制规定(19/22)

- 绘制规定——正确与否的规定
 - 分层的数据流程图各层之间应保持平衡
 - 每次细化时，子图与父图对应的数据处理所执行的功能必须是相同的，并且其输入数据流和输出数据流也要一致，同时流入和流出同一外部实体及同一数据存储的数据流也要一致。



5.3 传统——数据流程图绘制规定(20/22)

- 绘制规定——绘制质量的规定
 - 图中的每个数据处理都应有编号，但编号不表示执行的先后顺序。一般总体图中的数据处理编号为0，细节图和零级图的编号是相互关联的，处理2.2包含在处理2.0内，而处理2.2.1包含在处理2.2内。
 - 图中的每个数据存储都应有编号，一般使用“D+序号”的形式，如D1。



5.3 传统——数据流程图绘制规定(21/22)

- 绘制规定——绘制质量的规定
 - 为避免图中信息超量，降低图的复杂度，便于理解，图的绘制尽量遵守 7 ± 2 规则。每层单个图中的数据处理个数不超过9个；保持接口最小化，图中某个元素与其它元素之间的连接不超过9个，具体为每层每个图中不应有超过9个数据流流入或流出每个数据处理、数据存储或外部实体。



5.3 传统——数据流程图绘制规定(22/22)

- 绘制规定——绘制质量的规定

- 为保持图的整洁，避免数据流线的相互交叉，增加图的可读性，同一个数据处理、数据存储或外部实体可以在图中不同位置重复出现，但其编号和名称应完全相同。

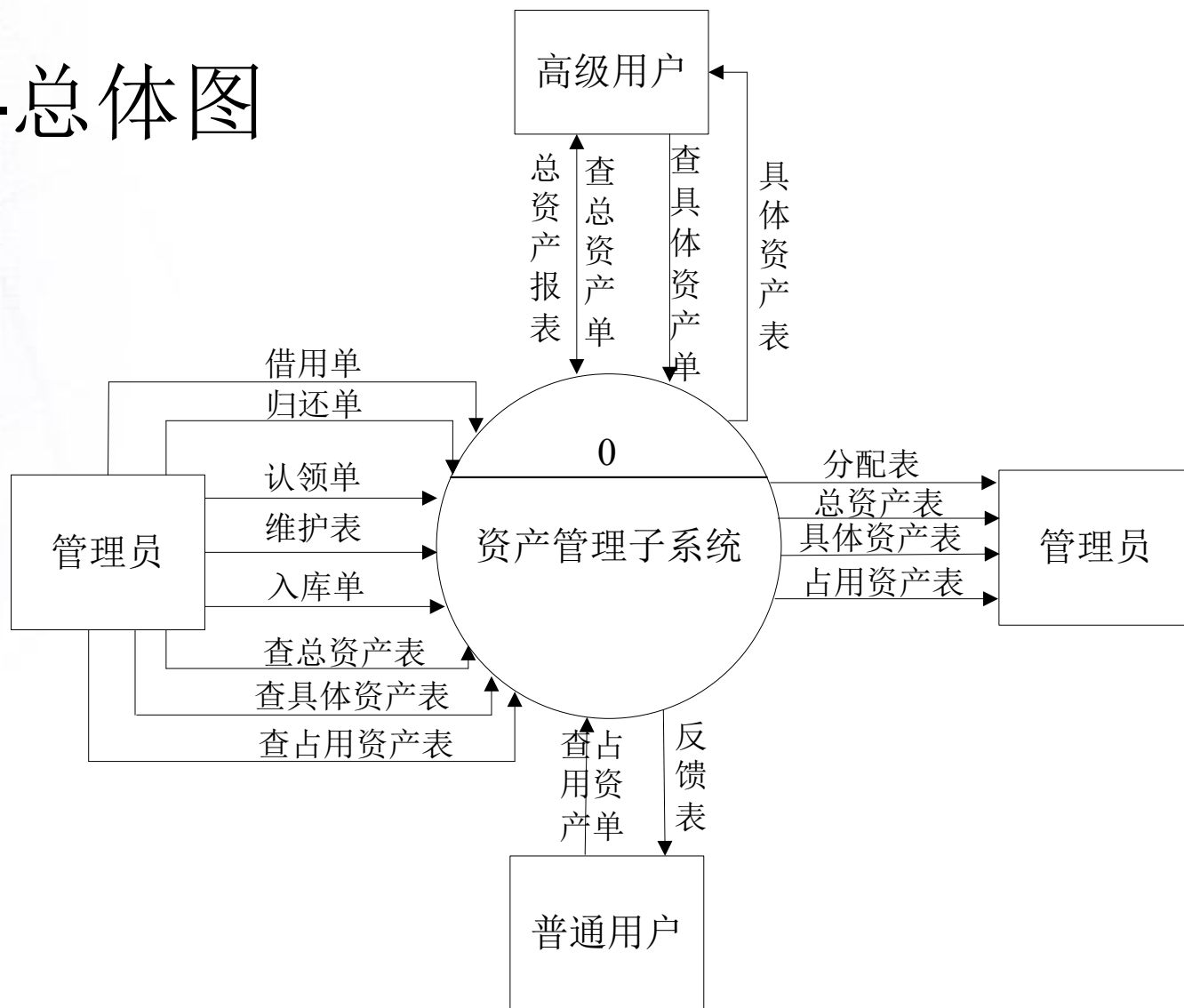
- 例：此[零级图](#)，出现两个订单管理的数据处理，两个订单信息的数据存储以及两个用户信息的数据存储。

- 一个数据处理的输出不应等同于其输入，因为数据处理的功能是对数据进行变换。

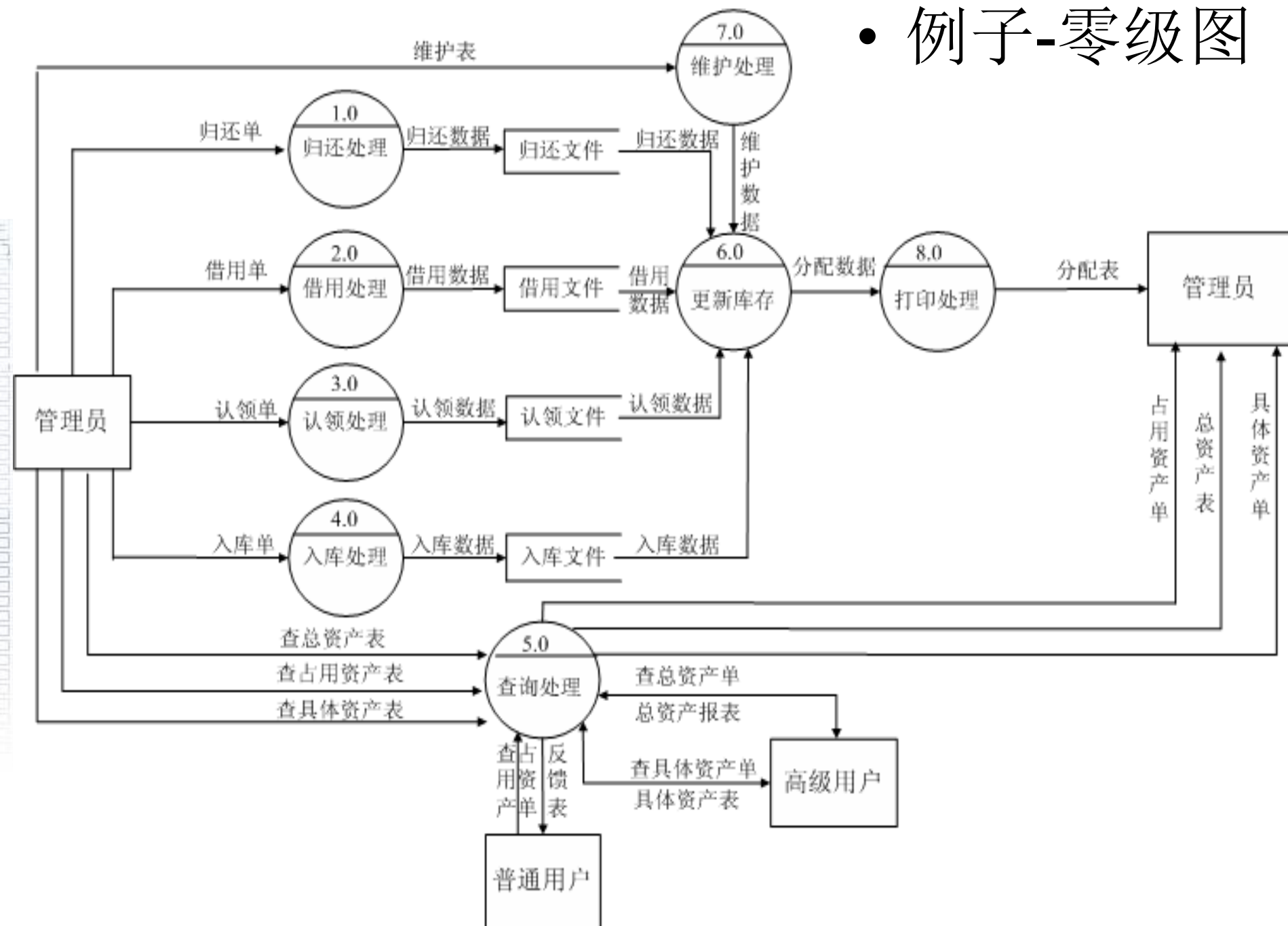


5.3 传统——数据流程图示例(1/9)

• 例子-总体图



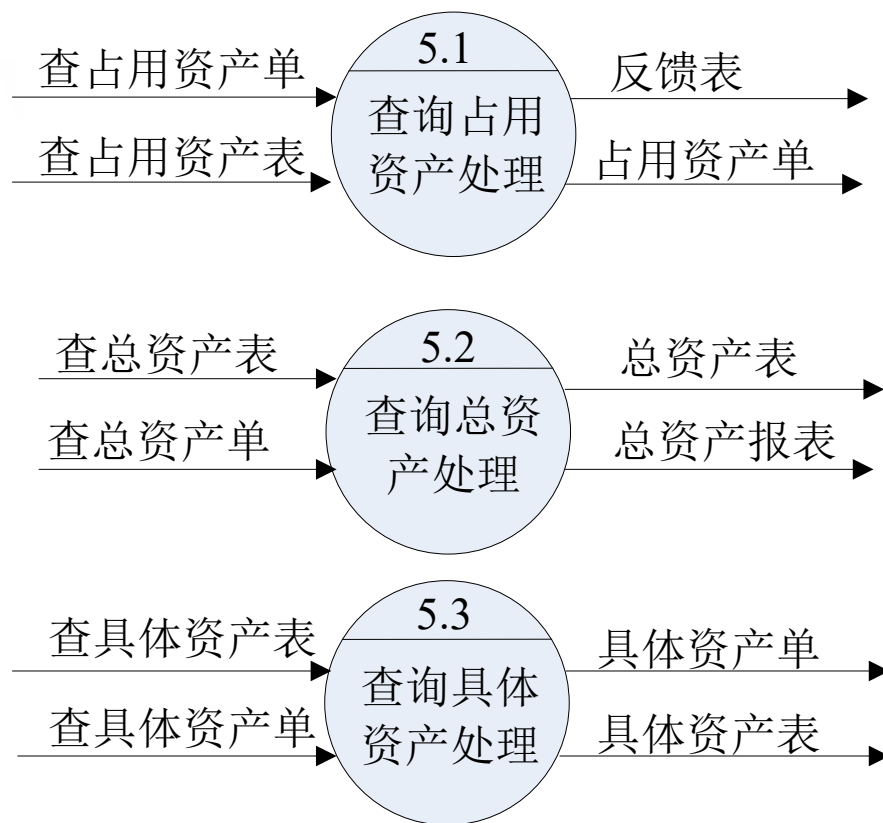
• 例子-零级图





5.3 传统——数据流程图示例(3/9)

• 例子-细节图





5.3 传统——数据流程图示例(4/9)

- 数据流程图的画法
 - 第一步：确定外部实体；
 - 第二步：确定数据处理；
 - 第三步：确定数据存储；
 - 第四步：画**DFD**：补充数据流
 - 注意绘制要求



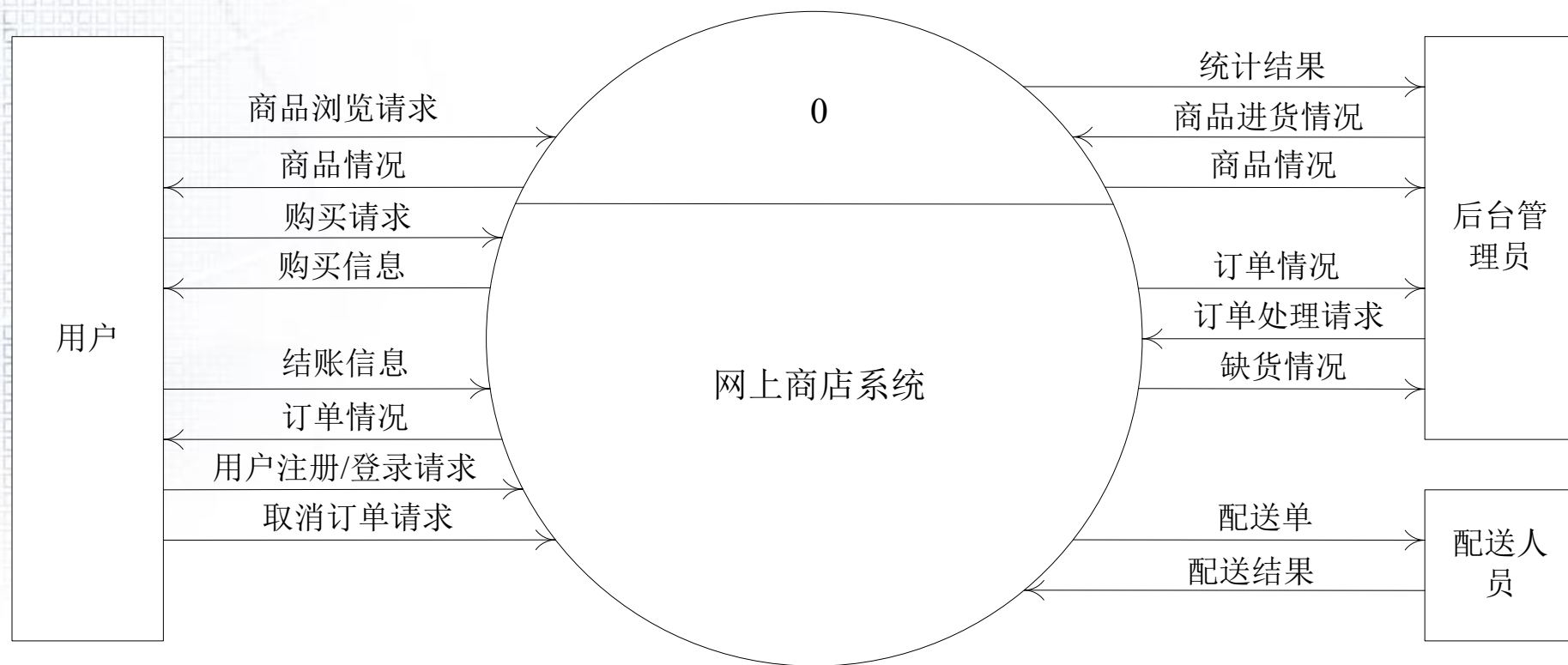
5.3 传统——数据流程图示例(5/9)

- 请根据网上商店系统功能，分析并画出该系统的数据流程图
- 功能需求：
 - **用户**可以注册、登录系统、浏览商品、添加购物车(购买商品)、商品结账、取消订单
 - **管理员**可以进行信息统计、进货、订单管理、缺货登记
 - **配送人员**可以根据订单实现配送



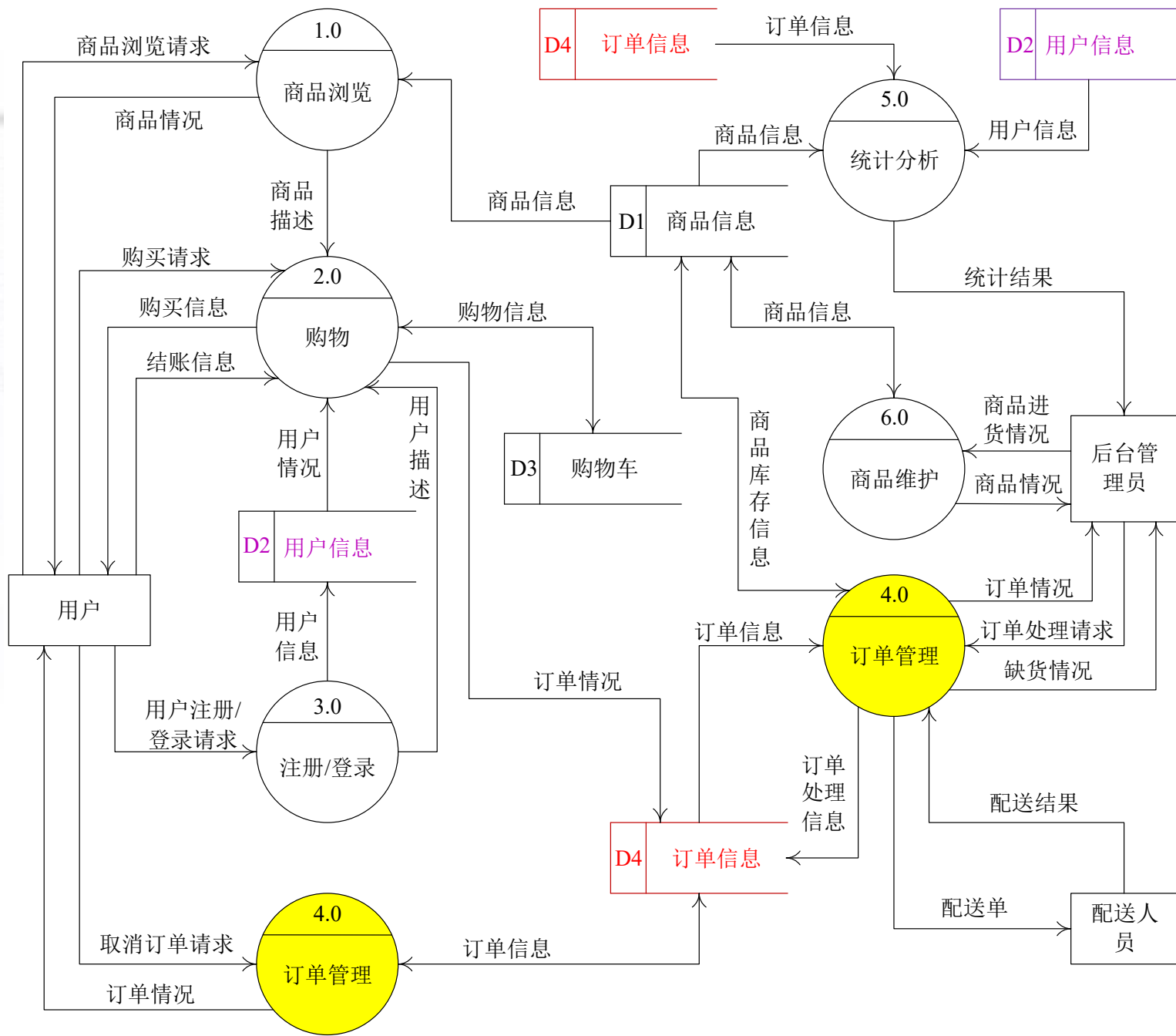
5.3 传统——数据流程图示例(6/9)

• 例子-总体图





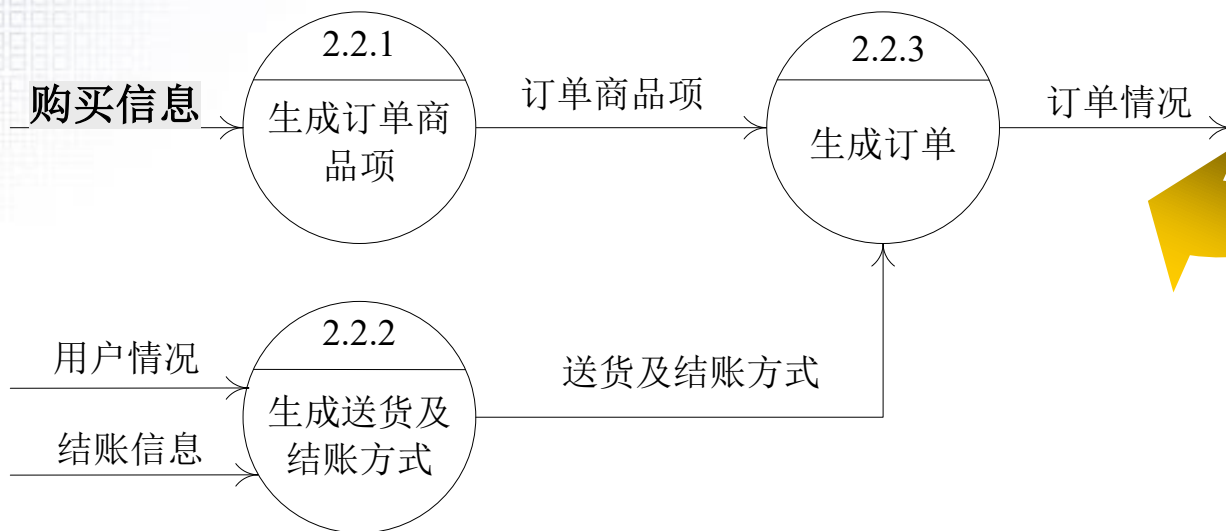
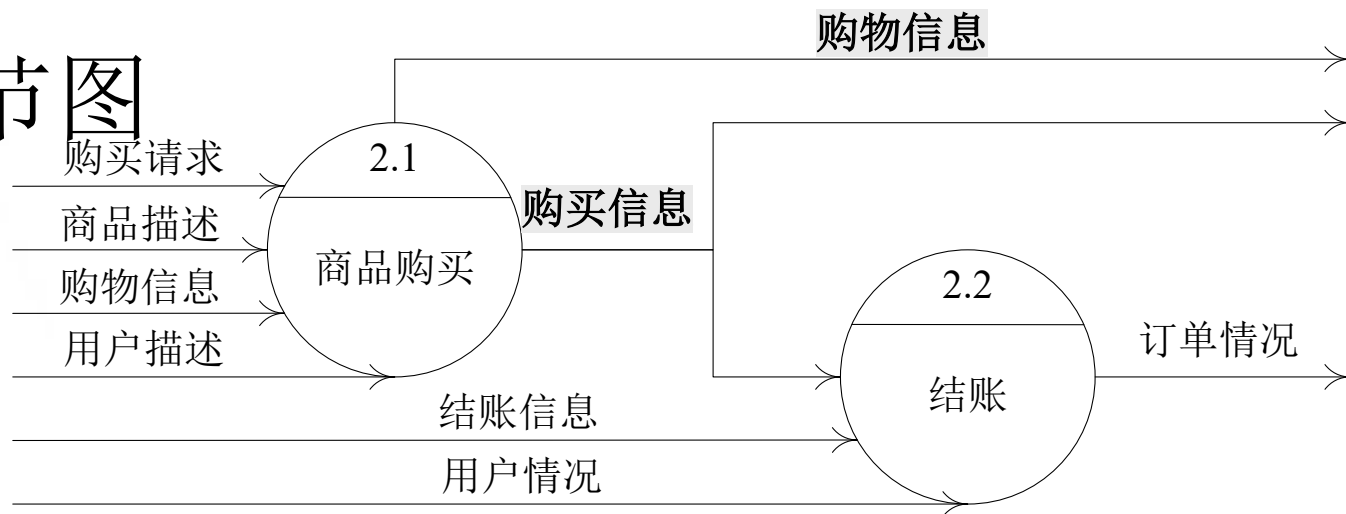
•例子- 零级图





5.3 传统——数据流程图示例(8/9)

• 例子-细节图



进一步细化



5.3 传统—数据流程图示例(9/9)

- 练习画DFD



5.3 传统需求分析建模方法

- 系统静态特征：数据流程图
- 系统动态特征？(并发、迁移...)
- 行为建模
 - 状态变迁图
 - Petri-net



5.3 传统——状态变迁图(1/7)

- 定义：是一种通过描述系统状态以及导致改变状态的事件来表示系统行为的图形手段

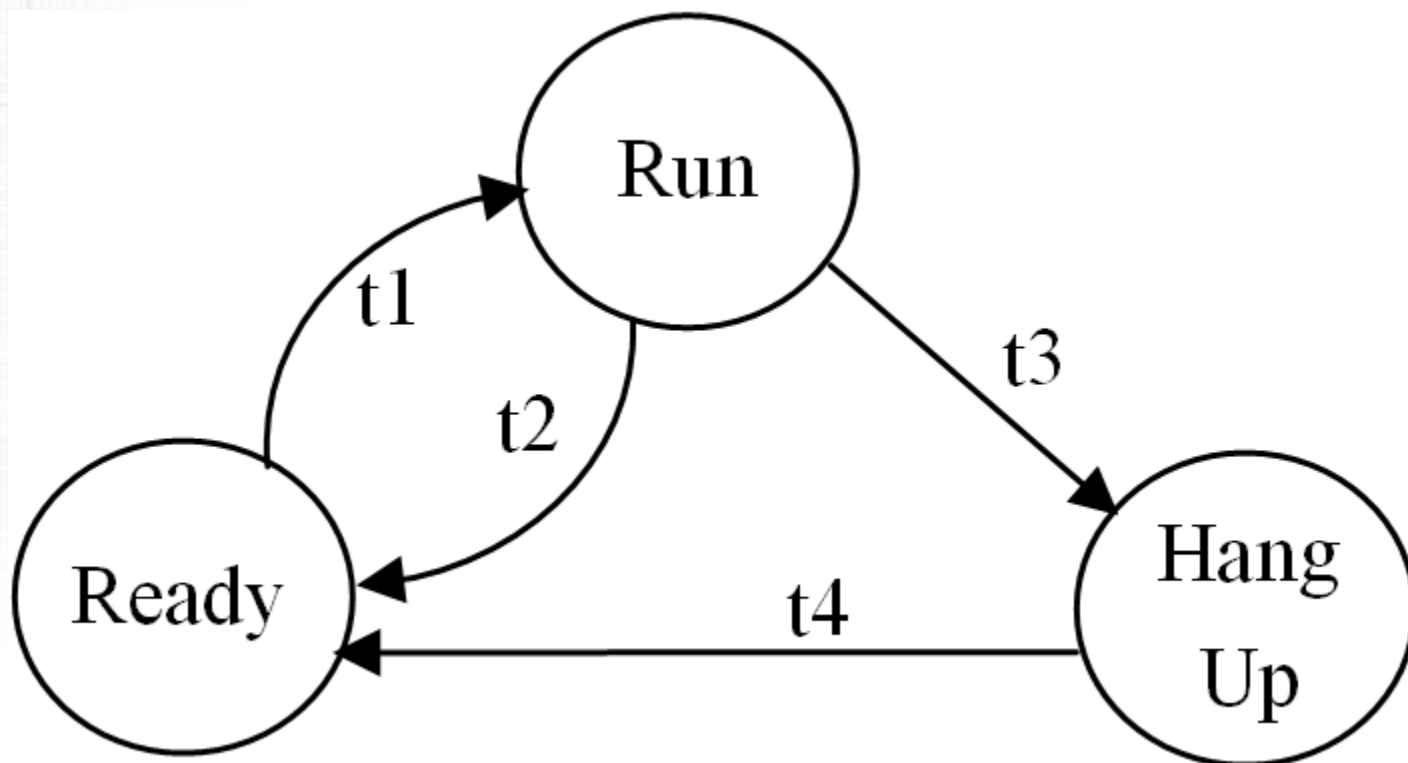
STD: State-Transition Diagram

- 表示符号
 - 状态：
 - 是可以被观察到的系统的行为模式
 - 圆圈或矩形表示，并在圆圈或矩形中标明状态的名字
 - 变迁
 - 表示一种状态向另一种状态的迁移
 - 带箭头的线来表示，在线上要标注出事件的名称，需要时也可以和把DFD图中相关的处理标注进来



5.3 传统——状态变迁图(2/7)

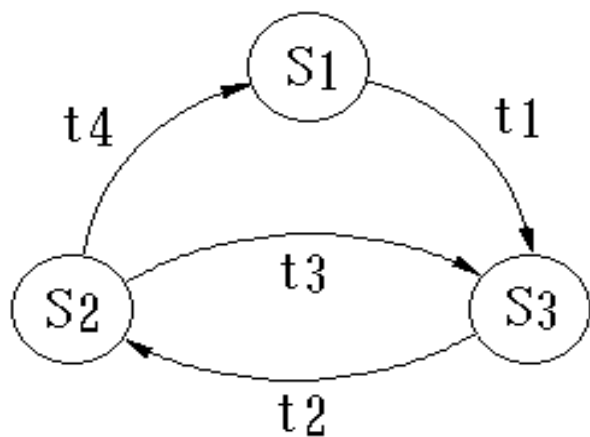
- 例子1：进程





5.3 传统——状态变迁图(2/7)

- 状态变迁图与状态迁移表



(a) 状态迁移图

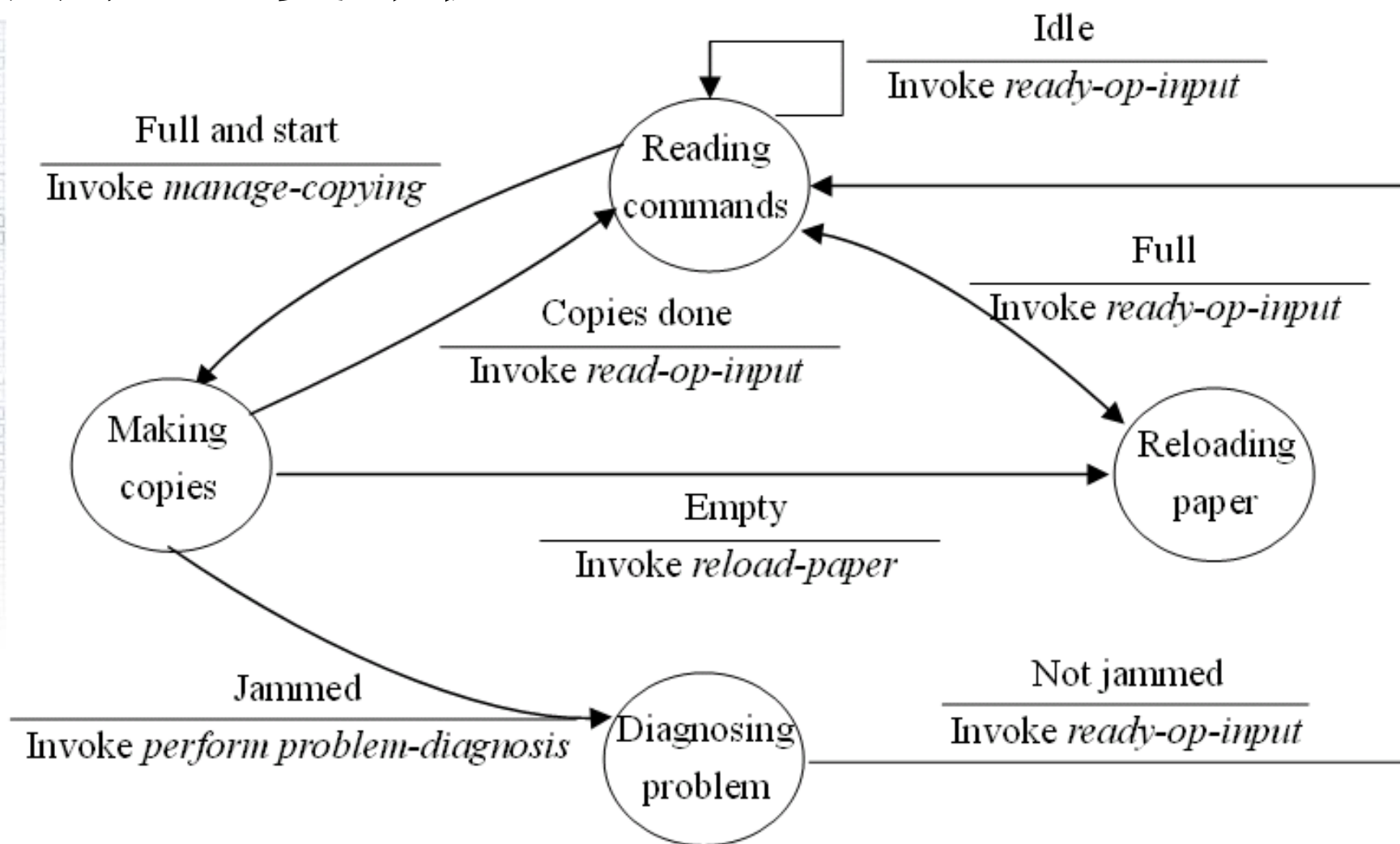
状态 事件	S1	S2	S3
t1	S3		
t2			S2
t3		S3	
t4		S1	

(b) 状态迁移表



5.3 传统——状态变迁图(3/7)

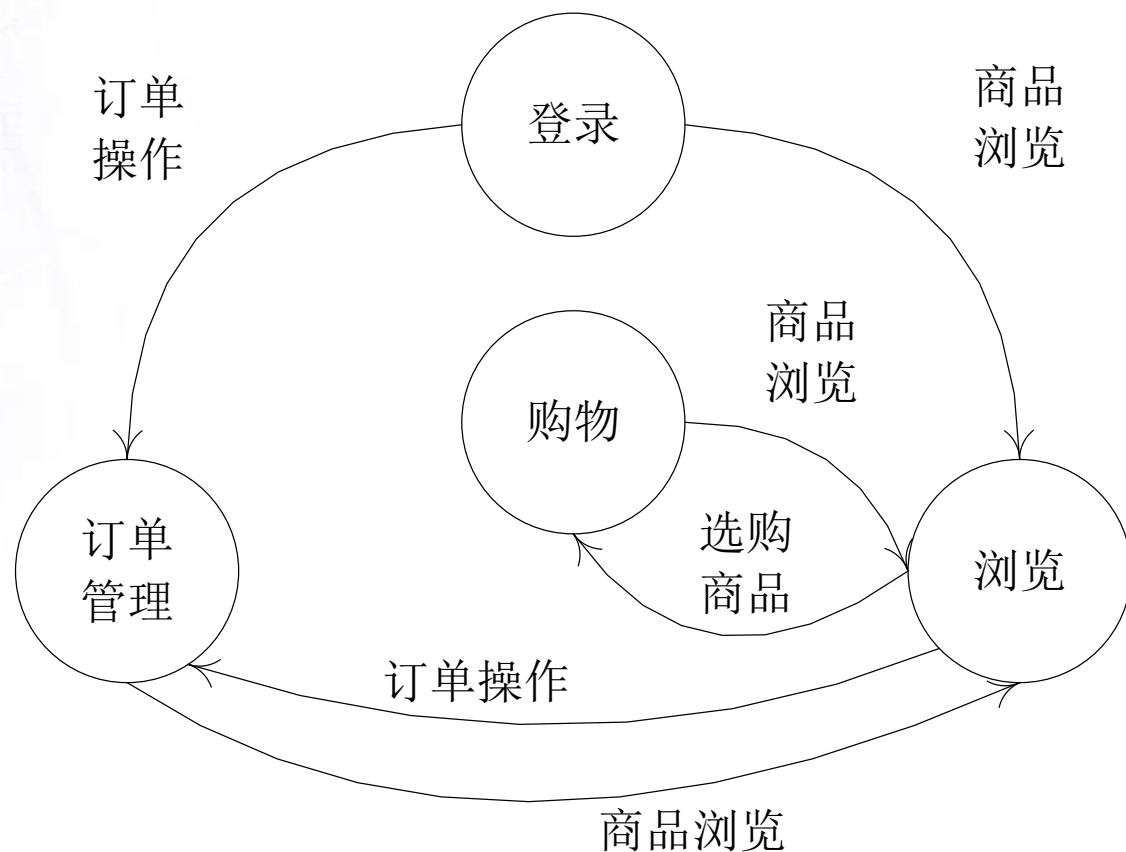
• 例子2：复印机





5.3 传统——状态变迁图(4/7)

- 例子3：网上宠物商店前台状态变迁图





5.3 传统——状态变迁图(6/7)

- 例子3：网上宠物商店前台状态变迁表

事件 \ 状态	登录	浏览	购物	订单管理
订单操作	订单管理	订单管理		
商品浏览	浏览		浏览	浏览
选购商品		购物		



5.3 传统——状态变迁图(7/7)

- STD练习

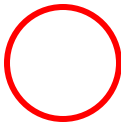


5.3 传统——Petri网(1/4)

- 表示符号

- 位置（**P**lace）节点

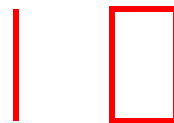
- 表示系统的状态

- 用圆形符号表示 

- 跃迁（**T**ransition）节点

- 表示系统中的事件

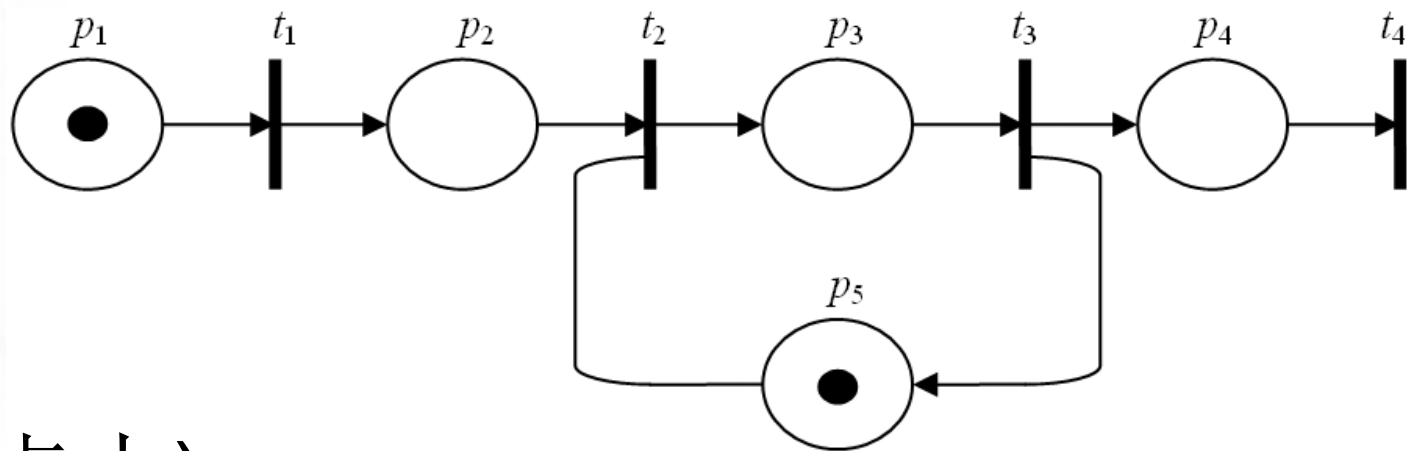
- 用短粗线或矩形表示



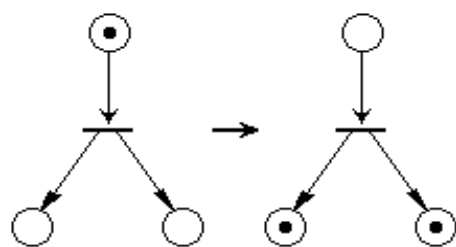


5.3 传统——Petri网(2/4)

- 例子



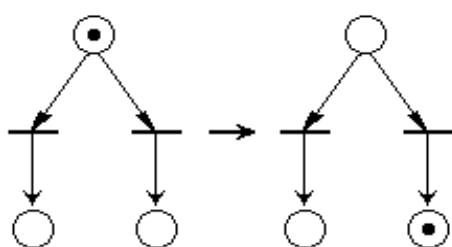
- 激发(点火)



激发前

激发后

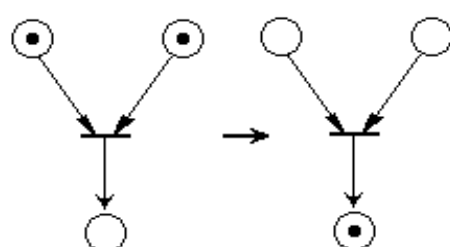
(a)



激发前

激发后

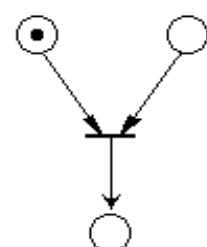
(b)



激发前

激发后

(c)



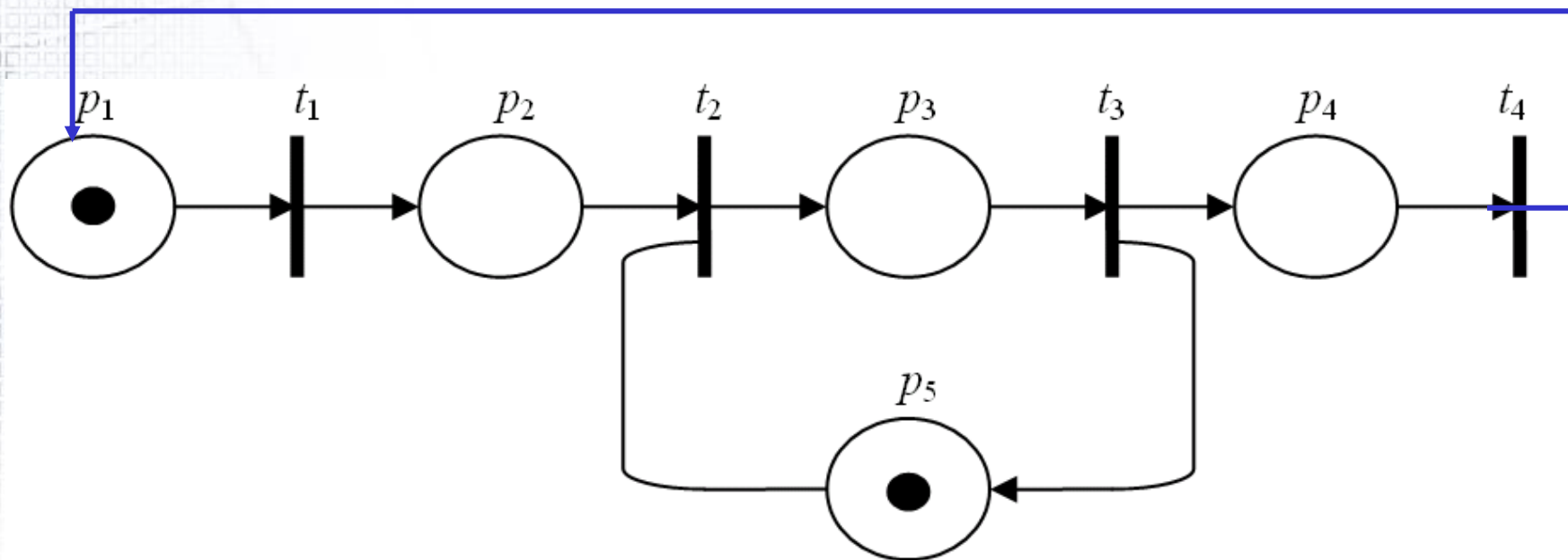
不能激发

(d)



5.3 传统——Petri网(3/4)

- 例子

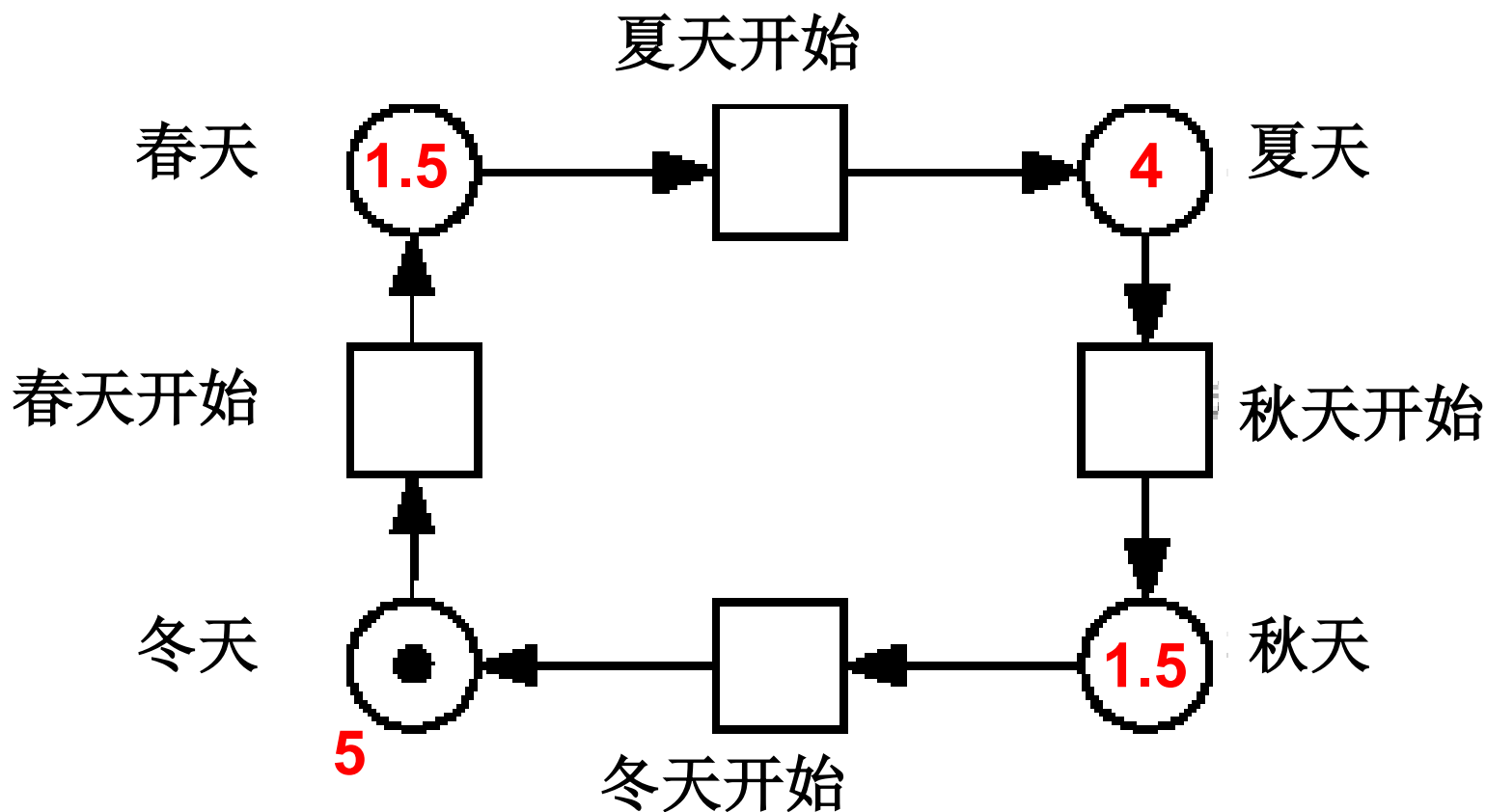




5.3 传统——Petri网(4/4)

- 对象合成Petri网(OCPN)

—例子：四季交替





5.3 传统——数据字典(1/6)

- 数据流程图的问题
 - 数据流程图描述了一个系统的主要处理逻辑，所存取的数据文件或数据库及其输入和输出的关系。但不能反映系统的具体细节。
- 什么是数据字典
 - 配合数据流程图，反映具体细节。二者结合，精确描述。
- 作用： 统一定义，便于通讯，便于共享



5.3 传统——数据字典(2/6)

- 元素

- 数据元素

- 最小数据单元

- 数据流

- 基本数据单元，有关的数据元素所组成的动态的数据结构

- 数据存储

- 数据结构的载体，静态的数据结构

- 处理

- 具体处理逻辑



5.3 传统——数据字典(3/6)

• 数据元素

数据元素定义

名称：材料数量

说明：出入库材料重量数量，以吨为单位

相关文件或记录：入库单、出库单、入库文件、出库文件

相关处理：

处理名称	DFD 参考号	结构图号
入库处理	1.0	
出库处理	2.0	

数据类型及长度：N(4,2)

数据范围：大于 0.00，小于或等于 99.99



5.3 传统——数据字典(4/6)

- 数据流

数据流定义

名称：入库单

说明：材料入库的原始数据

组成：材料名称+材料型号+材料规格+材料数量+单位|制造厂+入库时间
+入库人姓名

来源：保管员

终点：入库处理

容量：2000 份/天，星期一至星期五

峰值：1000 份 9:00~10:00 A.M.

注解：



5.3 传统——数据字典(5/6)

- 数据存储

数据存储定义

名称：库存台帐

说明：含有库存材料的全部信息

类型：文件 ☐ 数据库 ☒

组成：材料名称+材料型号+材料规格+材料单价+库存量+临界量+单位

相关过程：

处理名称	DFD 参考号	结构图号
更新台帐	3.0	
产生报表	4.0	



• 处理

处理过程定义

名称: 更新台帐

说明: 该过程在停止出入库活动后, 开始更新台帐中的库存数据

相关的 DFD 编号: 3.0

相关的结构图号:

输入数据流: 入库数据、出库数据、库存数据

输出数据流: 库存数据

存取的数据库:

DB 名	存取信息	检索数据
------	------	------

入库文件		
------	--	--

出库文件		
------	--	--

库存台帐		
------	--	--

处理逻辑

while (当日入库记录未完) do

 读一个入库记录

 检索库存台帐中的对应记录

 if 记录存在 then

 库存量=库存量+入库量

 else

 ...



5.3 传统——判定表和判定树(1/12)

- 判定表

- 左上部：条件或数据元素名称
- 右上部：所有条件组合
- 左下部：处理活动名称
- 右下部：注明条件组合和相应活动对应关系

	1	2	3	4
C ₁	T	T	F	F
C ₂	T	F	T	F
A ₁	√		√	
A ₂		√		
A ₃				√



- 假设某软件公司对某产品每个月销售政策如下：
 - 如果“月销售额” $< ¥10,000$ ，则“基础佣金” = 销售额的10%；
 - 如果 $¥10,000 \leq$ “月销售额” $< ¥100,000$ ，则“基础佣金” = 销售额的13%；
 - 如果“月销售额” $\geq ¥100,000$ ，则“基础佣金” = 销售额的18%，再加¥5000；
- 另外销售单价和销售数量对佣金也有影响。
 - 若有单价少于¥1000/套的，则减去基础佣金的10%；
 - 若单价超过¥2000/套的，则外加基础佣金的5%；
 - 若销售数量至少25套，则外加佣金2%。
- 请用判定表画出以上销售政策，注明绘制过程



5.3 传统——判定表和判定树(3/12)

• 举例1

数据元素	数据元素值
销售额	S: Small (<10000)
	M: Medium ($10000\sim99999.99$)
	L: Large (≥ 100000)
单价	C: Cheap (<1000)
	A: Average ($1000\sim2000$)
	E: Expensive (>2000)
销售数量	B: Below (<25)
	O: Over (≥ 25)



5.3 传统——判定表和判定树(4/12)

- 举例1

销售额的 10%

销售额的 13%

销售额的 18%+5000

减去基础佣金的 10%

外加基础佣金的 5%

外加基础佣金的 2%



5.3 传统——判定表和判定树(5/12)

• 举例1

软件公司销售政策的判定表

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
销售额	S	S	S	S	S	S	M	M	M	M	M	M	L	L	L	L	L	L
单价	C	C	A	A	E	E	C	C	A	A	E	E	C	C	A	A	E	E
销售数量	B	O	B	O	B	O	B	O	B	O	B	O	B	O	B	O	B	O
销售额的 10%	✓	✓	✓	✓	✓	✓												
销售额的 13%							✓	✓	✓	✓	✓	✓						
销售额的 18%+5000													✓	✓	✓	✓	✓	✓
减去基础佣金的 10%	✓	✓					✓	✓					✓	✓				
外加基础佣金的 5%					✓	✓					✓	✓					✓	✓
外加基础佣金的 2%		✓		✓		✓		✓		✓		✓		✓		✓		✓



5.3 传统——判定表和判定树(6/12)

• 举例2

商店业务处理系统中“检查订单”的判定表。

		1	2	3	4
条件	订货单金额	>5000 元	>5000 元	≤5000 元	≤5000 元
	偿还欠款情况	>60 天	≤60 天	>60 天	≤60 天
动作	在偿还欠款前 不予批准	√			
	发出批准书		√	√	√
	发出发货单		√	√	√
	发催款通知书			√	



5.3 传统——判定表和判定树(7/12)

判定表的改进

如果表中有两条或更多的处理规则具有相同的动作，并且其条件项之间存在着某种关系，就可设法将它们合并。

改进的“检查订货单”的决策表

		1	2	3
条件	订货单金额	>5000 元	—	≤5000 元
	偿还欠款情况	>60 天	≤60 天	>60 天
动作	在偿还欠款前不予批准	√		
	发出批准书		√	√
	发出发货单		√	√
	发催款通知书			√



5.3 传统——判定表和判定树(8/12)

- 判定表优点
 - 表达逻辑清楚，比自然语言容易理解
 - 问题中的条件或数据元素在表中只出现一次
- 判定表缺点
 - 用户难以理解
 - 随着问题中的条件增多，判定表会变的相当复杂，有时候一个有经验的分析员也难以画出某些判定表



5.3 传统——判定表和判定树(9/12)

- 判定树
 - 是一个树状图
 - 根节点
 - 表示问题的名字
 - 内部节点
 - 表示条件，叶子节点表示活动。



5.3 传统——判定表和判定树(10/12)

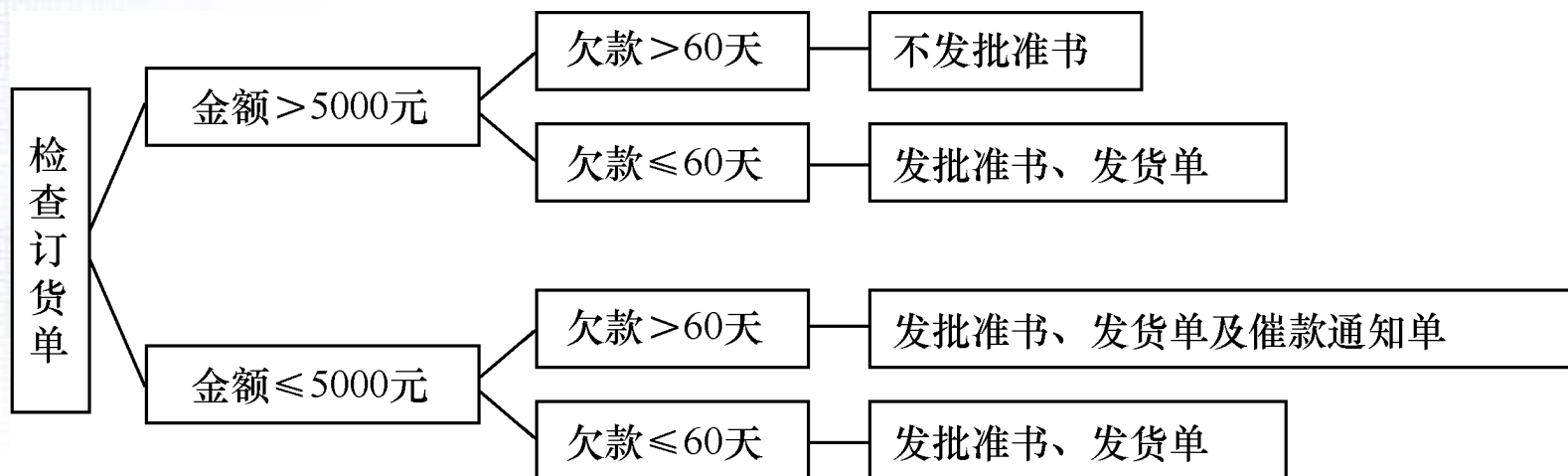
• 例子1





5.3 传统——判定表和判定树(11/12)

- 举例2
- 检查订货单的判定树





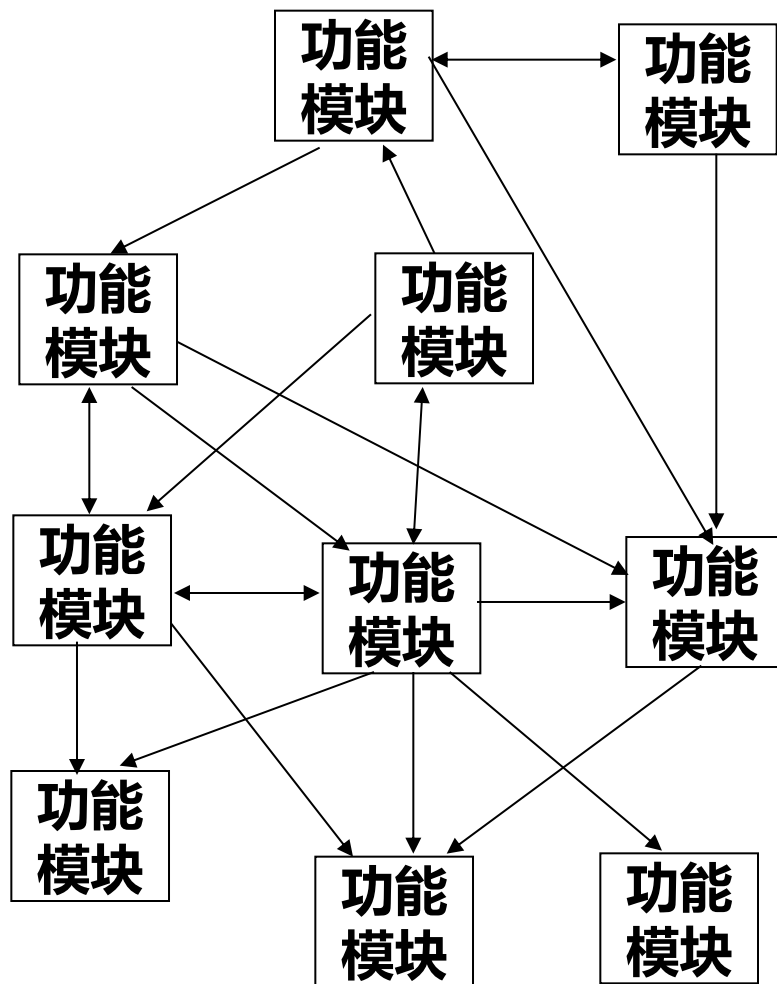
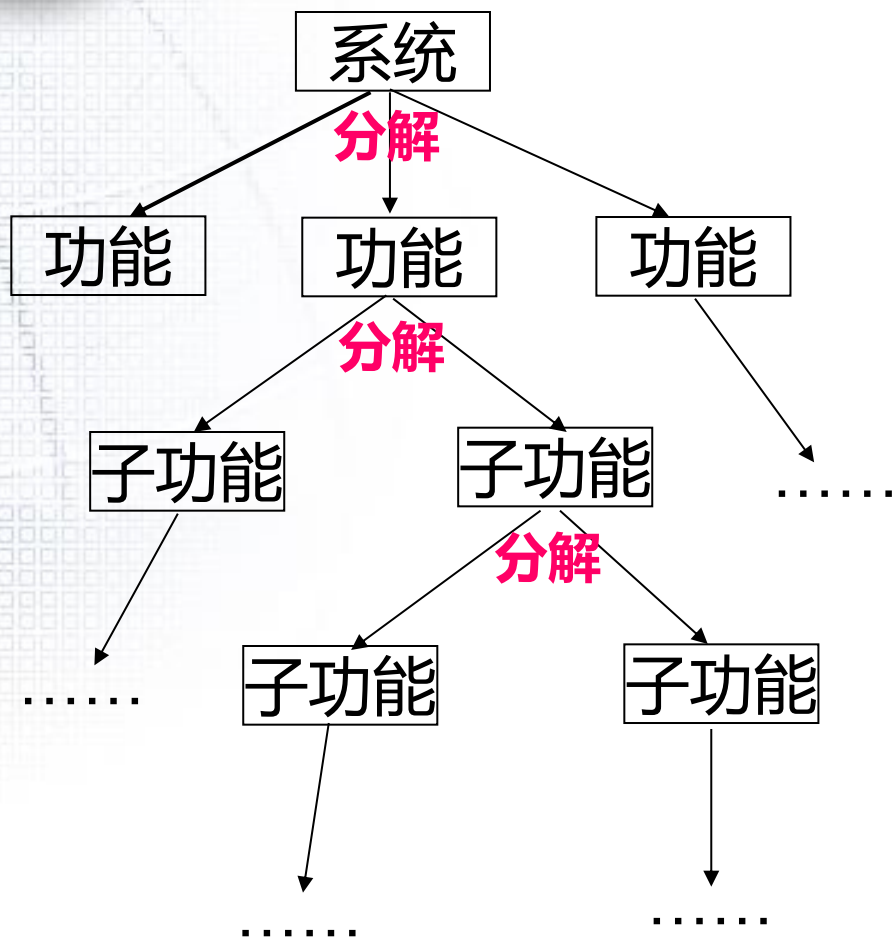
5.3 传统——判定表和判定树(12/12)

- 判定树优点：
 - 理解容易，不需要用户培训
 - 绘制方法更直观
- 判定树缺点：
 - 繁琐，同一条件要书写多次（如前面股票佣金政策用判定树表示，则会非常复杂）



传统软件开发机制

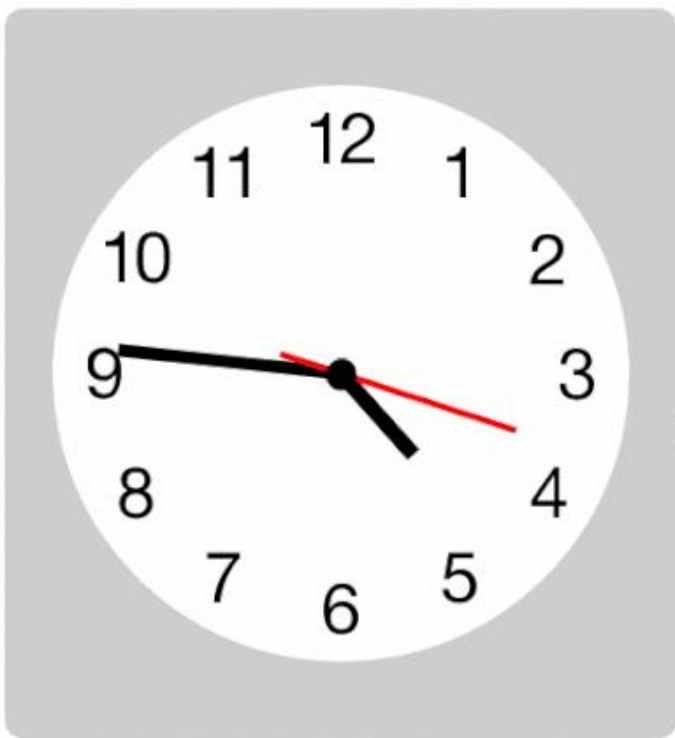
得到的系统模型：
由模块及其接口构成





面向对象

- 00



https://blog.csdn.net/weixin_38004595



传统向面向对象的演进

传统方法
系统是处理的集合
处理与数据实体的交互
处理所接受的输入并产生输出



面向对象方法
系统是交互对象的集合
对象与人或其他对象交互
对象发送和响应消息

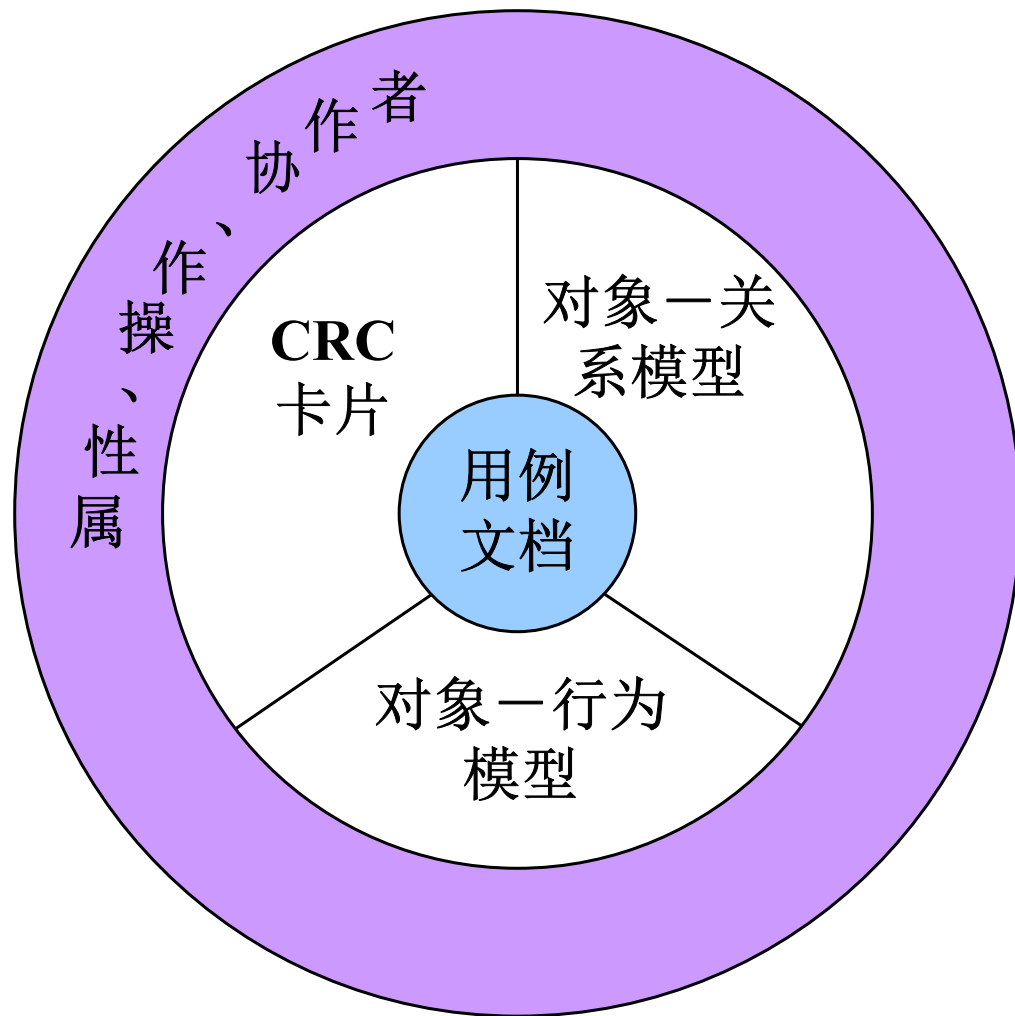
DFD

UC



5.4 面向对象的需求分析建模方法

- UML
- 用例建模
- CRC建模
- 对象-关系模型
- 对象-行为模型





5.4.0 UML简介 (1/6)

- 三剑客的贡献(50->1)



Grady Booch



Jim Rumbaugh



Ivar Jacobson



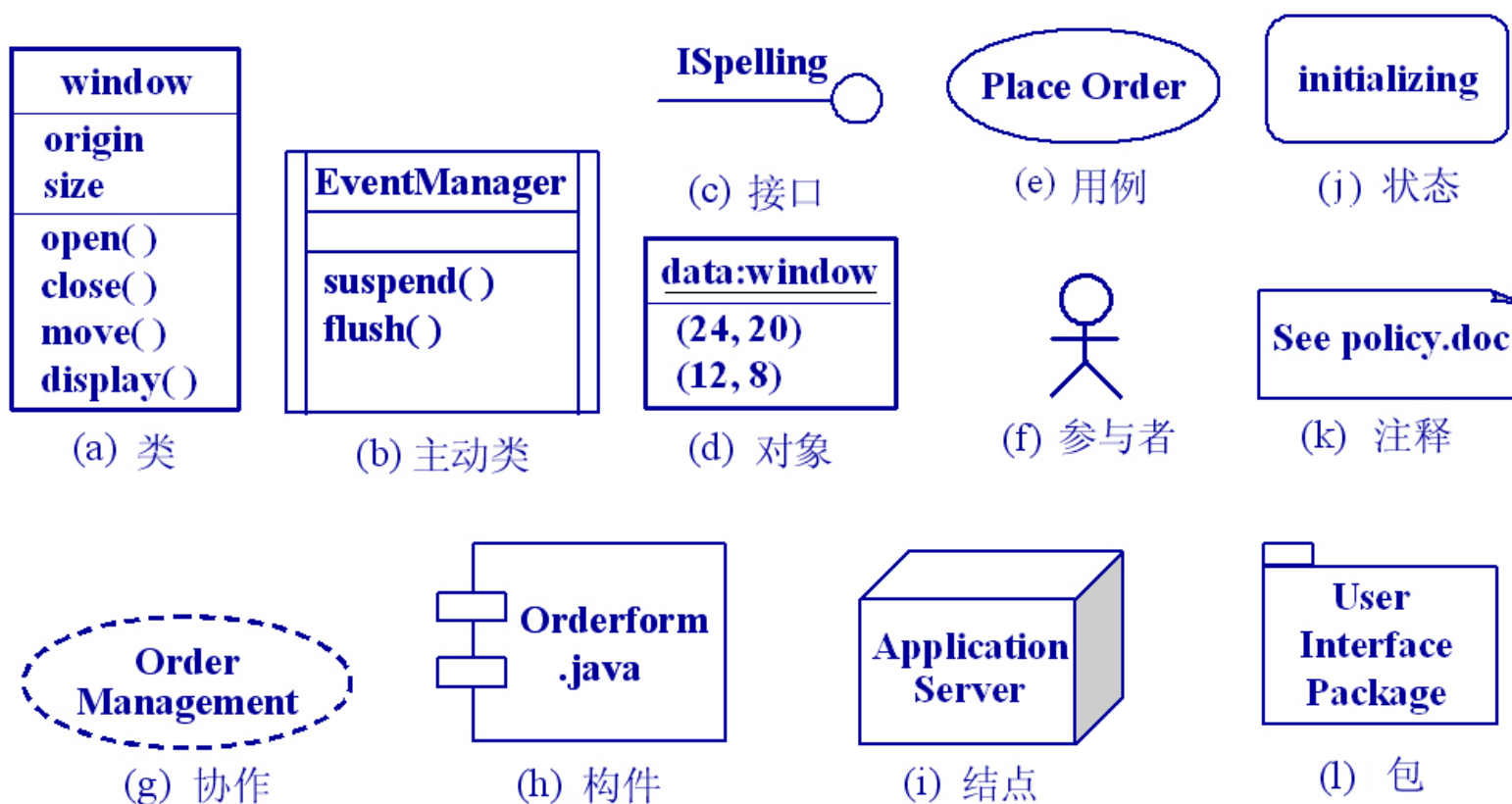
5.4 UML简介(2/6)

- UML特点
 - 统一的标准
 - 面向对象
 - 可视化、表示能力强大
 - 独立于软件开发过程
 - 概念明确，建模简洁，图形清晰，容易掌握
- UML与程序设计语言的区别
- UML与软件开发过程的关系



5.4 UML简介(3/6)

- UML事物：对模型中最具代表性成分的抽象，在UML中，可以分为结构事物、行为事物、分组事物和注释事物4类。





5.4 UML简介(4/6)

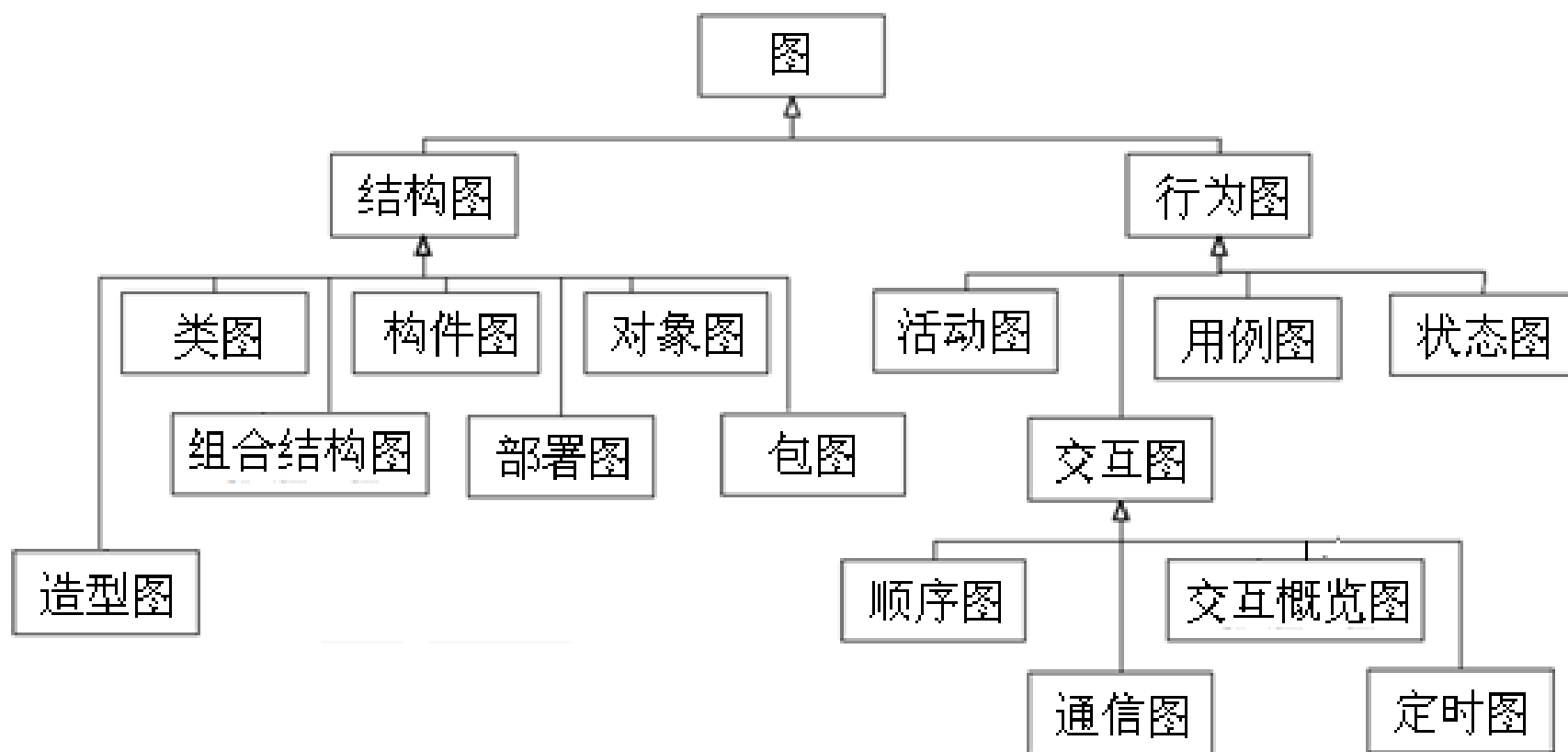
- UML的关系





5.4 UML简介(5/6)

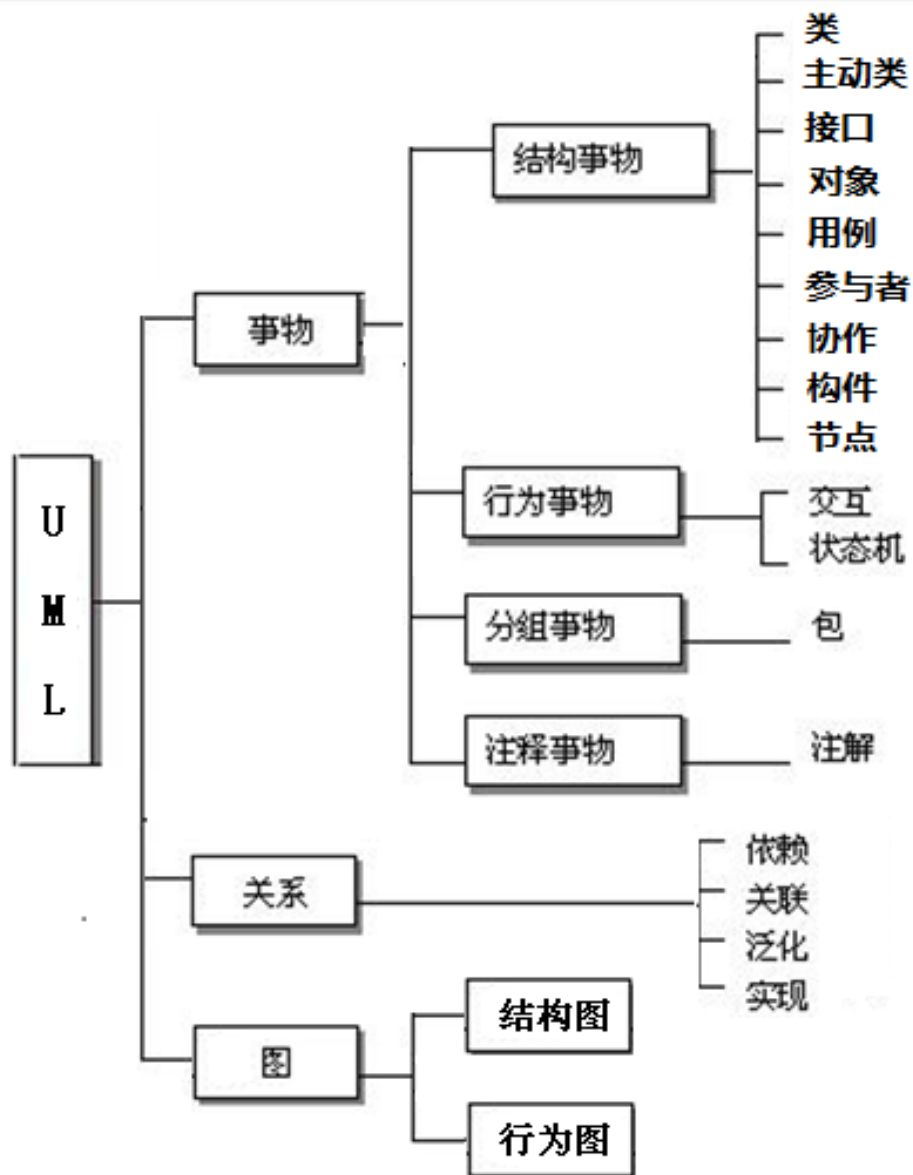
- UML的图





5.4 UML简介(6/6)

• 基本模型





5.4 OOA方法——用例(1/2)

- 定义:

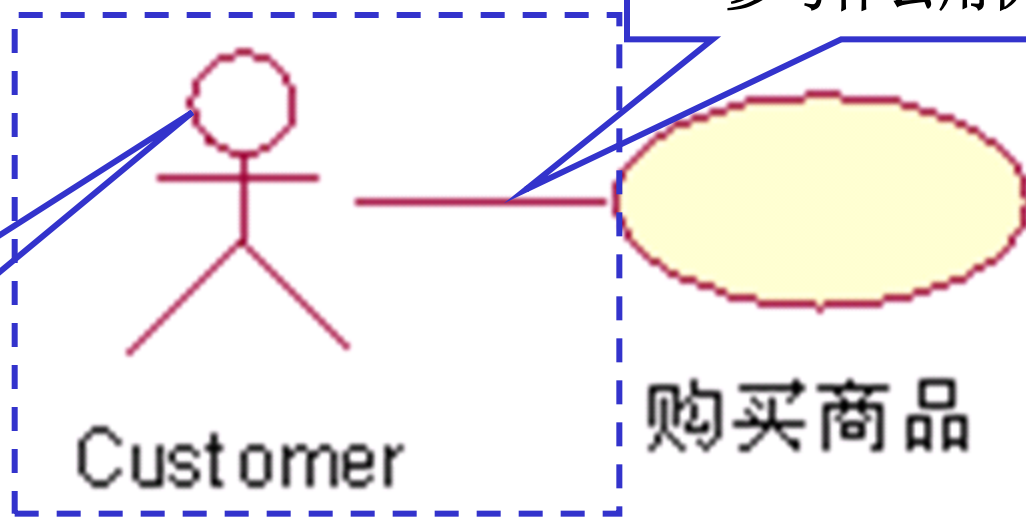
- 用例是系统、子系统或类和外部的参与者 (**Actor**) 交互动作序列的说明, 包括可选的动作序列和会出现异常的动作序列

- 表示方法

- 一个椭圆
- 用例名称

- 用与例

小人: 参与者
代表一个角色

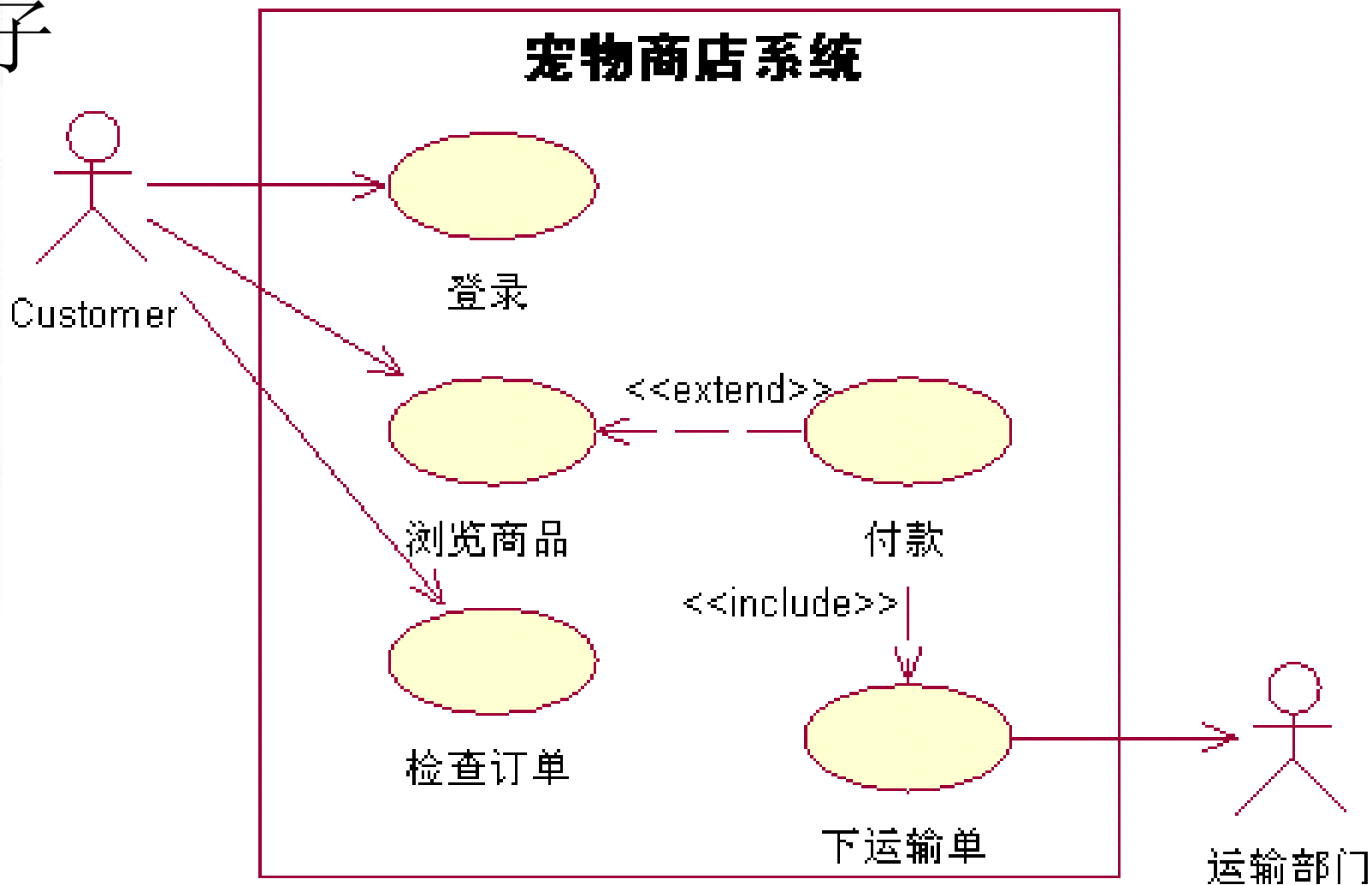


连线: 哪个参与者
参与什么用例



5.4 OOA方法——用例(2/2)

- 例子





5.4 OOA方法——开发用例(1/3)

- 开发用例图的步骤
 - (1)找出系统外部的参与者和外部系统，确定系统的边界和范围
 - (2)确定每一个参与者所期望的系统行为
 - (3)把这些系统行为命名为用例
 - (4)使用泛化、包含、扩展等关系处理系统行为的公共或变更部分



5.4 OOA方法——开发用例(2/3)

- (5)编制每一个用例的脚本
- (6)绘制用例图
- (7)区分主事件流和异常事件流，如果需要，可以把表示异常情况的事件流作为单独的用例处理
- (8)细化用例图，解决用例间的重复与冲突问题



5.4 OOA方法——开发用例(3/3)

- 难点
 - 应该包含哪些用例，如何发现它们
 - 依赖于分析人员的个人经验和领域知识
- 启发性原则
 - 和用户交互
 - 把自己当作参与者
 - 确定用例和确定参与者不能截然分开
- 学习用例时应该掌握几点？



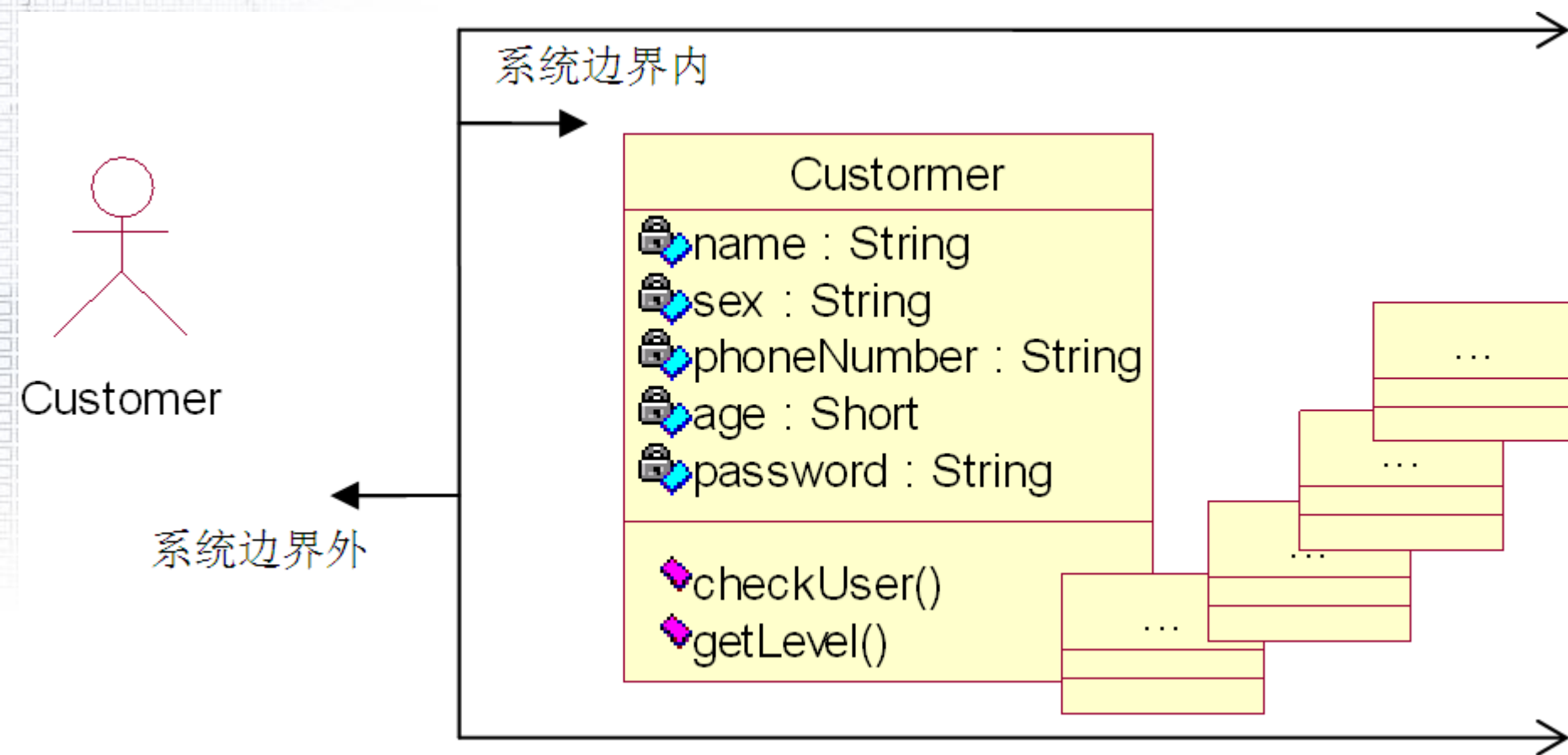
5.4 OOA——识别Actor(1/6)

- 识别Actor: Actor在系统之外, 透过系统边界与系统进行有意义交互的任何事物。
 - Actor在系统边界外部
 - Actor直接与系统交互
 - Actor与系统的交互应该是有意义的
 - Actor可能是任何事物
 - 一个人可以担任多个参与者



5.4 OOA——识别Actor(2/6)

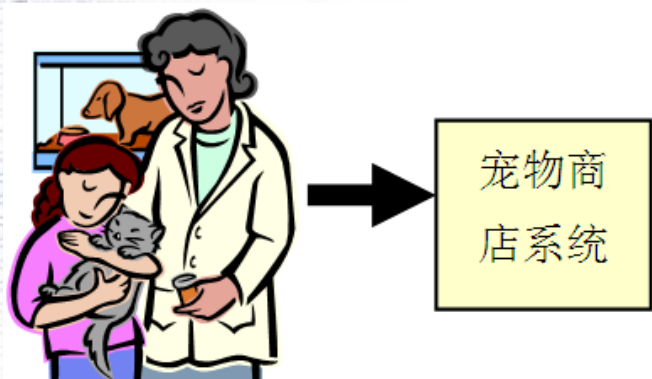
- Actor在系统边界外部，不是系统的一部分





5.4 OOA——识别Actor(3/6)

- Actor直接与系统交互
- Actor与系统的交互应该是有意义的



消费者

模式 1



消费者

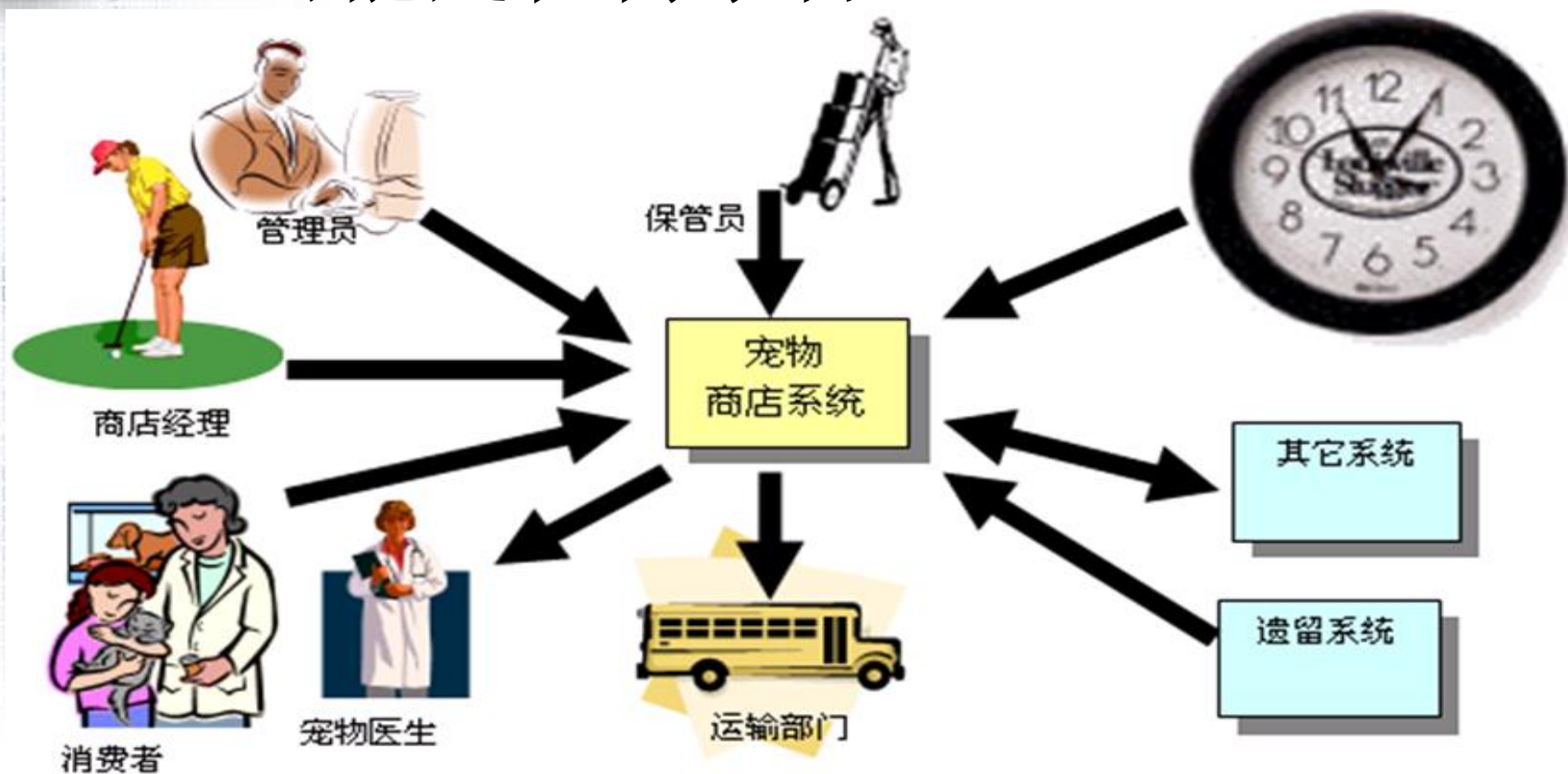
代理商

模式 2



5.4 OOA——识别Actor(4/6)

- Actor可能是任何事物

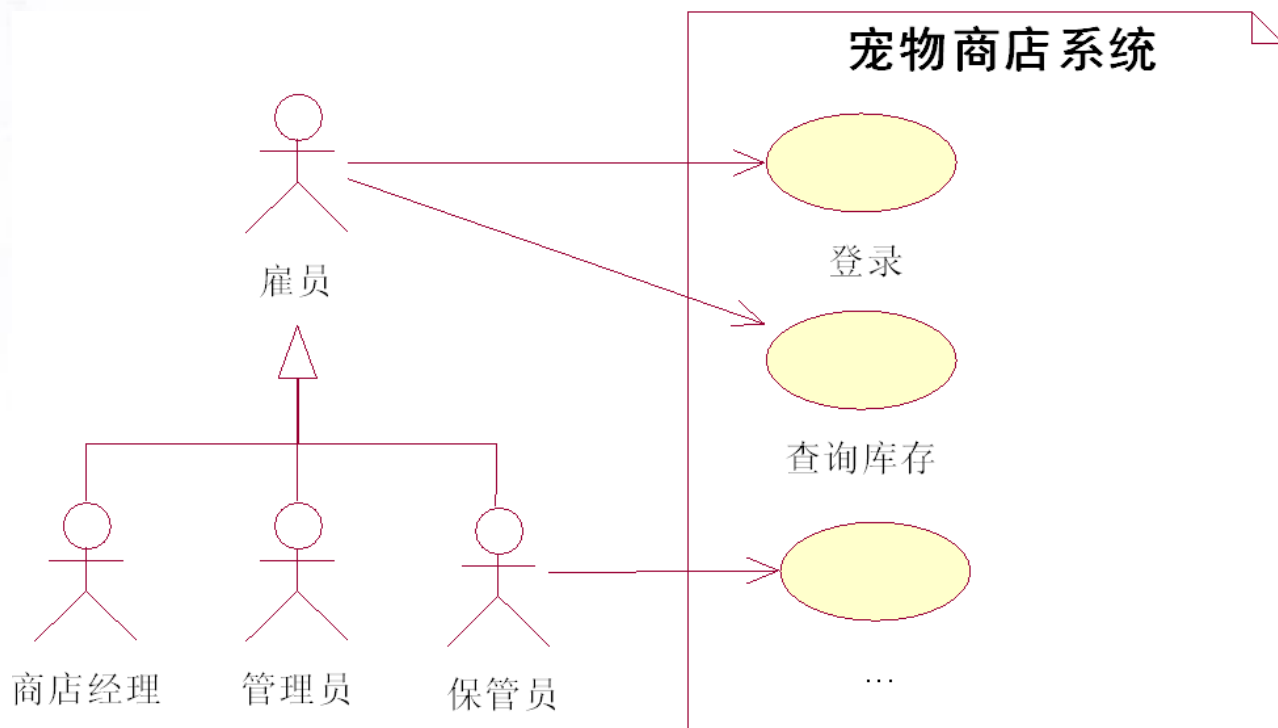


参与者的图标虽然使用人形图标来表示，但参与者不一定是人，也可能是其它子系统、其它系统、温度传感器、光线传感器等其它因素。



5.4 OOA——识别Actor(5/6)

- 一个人可以担任多个参与者
- Actor之间的关系：泛化





5.4 OOA——识别Actor(6/6)

- 识别Actor练习题

- 如果打算做一个在线的课程网站，你觉得应该有哪些Actor？

- 学生
 - 教师
 - 管理员
 - 遗留系统
 - 时间
 -



5.4 OOA——识别用例(1/4)

- 识别用例

- 用例是系统、子系统或类和外部的参与者（**Actor**）交互的动作序列的说明，包括可选的动作序列和会出现异常的动作序列
- 从**使用**系统的角度描述系统信息
- 描述了用户提出的一些**可见**需求，对应一个具体的用户**目标**
- 在用例图中每个用例被抽象为一个椭圆



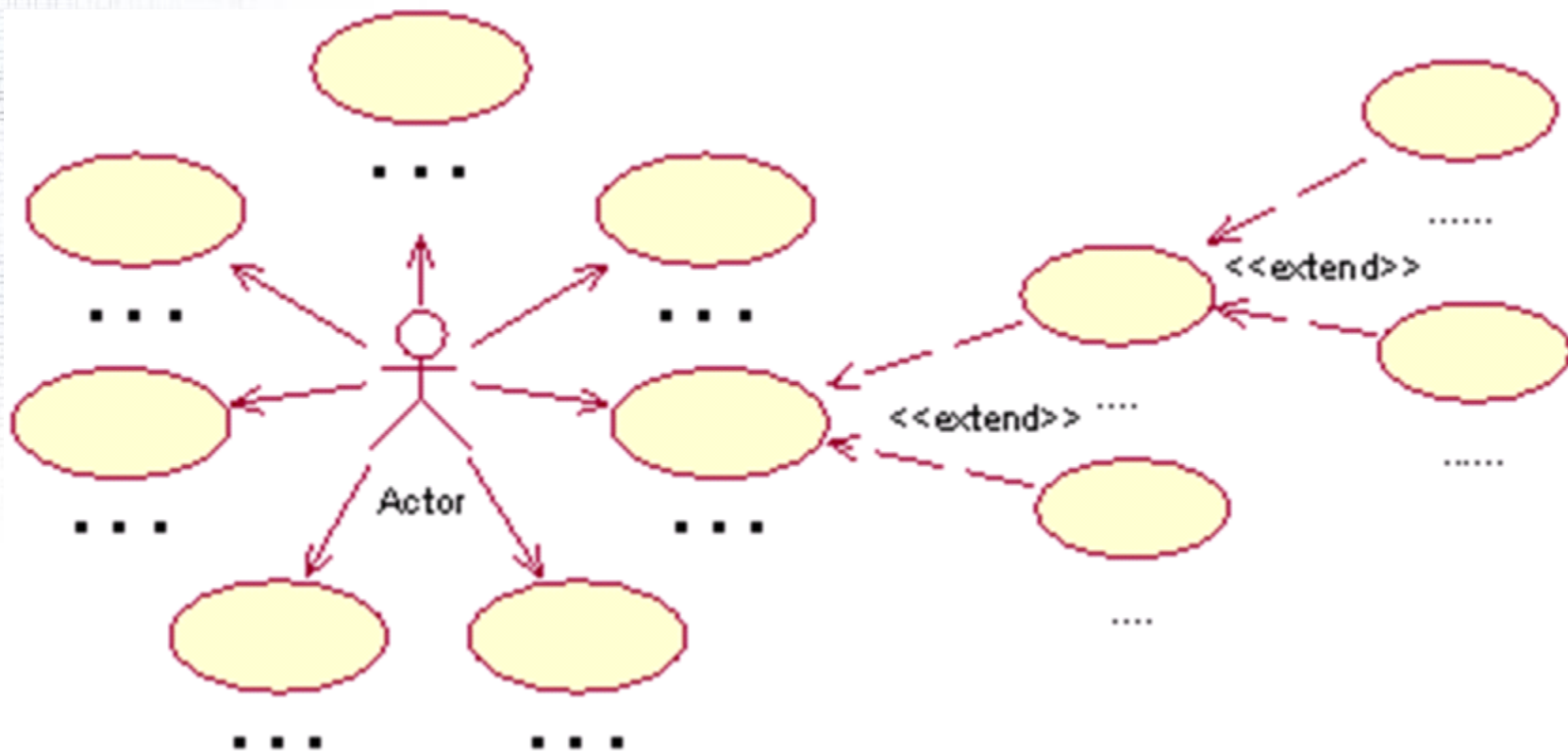
5.4 OOA——识别用例(2/4)

- 识别Use Case
 - 用例止于系统边界
 - 用例是一个交互的抽象
 - 从Actor的角度去描述用例
 - 粒度适度



5.4 OOA——识别用例(3/4)

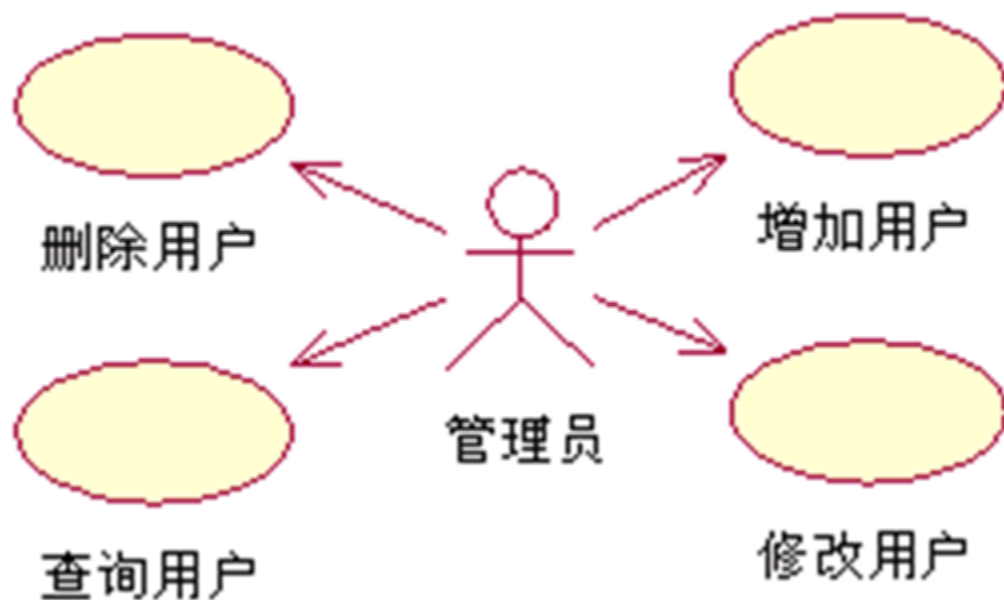
- 例子1:
 - 粒度过细，陷入功能分解





5.4 OOA——识别用例(4/4)

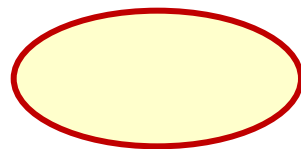
- 例子2:
—CRUD



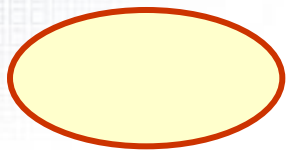


5.4 OOA——用例命名(1/1)

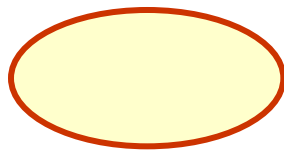
—命名：动宾结构或主谓结构



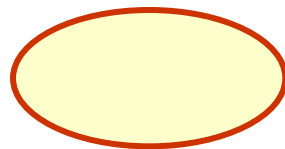
Withdraw Money



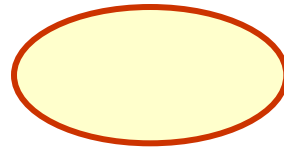
进行宠物搜索



宠物搜索



进货管理



结帐功能



5.4 OOA——用例关系(1/3)

- 识别Use Case 之间的关系
 - 泛化（Generalization）
 - 包含（Include）
 - 扩展（Extend）



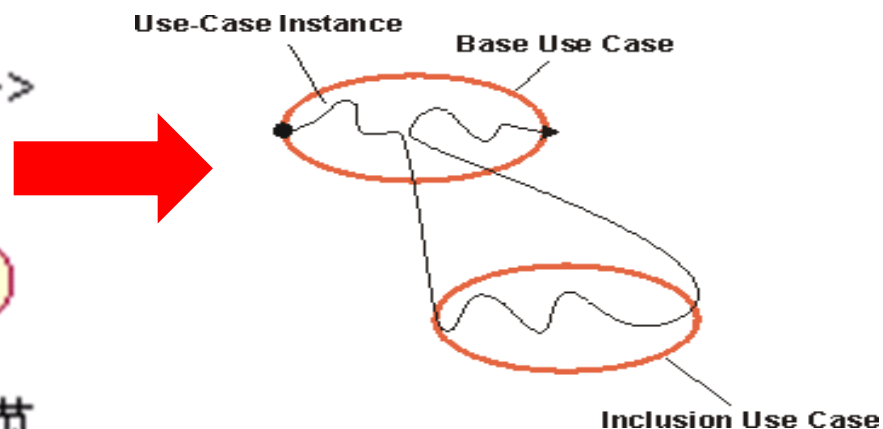
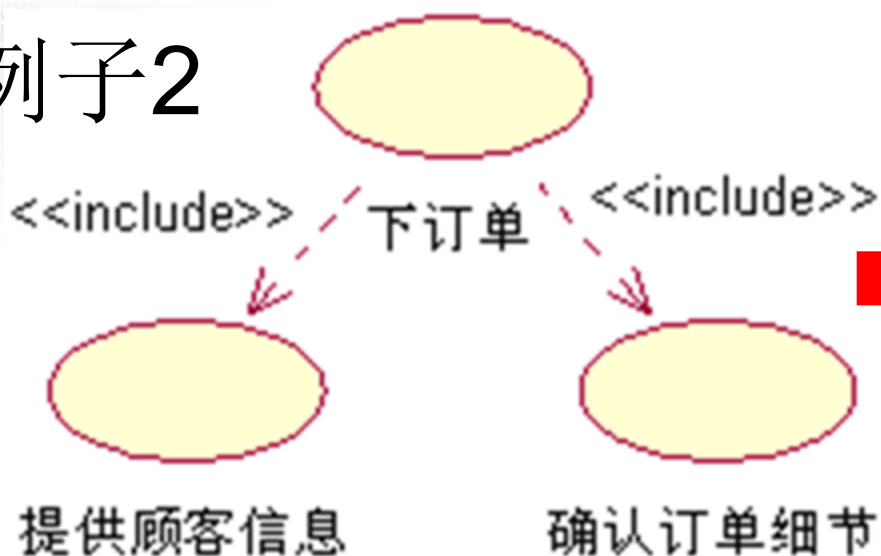


5.4 OOA——用例关系(2/3)

- 例子1



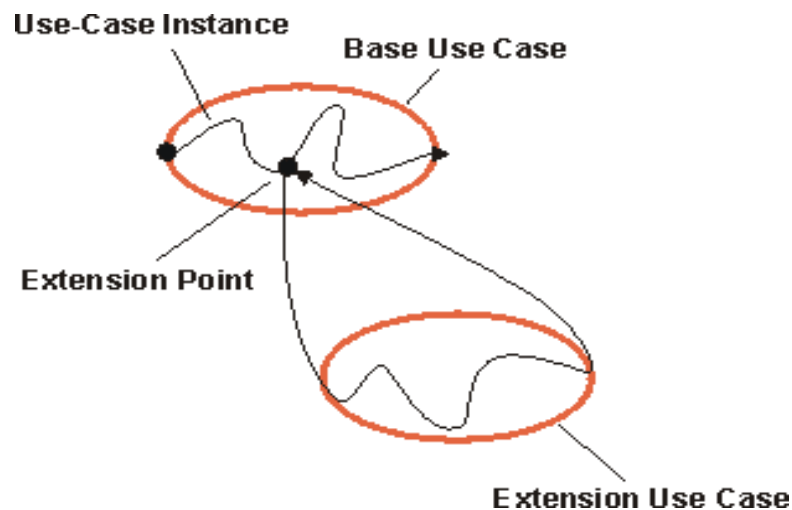
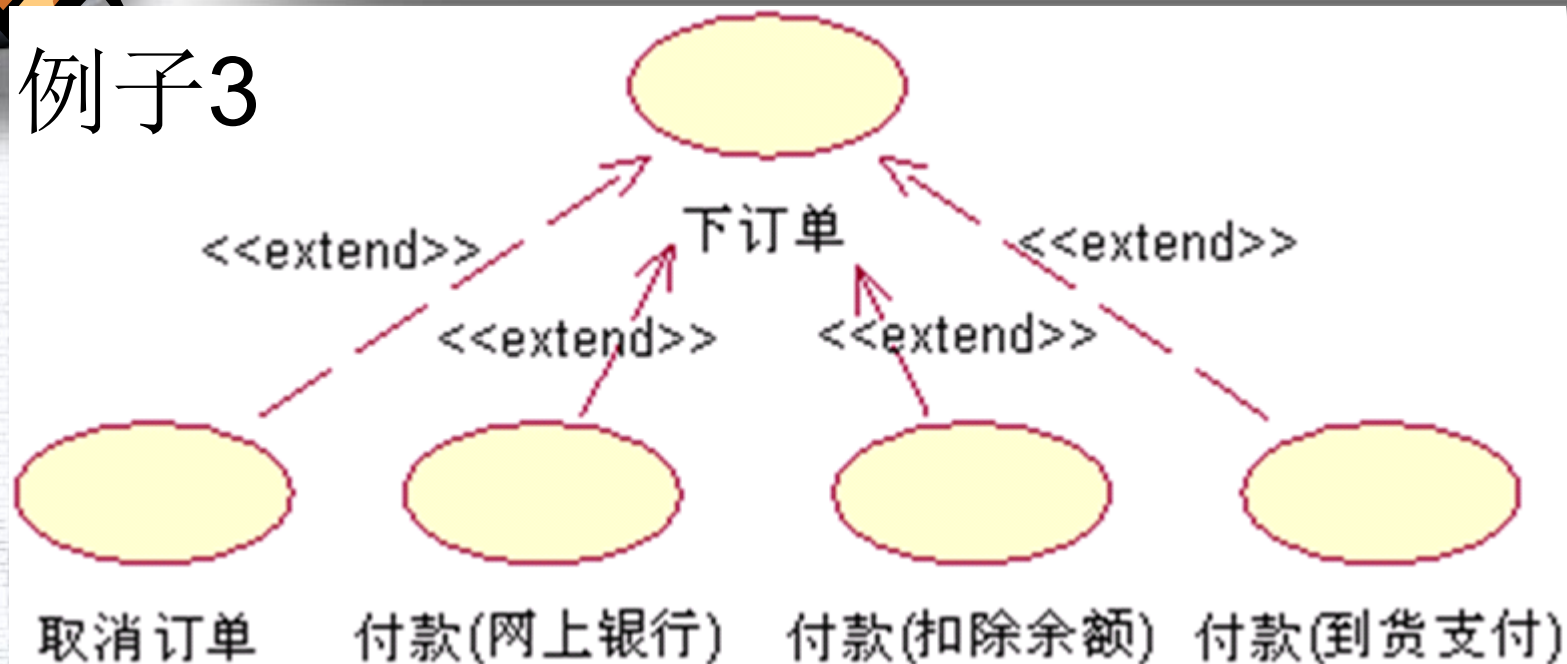
- 例子2





5.4 OOA——用例关系(3/3)

• 例子3



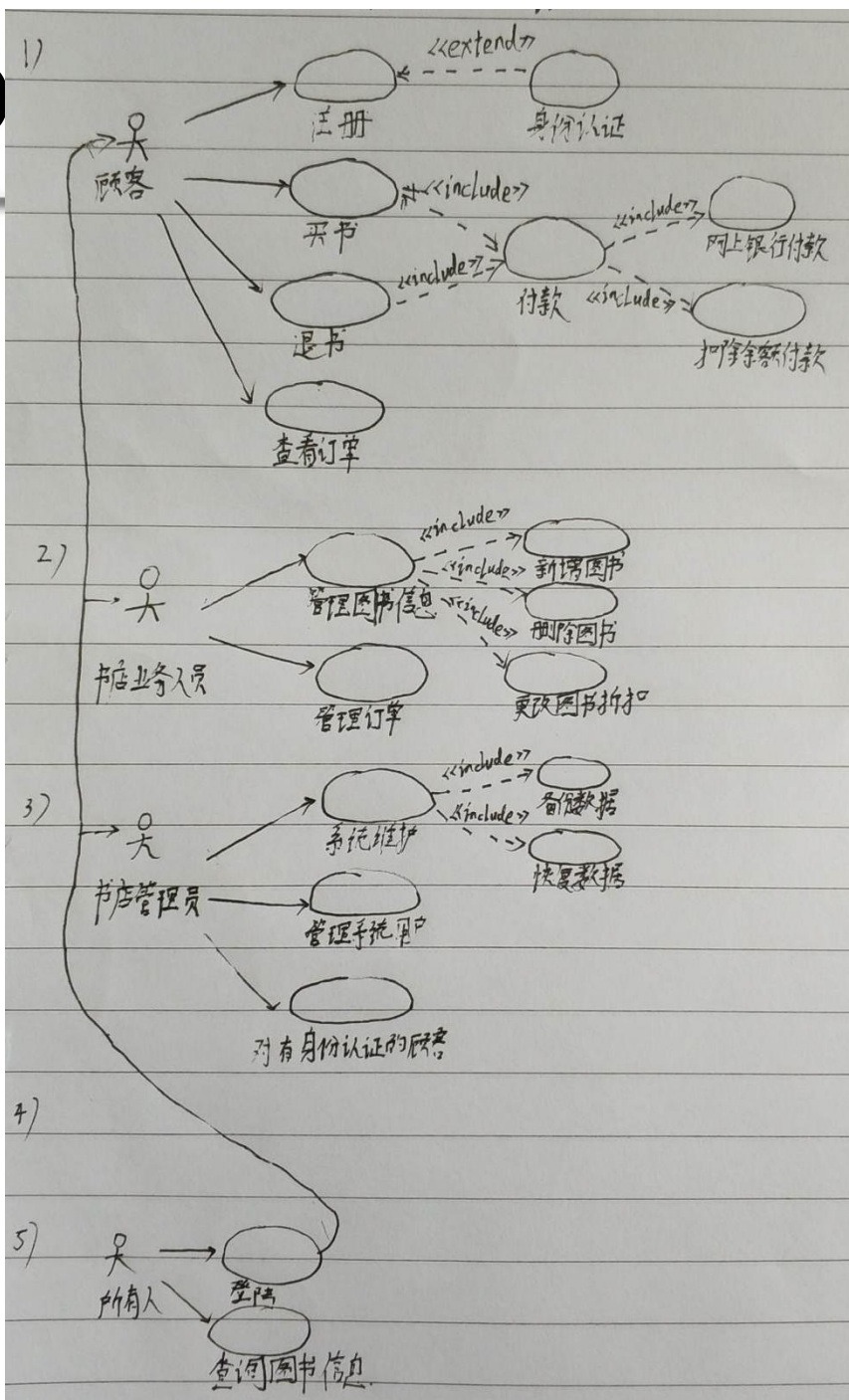


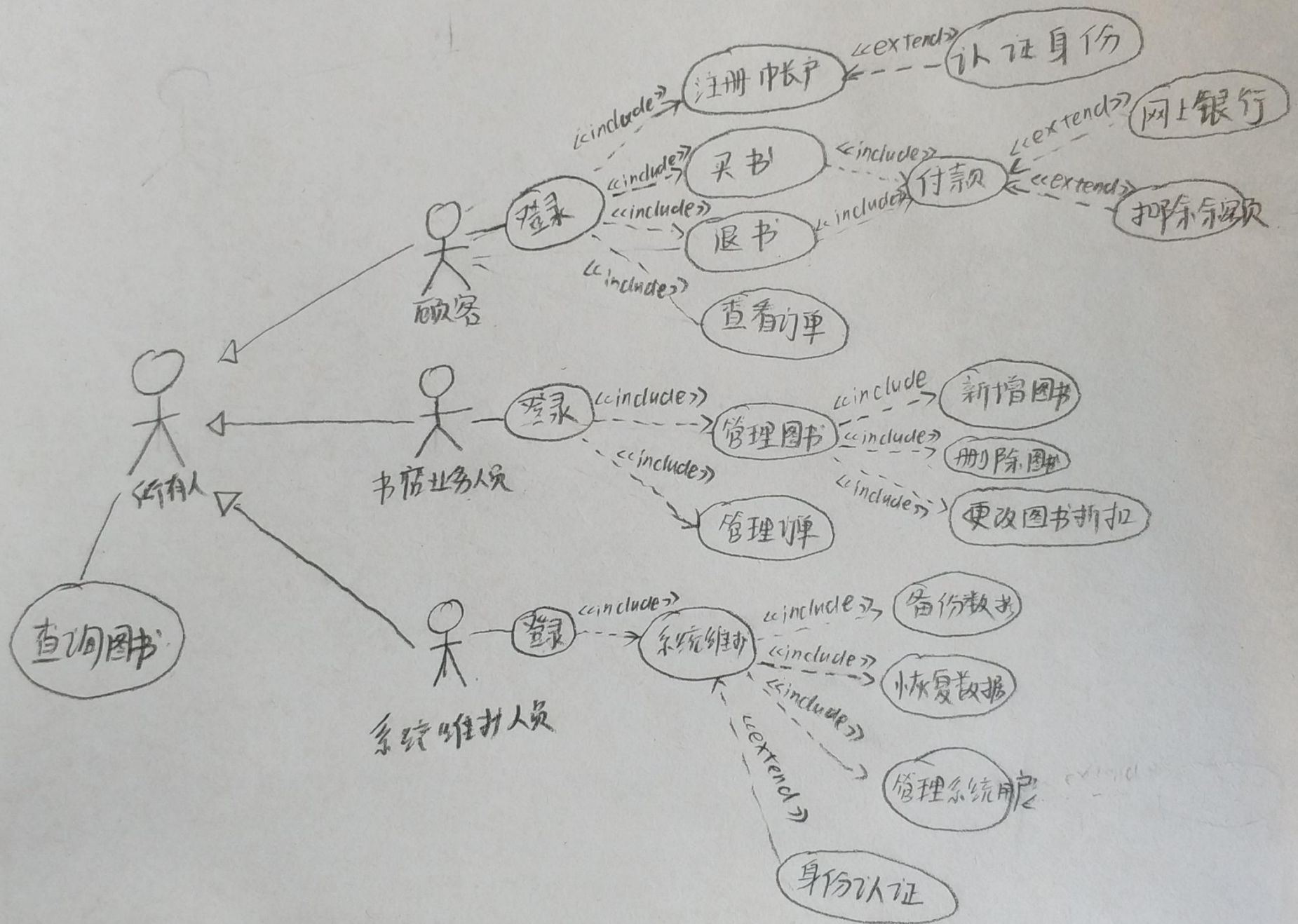
5.4 OOA方法—用例建模示例(1/9)

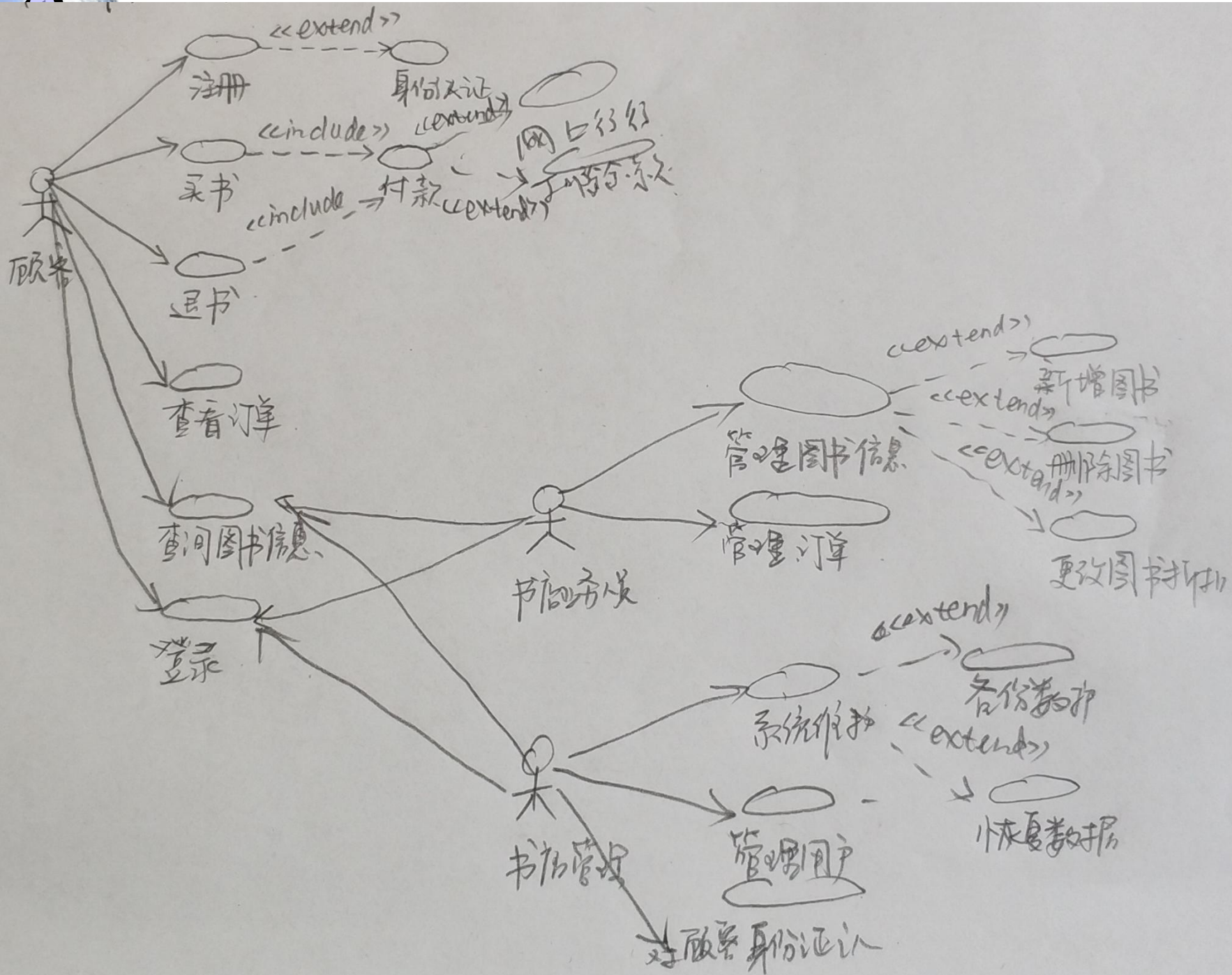
- 根据以下某网上书店的需求画出用例图。
 - 1) 顾客可以注册、买书、退书、查看订单。在注册时可以选择身份认证加强账户的安全性。在买书和退书时包括付款操作，付款可以选择通过网上银行进行，或选择扣除余额方式进行。
 - 2) 书店业务人员可以管理图书信息和订单。在管理图书信息时，可以新增图书、删除图书或更改图书折扣。
 - 3) 书店管理员可以进行系统维护（包括备份数据、恢复数据等），可以对所有系统用户进行管理，还可以对选择身份认证的顾客进行身份认证操作。
 - 4) 所有人员都可以查询图书信息。
 - 5) 所有人员都必须登录后，才能使用相应的功能。

5.4 O

模示例(2/9)

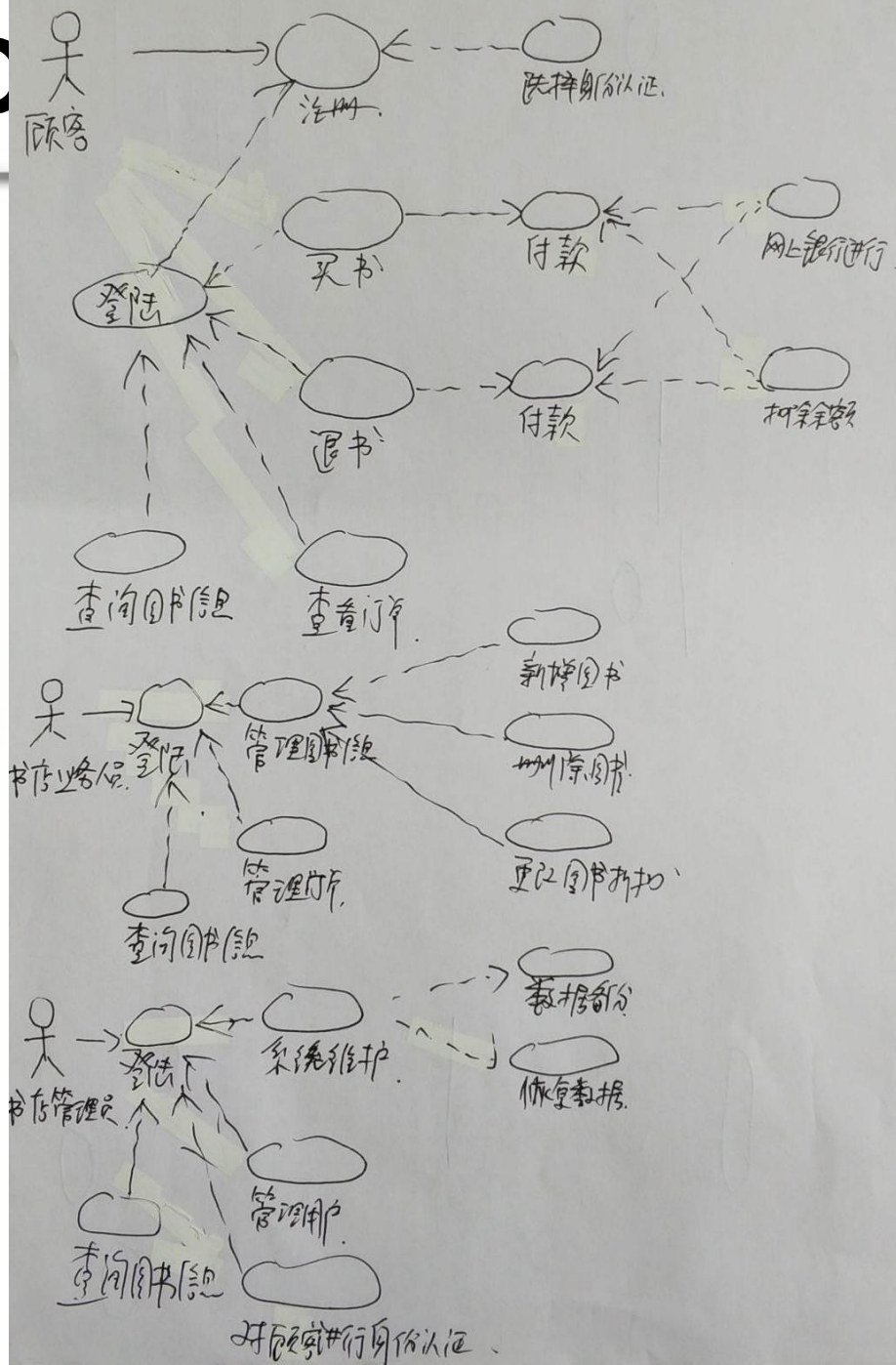








示例(5/9)





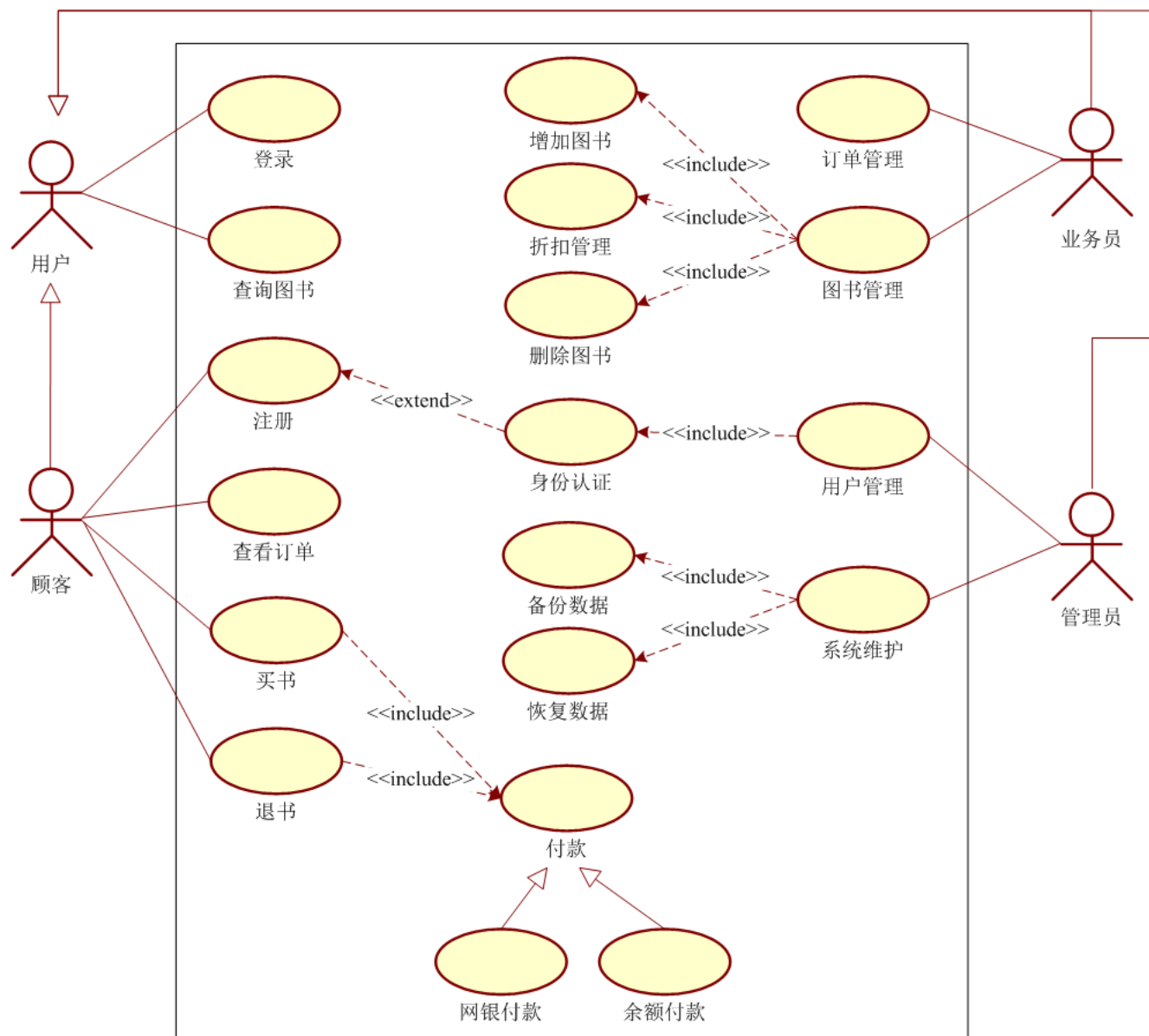
5.4 OOA方法—用例建模示例(6/9)

- 注意

- 用例不表示流程的先后顺序
- 用例之间有3种关系，不能直接画直线联系
- 《include》箭头从基用例指向子用例
- 《extend》箭头从子用例指向基用例
- 参与者只能是“人形”图例
- 画出参与者之间的关系（泛化关系）
- 把需求描述中所涉及的用户都画出来
- 用例不是针对某次交互，而是交互的抽象



5.4 OOA方法—用例建模示例(7/9)





5.4 OOA方法—用例建模示例(8/9)

- 相关说明

- 大功能中包含若干个子功能，可以使用《**include**》，如图书管理包含增加图书、删除图书、更改图书折扣
- 功能中包含可做可不做的功能，可以使用《**extend**》，如注册时可以选择身份认证
- 有多种实现手段，可以使用泛化关系，如付款可以使用网银付款和余额付款



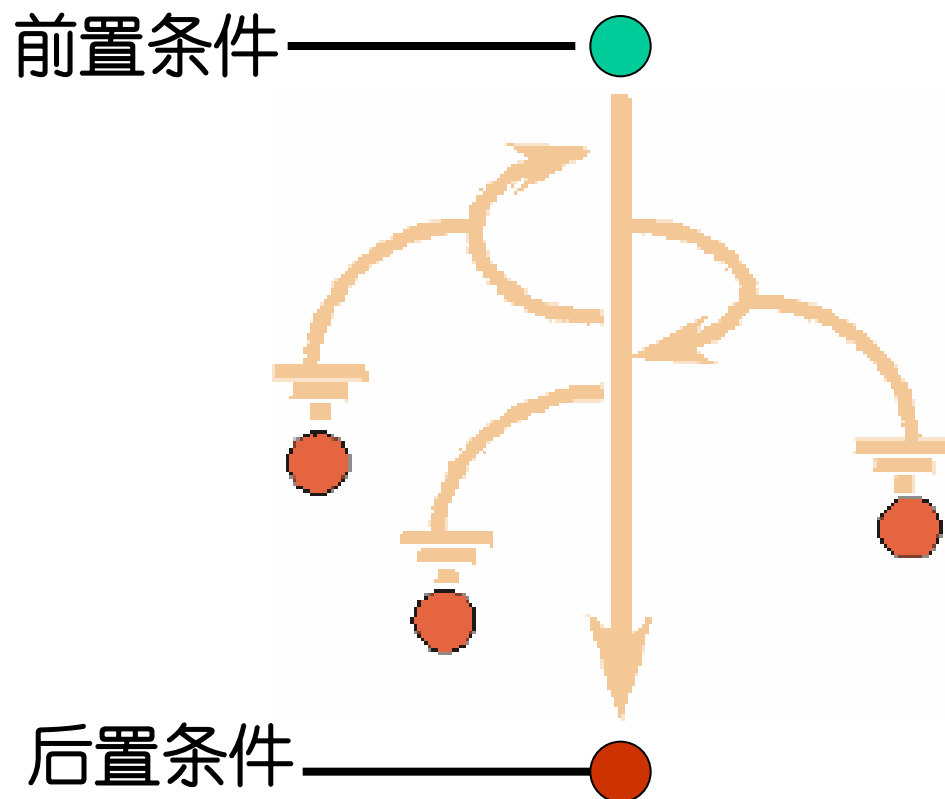
5.4 OOA方法—用例建模示例(9/9)

- 练习



5.4 OOA——用例文档(1/15)

- 用例文档
 - 用例编号
 - 用例名
 - 用例描述
 - 参与者
 - 前置条件
 - 后置条件





5.4 OOA——用例文档(2/15)

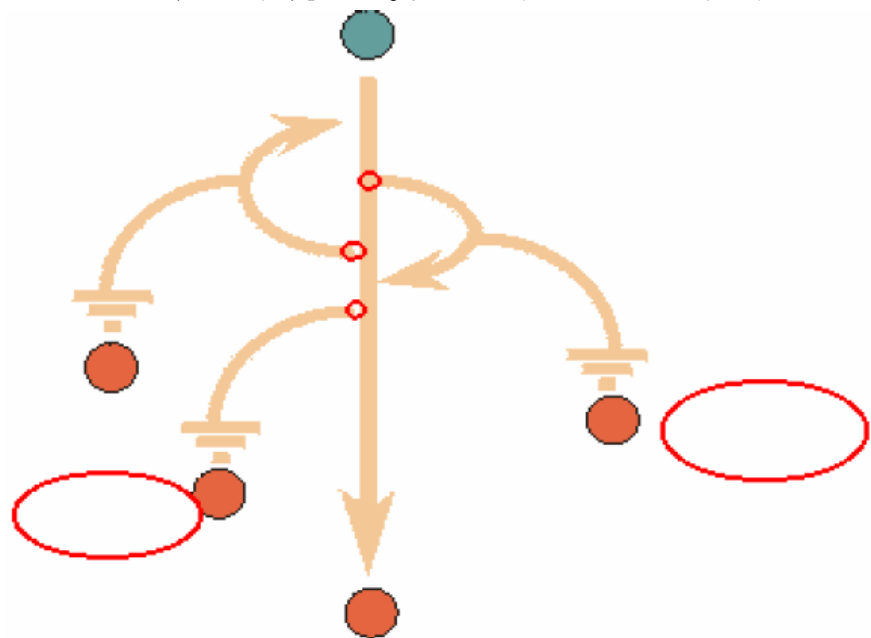
-基本路径/主事件流

- 1..... × × × ×
- 2..... × × × ×
- 3..... × × × ×

-扩展点/异常事件流/可选事件流或异常/替换事件流

- 2a. × × × ×
 - 2a1.... × × × × ×

-补充说明





用例编号：005

用例名称：订购货物

用例描述：用户创建一个新的请求商品的订单，并为其付款

参与者：客户、客户代表

前置条件：一个合法的客户已经登录到这个系统

后置条件：如果订单没有被取消，它将保存在系统中，并做上标记

事件流：

基本路径：

1. 当客户选择订购货物时，用例开始
2. 客户输入他的姓名和地址
3. 当客户输入产品代码
 - a. 系统给出产品描述和价格
 - b. 系统往客户订单中添加该物品的价格

循环结束

4. 客户输入信用卡支付信息
5. 客户选择提交
6. 系统检验输入的信息，把该订单作为未完成的交易保存，同时向记账系统转发支付信息。
7. 当支付确认后，订单就被标记上已经确认，同时返回给客户一个订单ID，用例结束。

可选路径：

- 6a. 如果有任何不正确的信息，系统提示客户去修改这些信息。
- 7a. 如果支付没有被确认，系统将提示客户改正支付信息或者取消。如果客户选择修改信息，就回到基本路径第4步；如果选择取消，用例结束



5.4 OOA——用例文档(4/15)

- 常见错误
 - 唯系统型
 - 只描述系统行为，没有描述Actor的行为
 - 唯参与者型
 - 只描述Actor的行为，没有描述系统行为
 - 过早界面型
 - 在用例描述中就设定对用户界面的设计要求
 - 冗长型
 - 描述过于冗长



5.4 OOA——用例文档(5/15)

- 例子(唯系统型)
 - Use Case: Withdraw Cash
 - 参与者: Customer
 - 主事件流:
 - (1)ATM系统获得ATM卡和密码
 - (2)设置事务类型为Withdrawal
 - (3) ATM系统获取要提取的现金数目
 - (4)ATM验证帐户上是否有足够的储蓄金额
 - (5)输出现金、数据和ATM卡
 - (6)系统复位



5.4 OOA——用例文档(6/15)

- 例子(唯参与者型)
 - Use Case: Withdraw Cash
 - 参与者: Customer
 - 主事件流:
 - (1)储户插入ATM卡, 并键入密码
 - (2)储户按Withdrawal按钮, 并键入取款数目
 - (3)储户取走现金、ATM卡并拿走收据
 - (4)储户离开



5.4 OOA——用例文档(7/15)

- 例子(改进)
 - Use Case: Withdraw Cash
 - 参与者: Customer
 - 主事件流:
 - (1)通过读卡机, 储户插入ATM卡
 - (2)ATM系统从卡上读取银行ID、帐号、加密密码, 并用主银行系统验证银行ID和帐号
 - (3)储户键入密码
 - (4)ATM系统根据(2)中读取的卡上密码对密码进行验证



5.4 OOA——用例文档(8/15)

- (5) 储户按下FASTCASH按钮，并键入取款数量，取款数量应该是5美元的倍数
- (6) ATM系统通知主银行系统，传递储户帐号和取款数量，并接收返回的确认信息和储户帐户余额
- (7) ATM系统输出现金、ATM卡和显示帐户余额的收据
- (8) 系统复位





5.4 OOA——用例文档(9/14)

- 例子(过早界面型)
 - Use Case: Buy Something
 - Actor: Customer
 - Basic Flow
 - (1)系统显示ID和Password窗口
 - (2)顾客键入ID和密码，然后按下OK按钮
 - (3)系统验证顾客ID和密码,并显示个人信息窗口
 - (4)顾客键入姓名、地址、邮编、电话号码，然后按下OK按钮
 - (5)系统验证用户是否为老客户



5.4 OOA——用例文档(10/15)

- (6)系统显示可以卖的商品列表
- (7)顾客在准备购买的商品图片上单击，并在图片旁边输入要购买的数量，选购商品完毕后按Done按钮
- (8)系统通过库存系统验证要购买的商品是否有足够库存
-





5.4 OOA——用例文档(11/15)

- 例子(改进的Buy Something)
 - Basic Flow
 - (1)顾客使用ID和密码进入系统
 - (2)系统验证顾客身份,并显示个人信息窗口
 - (3)顾客提供姓名、地址、邮编和电话号码
 - (4)系统验证用户是否为老客户
 - (5)顾客选择要购买的商品和数量
 - (6)系统通过库存系统验证要购买的商品是否有足够库存
 -



5.4 OOA——用例文档(12/15)

- 例子(冗长型)
 - Use Case: Buy Something
 - Actor: Customer
 - Basic Flow
 - (1)顾客使用ID和密码进入系统
 - (2)系统验证顾客身份,并显示个人信息窗口
 - (3)顾客提供姓名
 - (4)顾客提供地址
 - (5)顾客提供邮编
 - (6)顾客提供电话号码



5.4 OOA——用例文档(13/14)

- (7)顾客选取商品
- (8)顾客确定商品的数量
- (9)系统验证用户是否为老客户
- (10)系统打开到库存系统的连接
- (11)系统通过库存系统请求当前库存量
- (12)库存系统返回当前库存量
- (13)系统验证要购买的商品是否有足够库存量
-



5.4 OOA——用例文档(14/15)

- 例子(改进的Buy Something)
 - Basic Flow
 - (1)顾客使用ID和密码进入系统
 - (2)系统验证顾客身份,并显示个人信息窗口
 - (3)顾客提供个人信息(姓名、地址、邮编、电话号码), 选择要购买的商品及数量
 - (4)系统验证用户是否为老客户
 - (5)系统使用库存系统验证要购买的商品数量是否少于库存量
 - ...



5.4 OOA——用例文档(15/15)

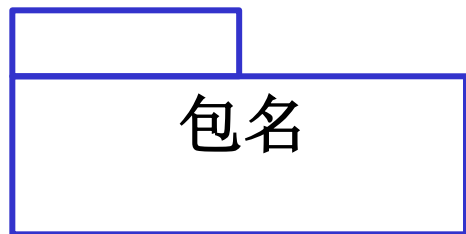
- 讨论：登录用例
 - 1. 系统显示输入用户名和密码的界面
 - 2. 会员输入用户名和密码，点击“确定”
 - 3. 如果用户名和密码正确，系统根据用户名从数据库“会员”表中查询该会员信息，系统显示会员定制界面
 - 4. 如果用户名不存在....
- 以上描述用例的文字有什么问题？



5.4 OOA——用例的组织(1/2)

- 为了降低复杂程度，系统可被进一步划分成不同的主题
 - 从功能分解角度来确定主题。
 - 按照使不同主题内的对象相互间依赖和交互最少的原则来确定主题。
 - 主题可以采用UML中的包来展现。用包来组织用例？

- 包

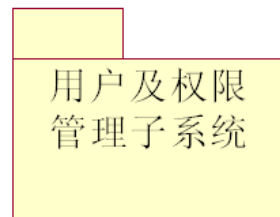
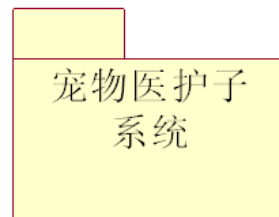
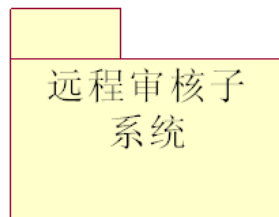
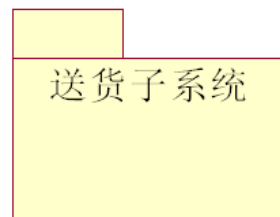
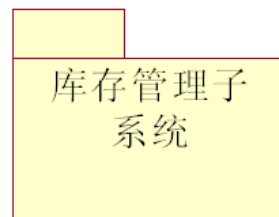
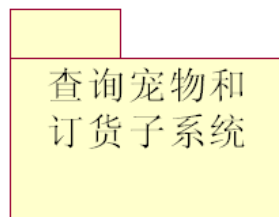




5.4 OOA——用例的组织(2/2)

- 分包方案

- 按主题分包
- 按Actor分包
- 按开发团队分包
- 按发布情况分包
- 混合方案：先主题，后团队/发布情况



- 优先级



5.4.2 OOA方法——CRC建模(1/3)

- 含义

- Class-Responsibility-Collaborator, 类-责任-协作者
- 识别候选类并指明它们的责任和协作
- 实际上是一组表示类的标准的索引卡片集合
 - 卡片的顶部：类的名字、类型和特征
 - 卡片的左边：列出类的责任
 - 卡片的右边：列出协作者



5.4.2 OOA方法——CRC建模(2/3)

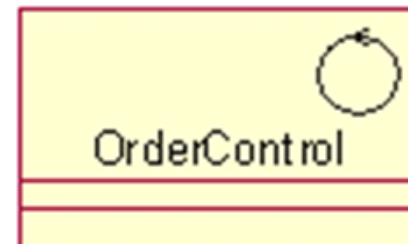
—卡片图例

类名:	
类的类型: (如: 设备、性质、角色、事件, ……)	
类的特征: (如: 有形的、原子的、并发的, ……)	
责任:	协作者:



5.4.2 OOA方法——CRC建模(3/3)

- 筛选出正确的候选类
 - 去掉不正确的、不必要的，仅保留确实应该记录其信息或需要其提供服务的那些类。
- 候选类的类型
 - 外部实体、事务、发生的事情或事件、角色、位置
 - 边界类、实体类、控制类





5.4.3 OOA方法——对象-关系建模 (1/14)

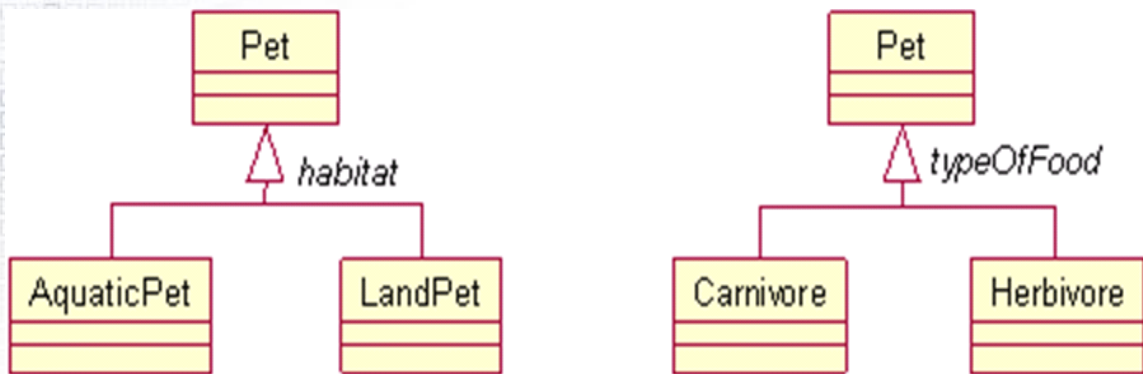
- 关系类型
 - 泛化（Generalization）关系
 - 由一个超类和几个直接子类构成的两层结构
 - 关联（Association）关系
 - 是模型元素间的一种语义联系
 - 聚集（Aggregation）和合成（Composition）关系
 - 是一种特殊的关联，表示类与类之间整体与部分的关系
 - 是一种强聚集，组合关系中的类具有相同的生存期
 - 依赖（Dependency）关系
 - 是类与类之间较弱的关系



5.4.3 OOA方法——对象-关系建模

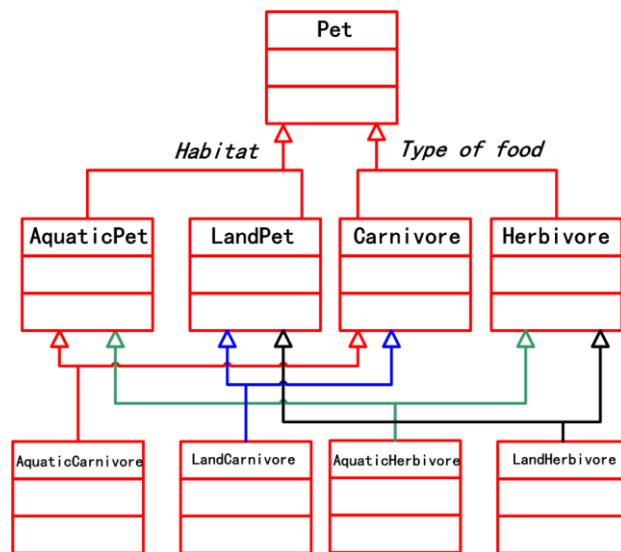
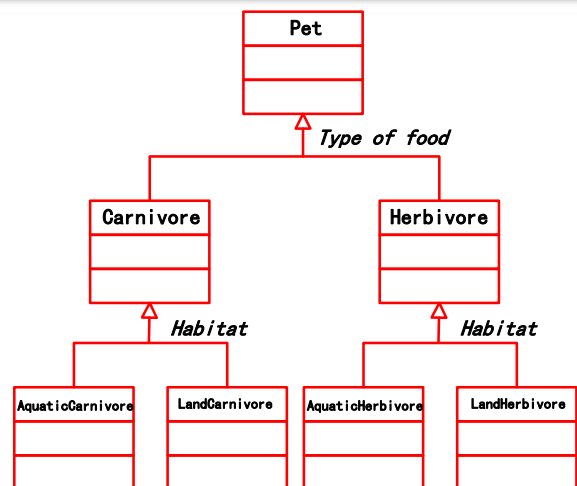
(2/14)

- 泛化?
- 为什么要泛化?
- 例子1



宠物的两个使用不同鉴别器的基本继承层次结构

?



单继承、多继承、多层泛化?



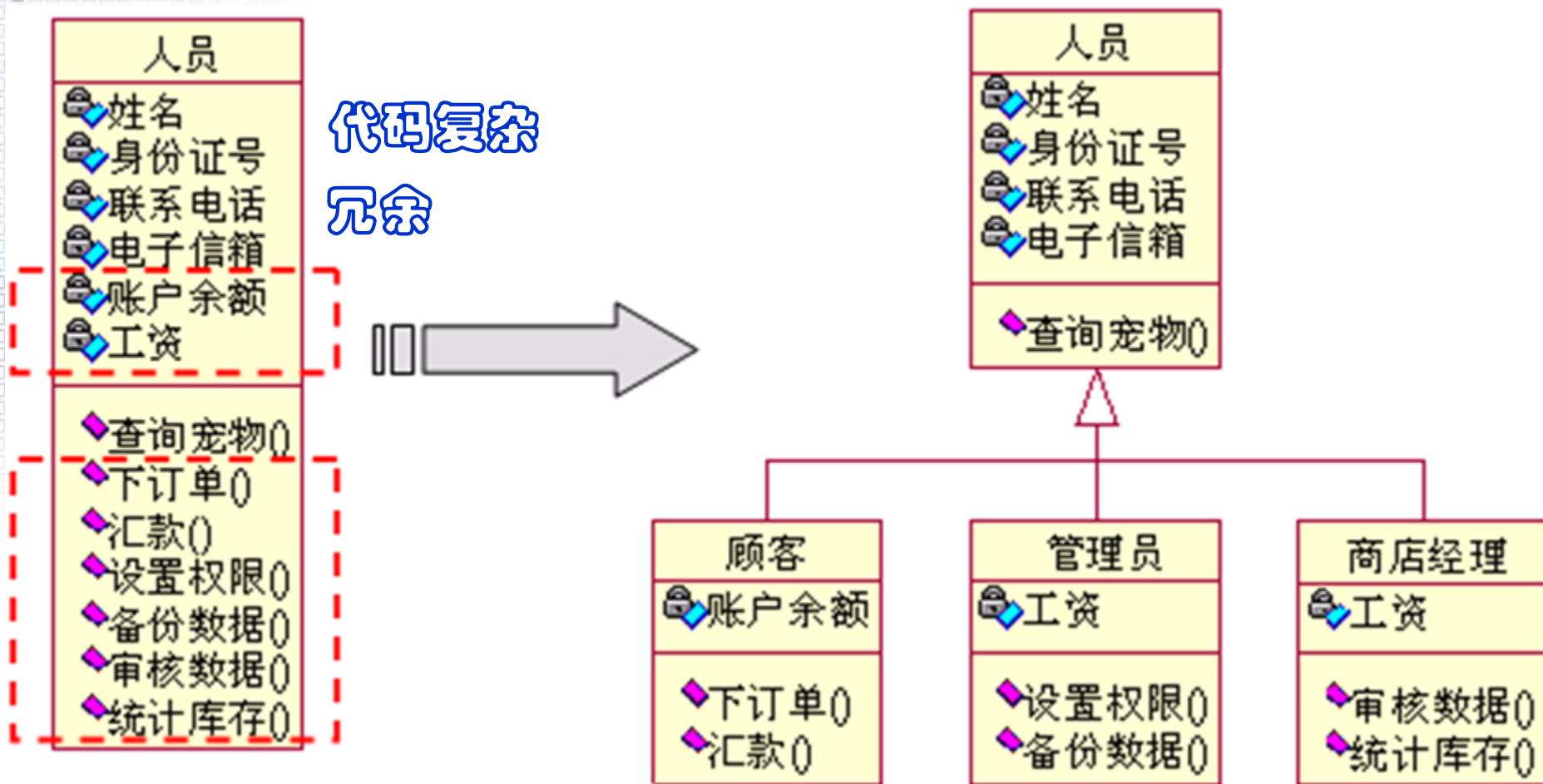
5.4.3 OOA方法——对象-关系建模 (3/14)

- 如何识别泛化关系？
- 根据领域知识识别泛化
 - 零件 ◀—— 螺钉 ◀—— 六角螺钉
 - 商品 ◀—— 电器 ◀—— 家用电器 ◀—— 洗衣机
 - 生物 ◀—— 动物 ◀—— 哺乳动物 ◀—— 灵长目



5.4.3 OOA方法——对象-关系建模 (4/14)

- 采用自上而下识别泛化
- 例子2

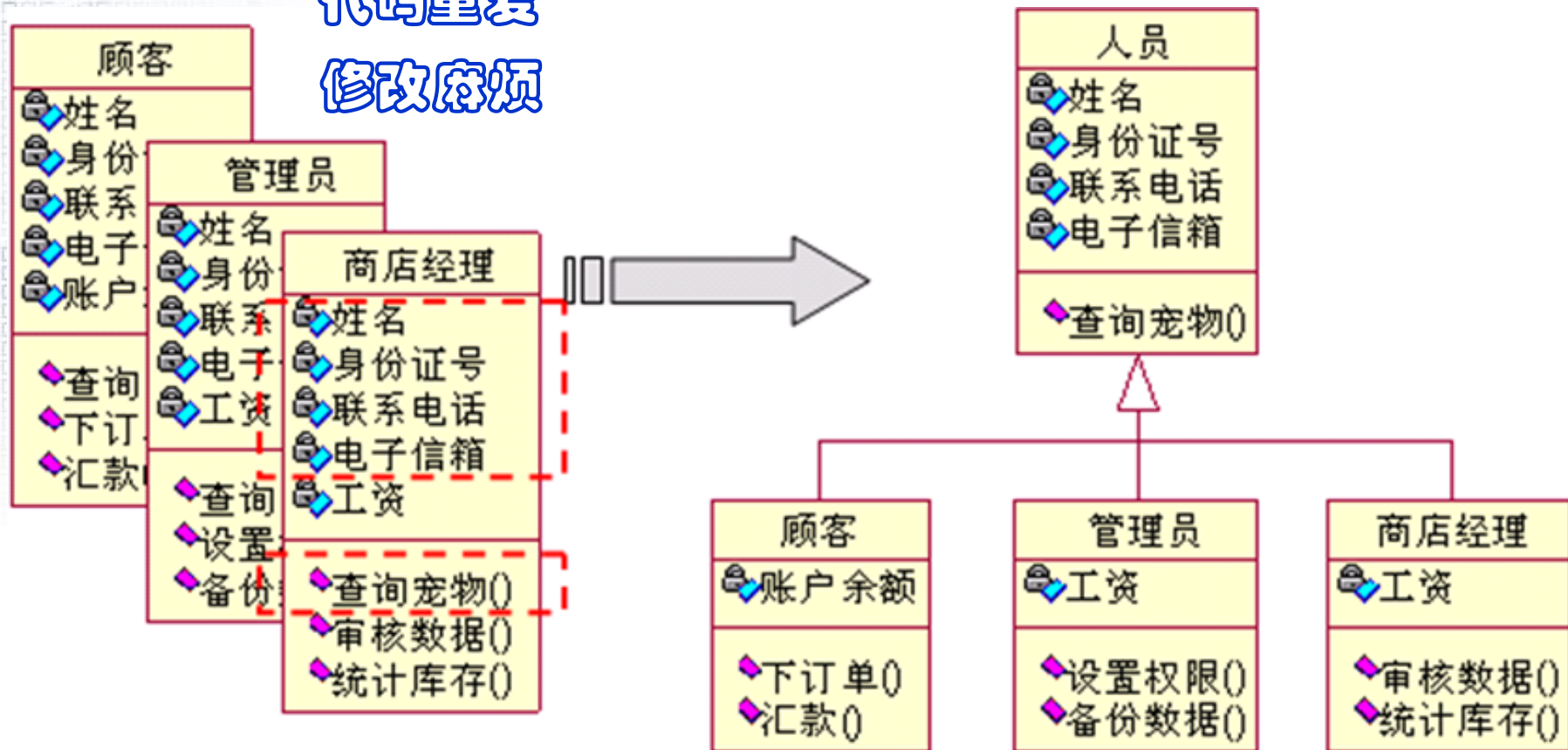




5.4.3 OOA方法——对象-关系建模 (5/14)

- 采用自下而上识别泛化
- 例子3

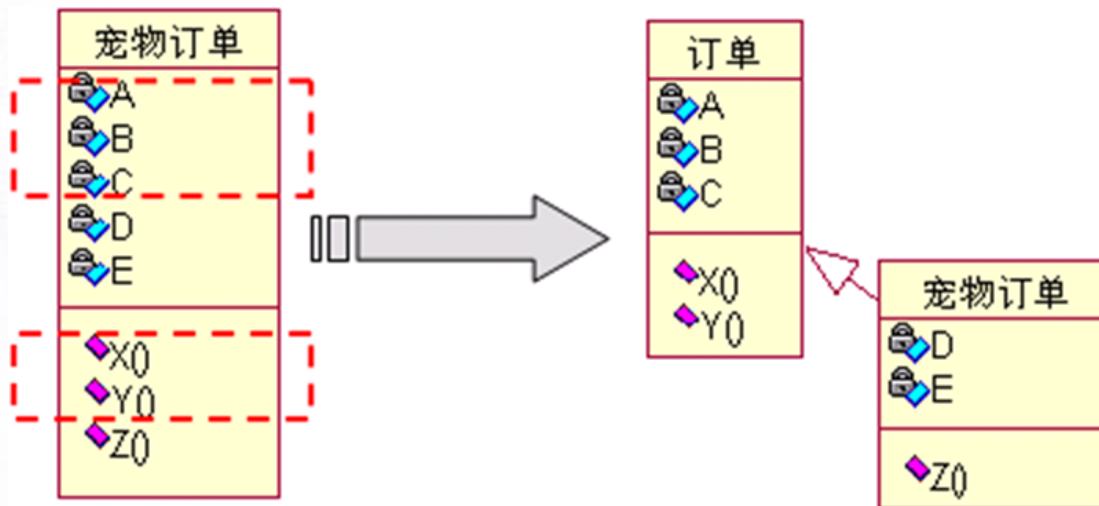
代码重复
修改麻烦





5.4.3 OOA方法——对象-关系建模 (6/14)

- 考虑领域内复用的例子4



- 系统是否有此需求而泛化的例子5
 - 人员-用户-经理-管理员
 - 用户-会员-普通用户
 - 会员-一级-二级-三级.....



5.4.3 OOA方法——对象-关系建模 (7/14)

- 是否处于同一业务范围
- 是否在结构上真正相似
- 子类的区别
- 例子5





5.4.3 OOA方法——对象-关系建模 (8/14)

- 关联关系：
 - 是模型元素间的一种语义联系
 - 类之间的关联关系用一条把类连接在一起的实线来表示
- 例子6



双向关联



单向关联

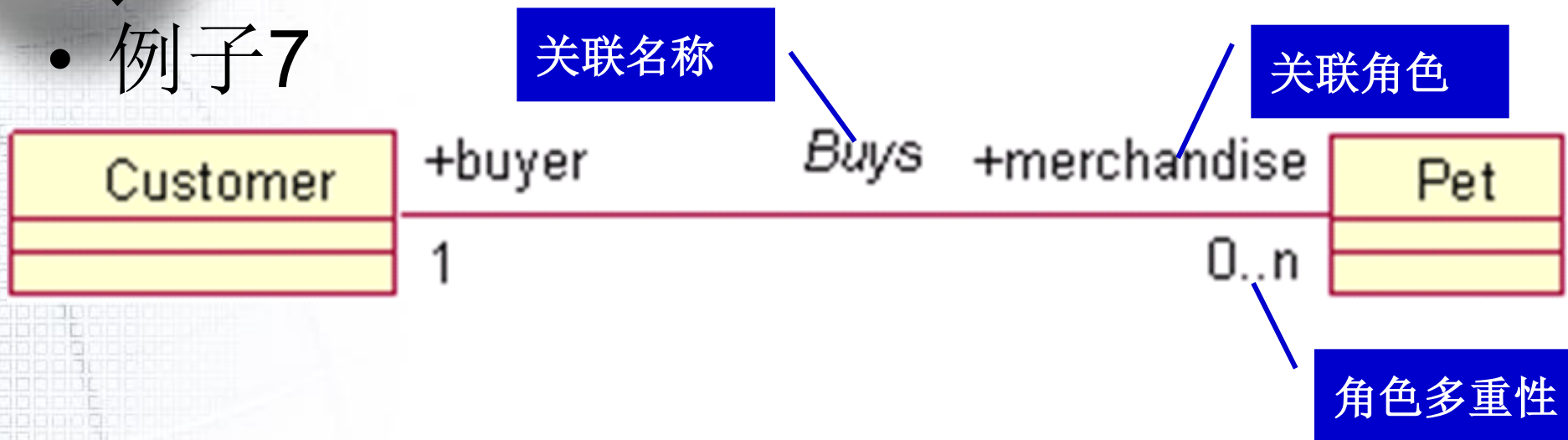
双向关联和单向关联的代码比较

类 A 和 B 的代码框架	类 C 和 D 的代码框架
<pre>public class A { public B theB; /** * @roseuid 44F10C5F003E */ public A() { } }</pre>	<pre>public class C { public D theD; /** * @roseuid 44F10A79033C */ public C() { } }</pre>
<pre>public class B { public A theA; /** * @roseuid 44F107C003C8 */ public B() { } }</pre>	<pre>public class D { /** * @roseuid 44F10A7A009C */ public D() { } }</pre>



5.4.3 OOA方法——对象-关系建模 (10/14)

• 例子7



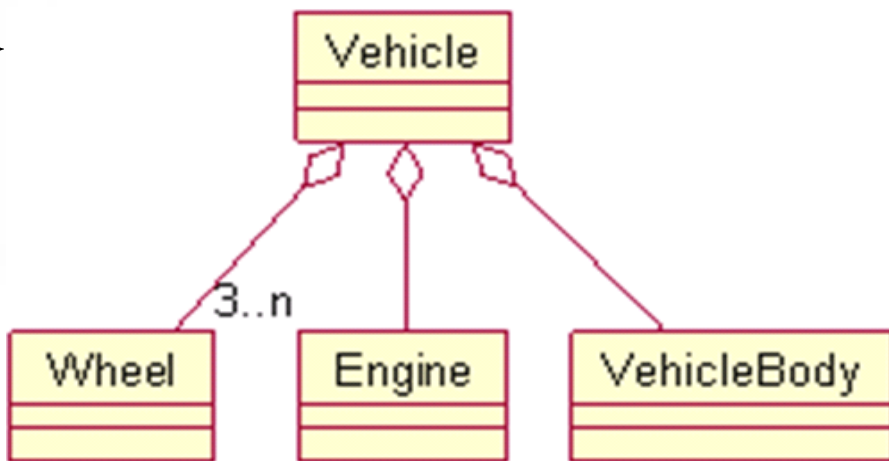


5.4.3 OOA方法——对象-关系建模 (11/14)

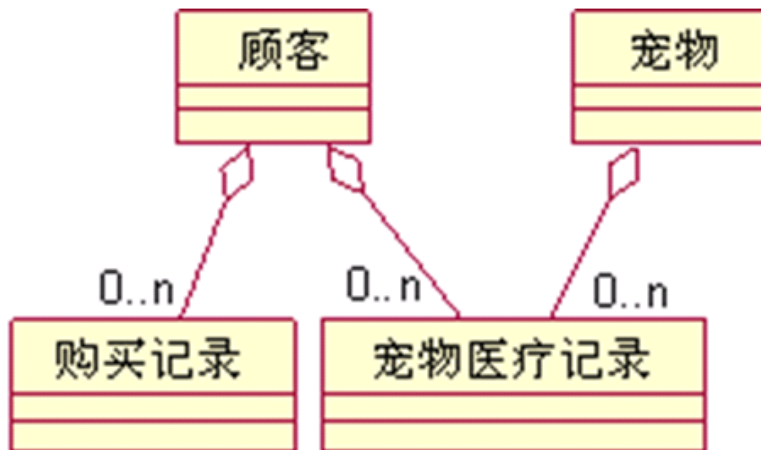
- 聚集：

- 一种特殊的关联，用来表示类和类之间整体和部分

- 例子8



- 例子9





5.4.3 OOA方法——对象-关系建模 (12/14)

- 合成关系
 - 一种更特殊的聚集，用来表示类和类之间整体和部分的关系
 - 特殊之处：共生共存

- 例子10





5.4.3 OOA方法—对象-关系建模 (13/14)

- 依赖关系

- 类和类之间较弱的关联
- 假设有两个元素X、Y，如果修改X的定义可能会导致对另一个元素Y的定义的修改，则称元素Y依赖于元素X



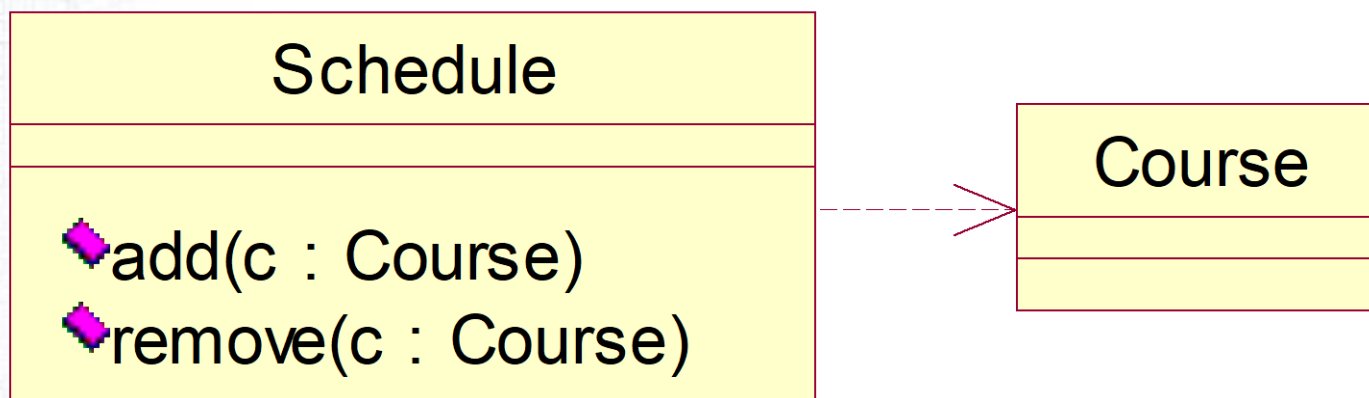
- 使用依赖关系的原因

- 一个类向另一个类发送消息
- 一个类是另一个类的数据成员类型
- 一个类是另一个类的操作的参数类型



5.4.3 OOA方法—对象-关系建模 (14/14)

- 使用依赖关系的原因(续)



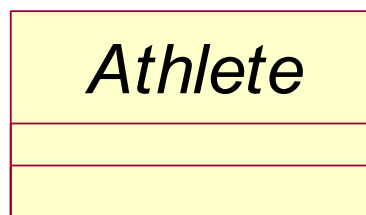
- 与关联关系的区别
 - 如果两个类之间有关联关系，那么它们之间就也就有依赖关系，但一般只要表示关联关系即可



5.4.3 OOA方法—对象-关系建模补充 (1/7)

- 抽象类

- 不能直接产生实例的类
- 其方法往往只是一些声明，没有具体实现

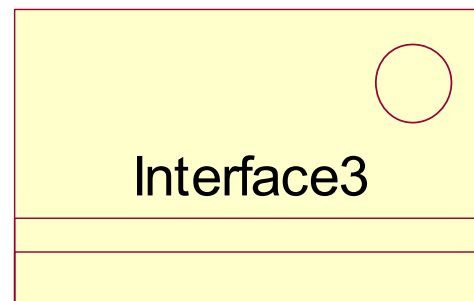
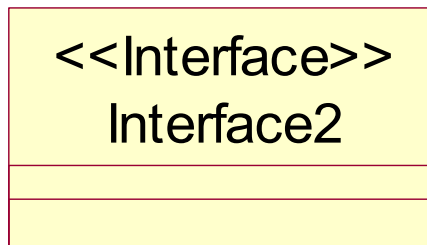


- 接口

- 不包含属性，只包含方法的声明
- 与抽象类区别
 - 不含属性
 - 没有方法实现



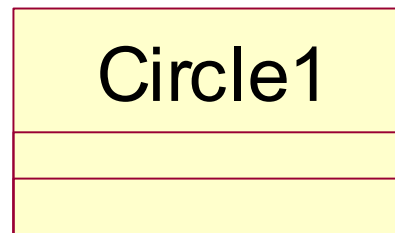
Interface1





5.4.3 OOA方法—对象-关系建模补充 (2/7)

- 类图概念
 - 类加上它们之间的关系就构成类图
 - 类图中可以包含接口、包、关系等建模元素，也可以包含对象、链等实例
- 类图的抽象层次
 - 概念层
 - 描述应用领域中的概念，不考虑实现问题，独立于具体的程序设计语言





5.4.3 OOA方法—对象-关系建模补充 (3/7)

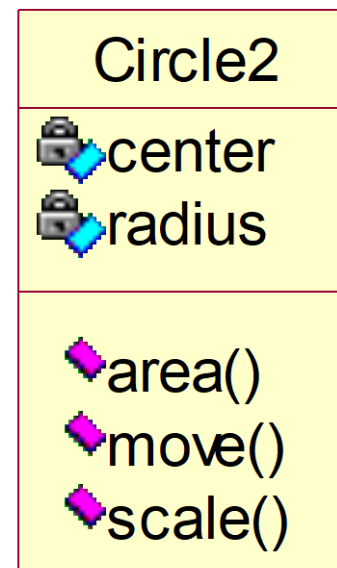
- 类图的抽象层次

- 说明层

- 描述软件的接口部分，而不是实现部分
 - 接口可能因为实现环境、运行特性或开发商不同而有多种不同的实现

- 实现层

- 描述类的实现问题，提供实现细节





5.4.3 OOA方法—对象-关系建模补充 (4/7)

- 类图的抽象层次
 - 从概念层到实现层的过渡是一个渐进的过程

Circle1

Circle2

 center
 radius

 area()
 move()
 scale()

Circle3

 center : float
 radius : float

 Circle()
 <<query>> area() : float
 <<update>> move(loaction : Point) : void
 <<update>> scale(ratio : float) : void



5.4.3 OOA方法—对象-关系建模补充 (5/7)

- 构造类图

- 寻找类的技巧

- 根据用例描述确定类的候选者
 - CRC(Class, Responsibility, Collaboration)分析法
 - 根据边界类、控制类和实体类的划分
 - 对领域进行分析，或利用已有的领域分析结果
 - 参考设计模式
 - 根据某些软件开发过程提供的指导原则，如RUP



5.4.3 OOA方法—对象-关系建模补充 (6/7)

- 构造类图

- 构造策略

- 不要试图使用所有符号
 - 不要过早陷入实现细节
 - 对于构造好的类图，应考虑模型的目标是否达到，粒度是否合适



5.4.3 OOA方法—对象-关系建模补充 (7/7)

- 构造类图

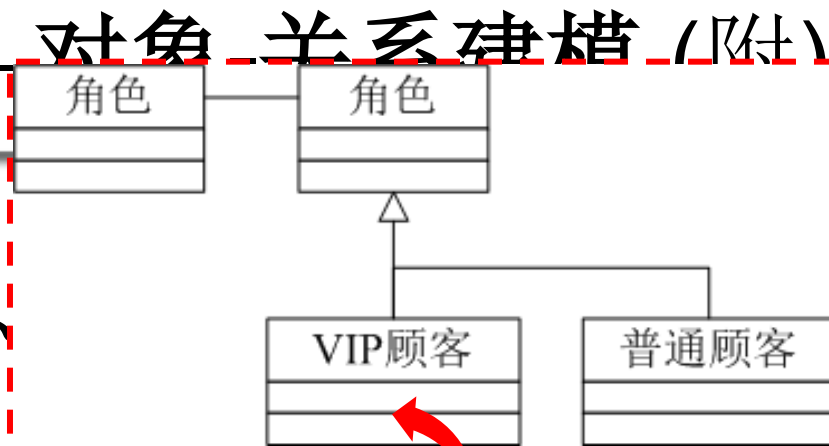
- 构造步骤

- 1)分析问题领域，确定系统需求
 - 2)确定类，明确类的含义和职责，确定属性操作
 - 3)确定类之间关系
 - 4)调整和细化类和类之间的关系
 - 命名冲突、功能重复、复用等
 - 5)绘制类图并增加相应说明

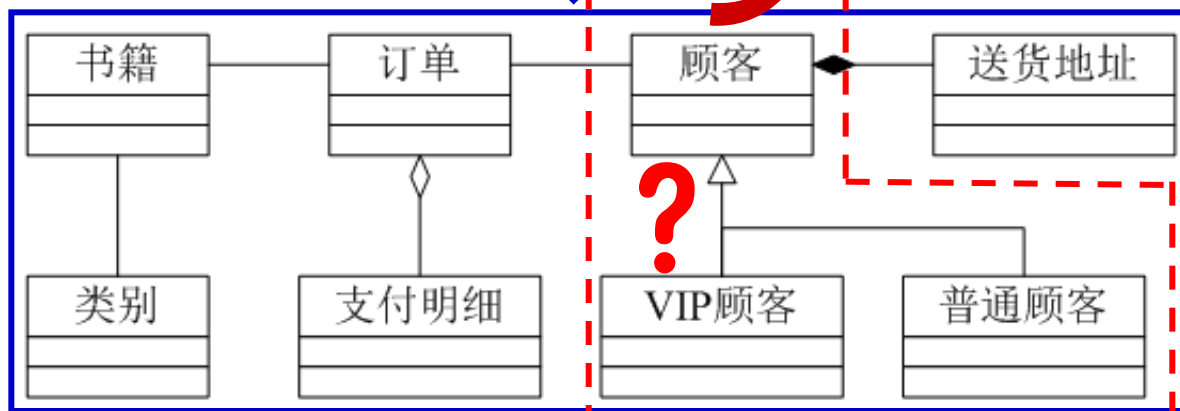
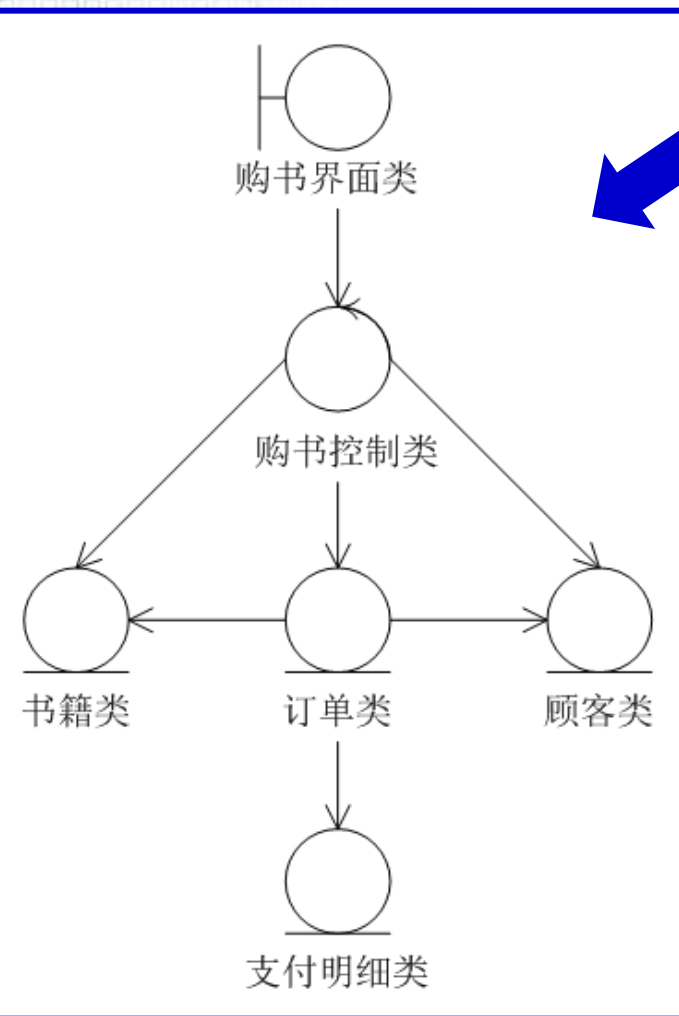


5.4 OOA方法——对象关系建模 (附)

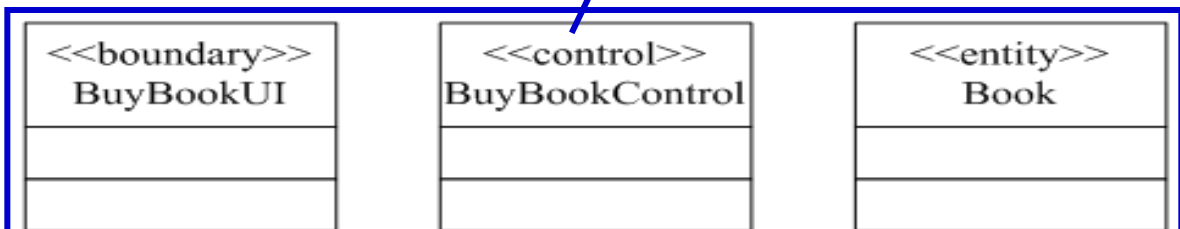
- 网上书店参与类类图、



采用玩家-角色模式进一步优化



构造型





5.4.4 OOA方法——对象-行为建模

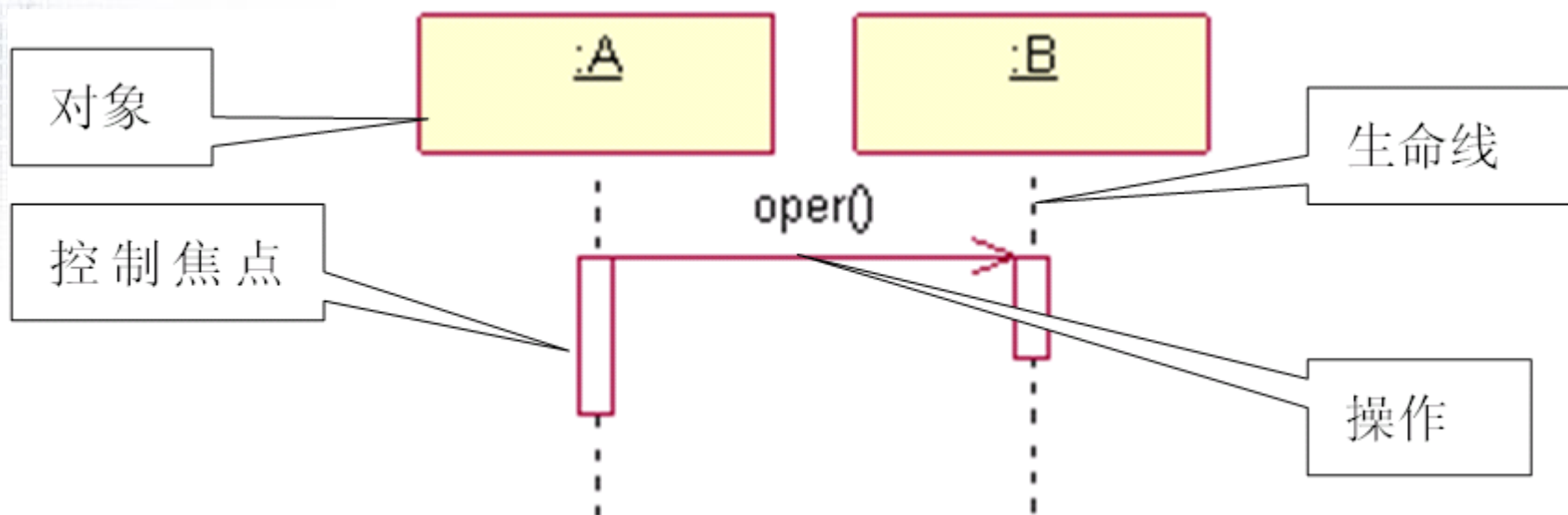
- 交互图（Interaction Diagram）
 - 顺序图（时序图）
 - 活动图
 - 状态图



5.4.4 OOA方法——对象-行为建模 (1/8)

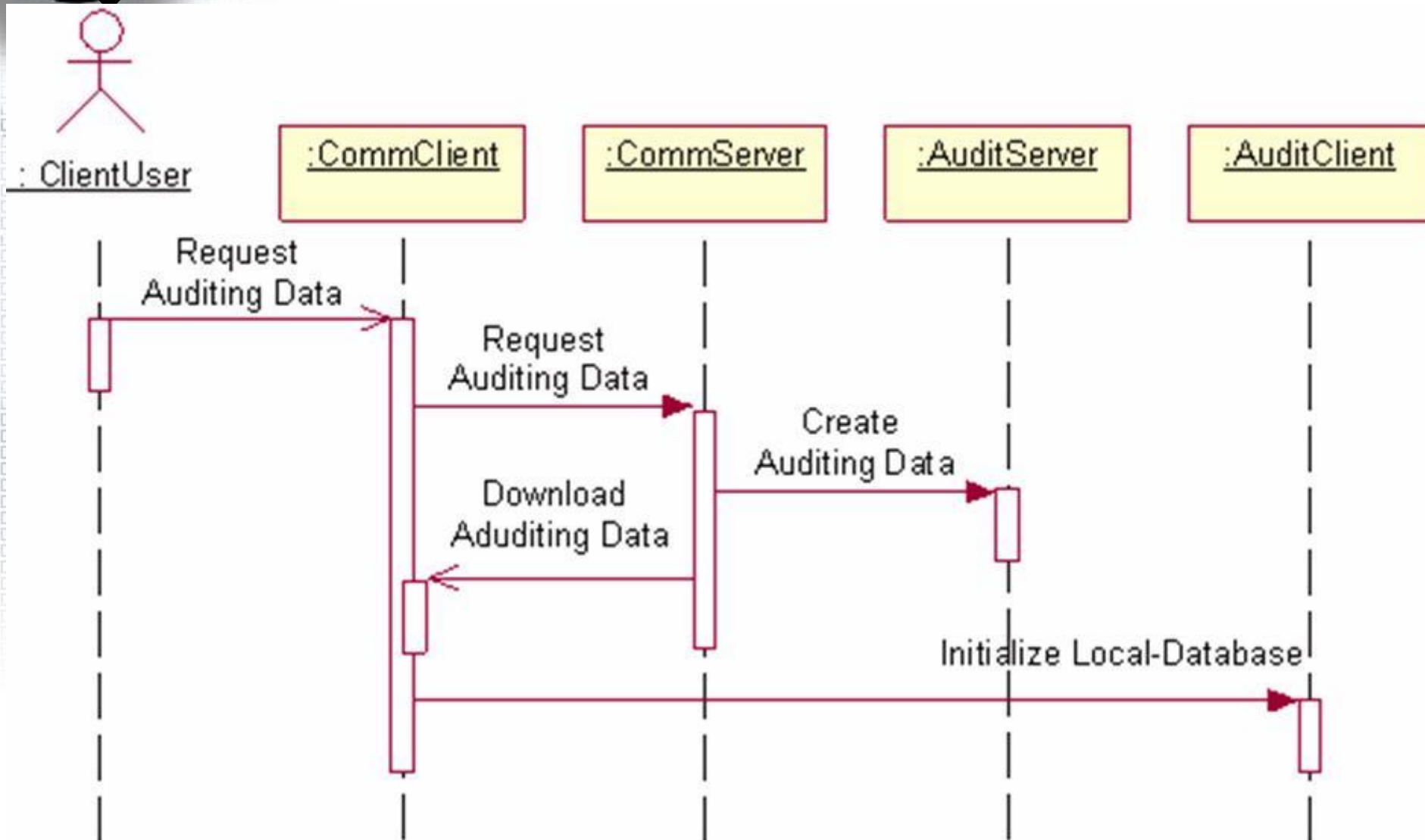
- 交互图（Interaction Diagram）
- 时序图

—是显示对象之间交互的图，这些对象是按照时间顺序排序的





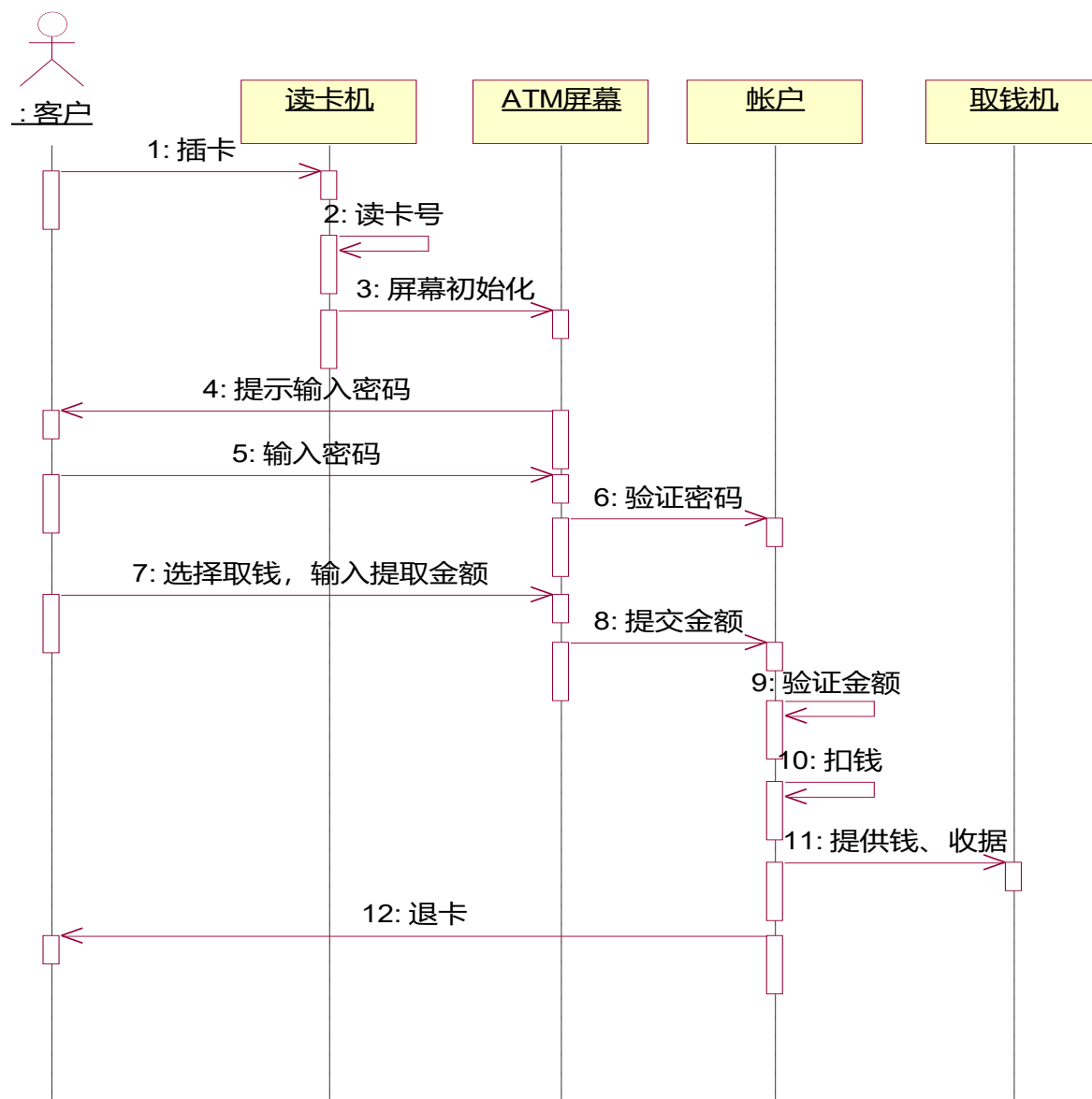
5.4.4 OOA方法——对象-行为建模 (2/8)





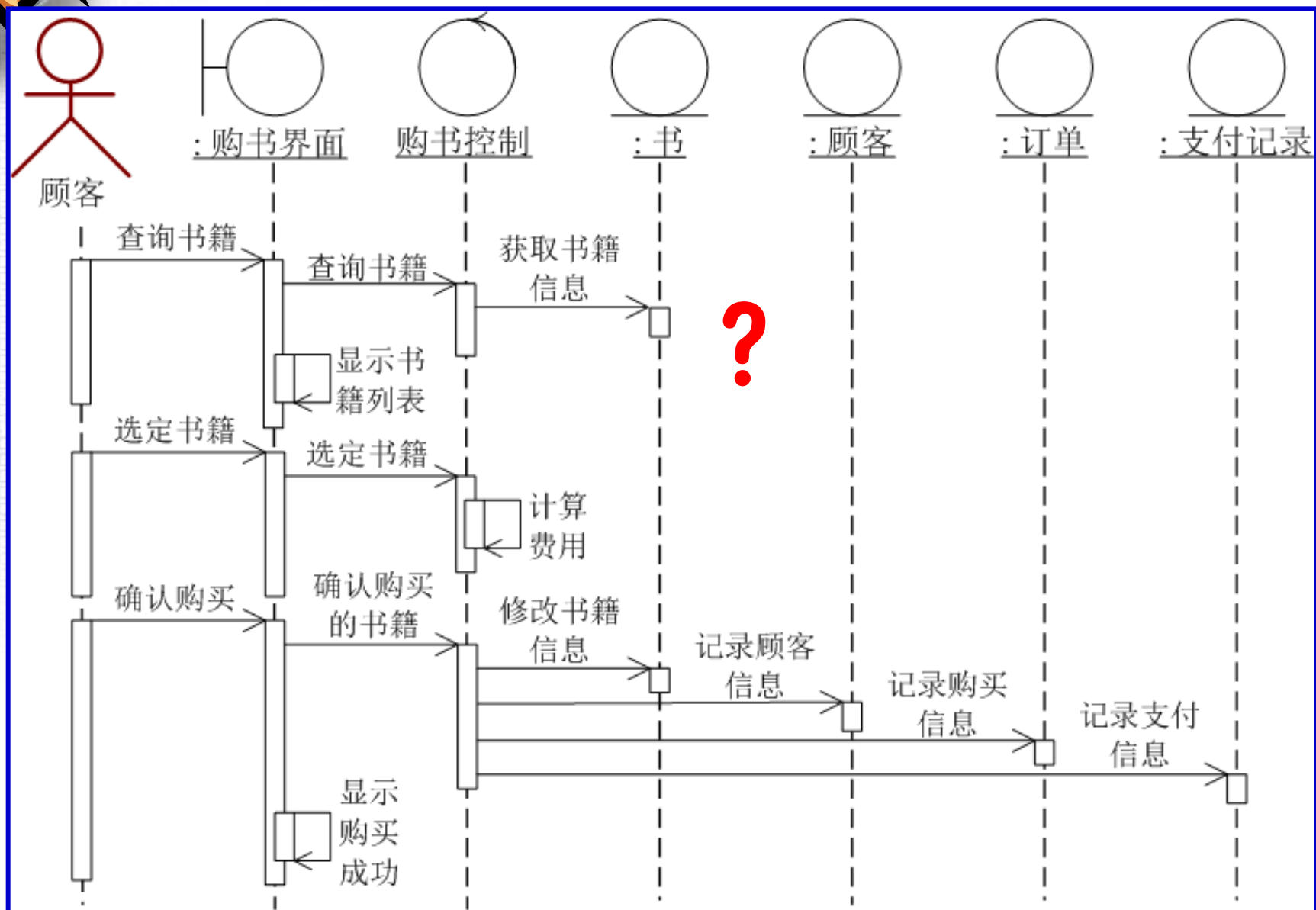
5.4.4 OOA方法—对象-行为建模 (3/8)

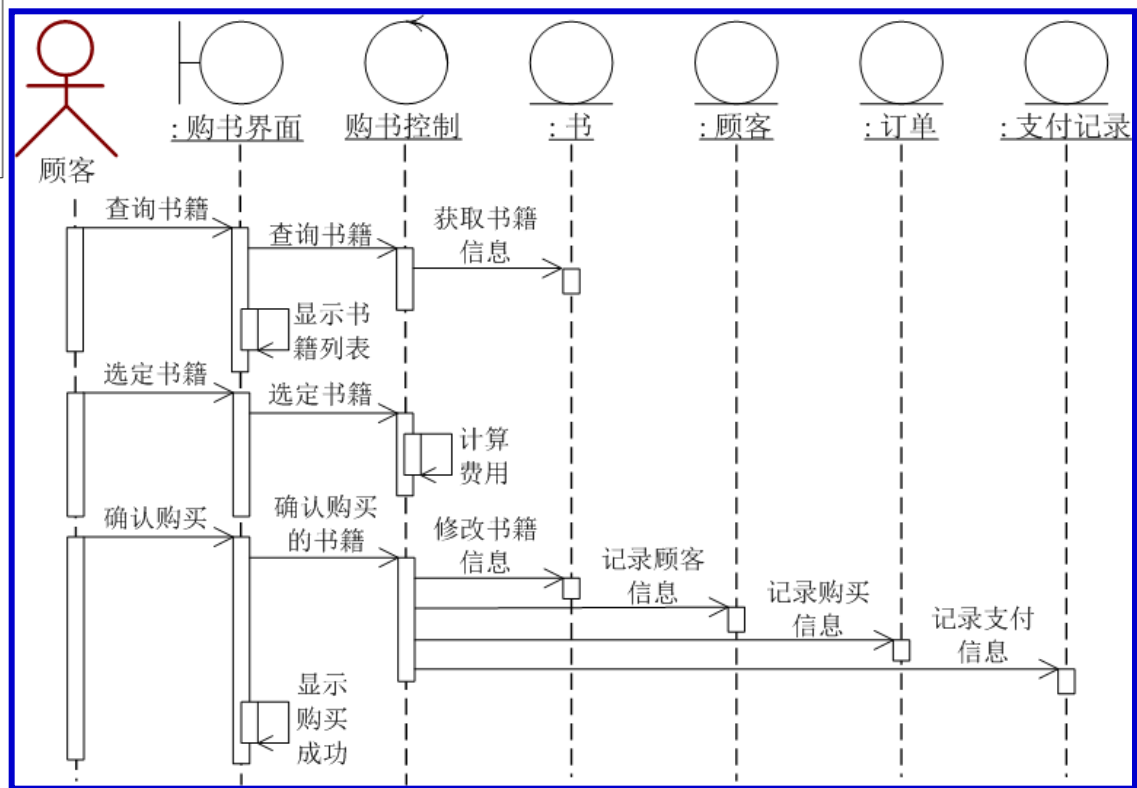
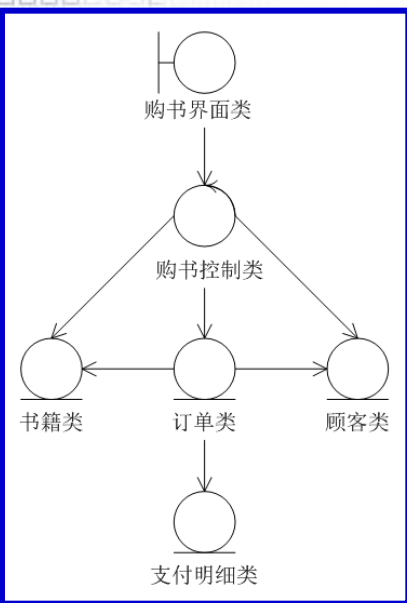
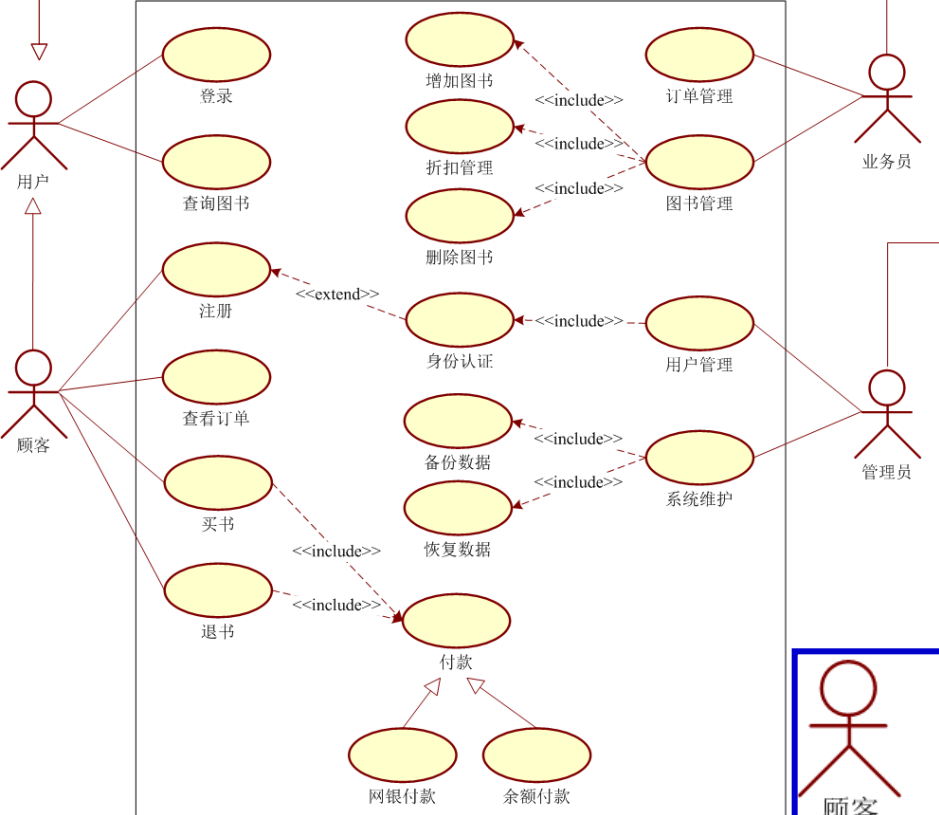
• 取款用例





5.4.4 OOA方法——对象-行为建模 (4/8)

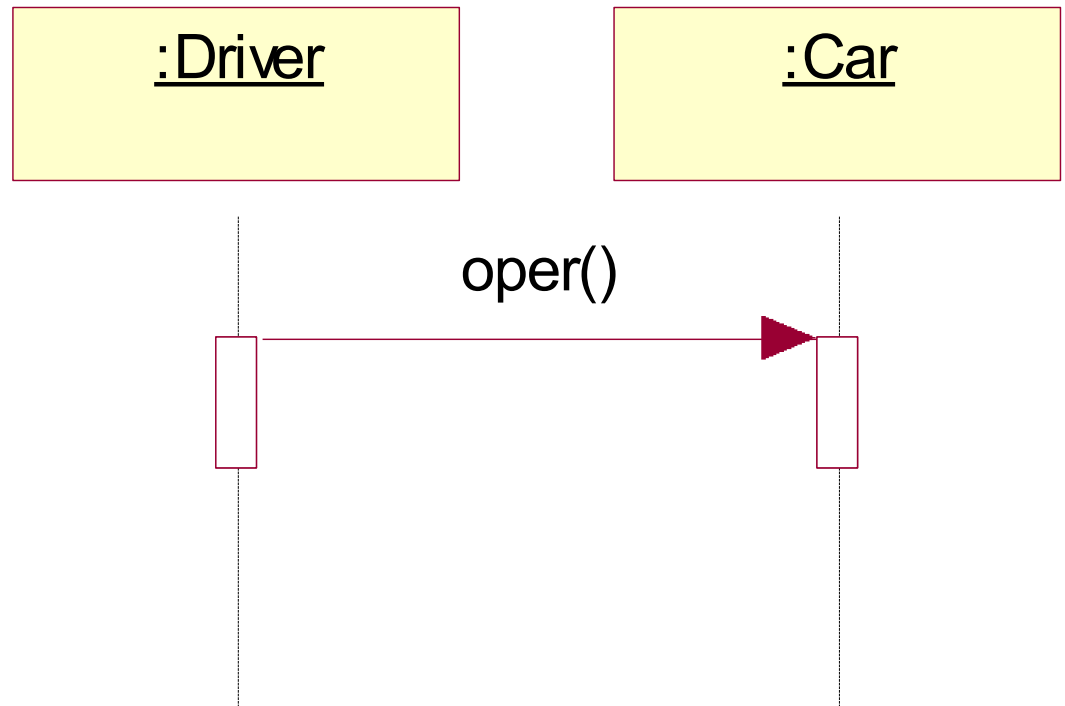






5.4.4 OOA方法—对象-行为建模 (5/8)

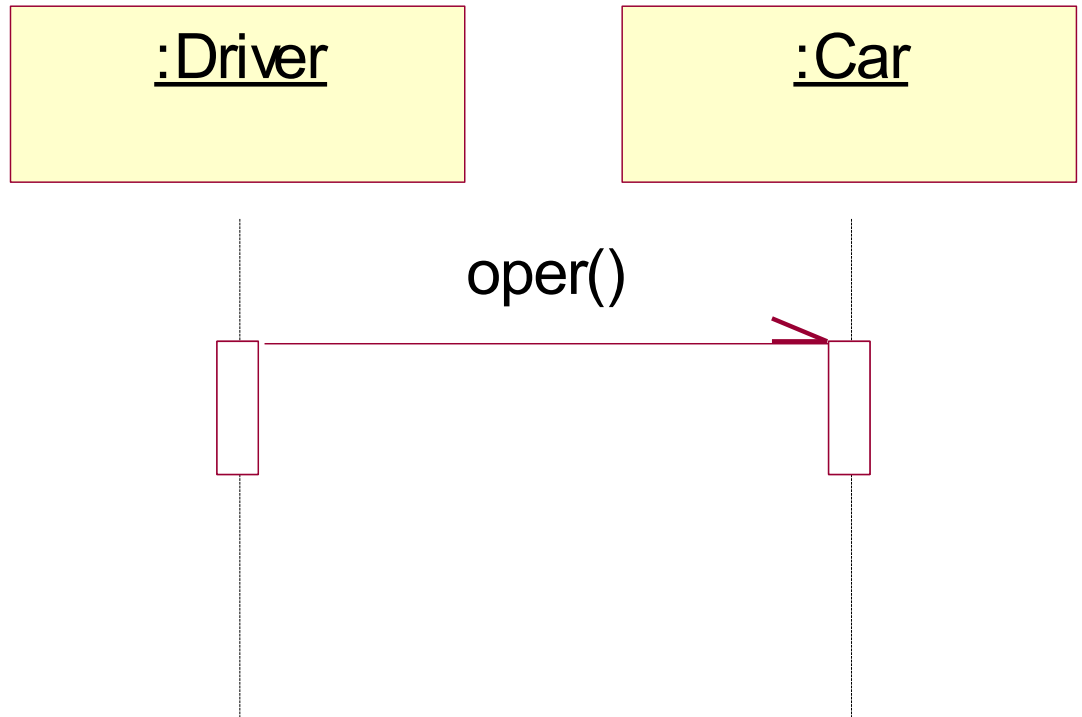
- 调用消息
 - 同步
 - 消息驱动
 - 返回消息





5.4.4 OOA方法—对象-行为建模 (6/8)

- 异步消息





5.4.4 OOA方法—对象-行为建模 (7/8)

- 问题： 图中的对象如何确定？
 - 分析用例描述；
 - 参考类图；
- 问题： 时序图是一气呵成吗？
 - 分析阶段，消息名可能只是一个说明
 - 设计阶段，消息名将会细化，最终对应到类图的方法中



5.4.4 OOA方法—对象-行为建模 (8/8)

- 问题：图是一步画成的吗？

—初始：

- 关注客户关心的高级信息，消息暂时不映射操作，对象暂时不映射类
- 作用：让涉众了解系统逻辑流程

—精化：

- 客户同意第一步流程后，小组加入更多的细节
- 作用：对客户作用不大，对开发、测试等人员有作用



5.4.4 OOA方法—活动图 (1/7)

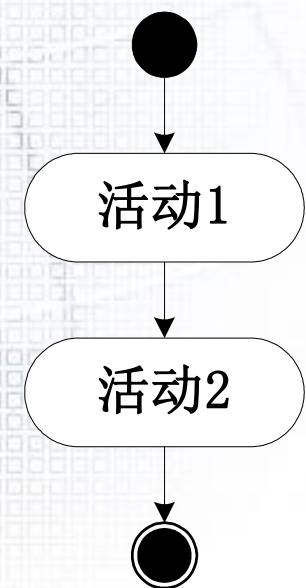
- 概念

- 用于描述系统的工作流程和并发行为
- 对系统的工作流程建模
 - 例： 用活动图对工作流程建模
- 活动： 表示某流程中的任务的执行， 可以表示某算法过程中语句的执行

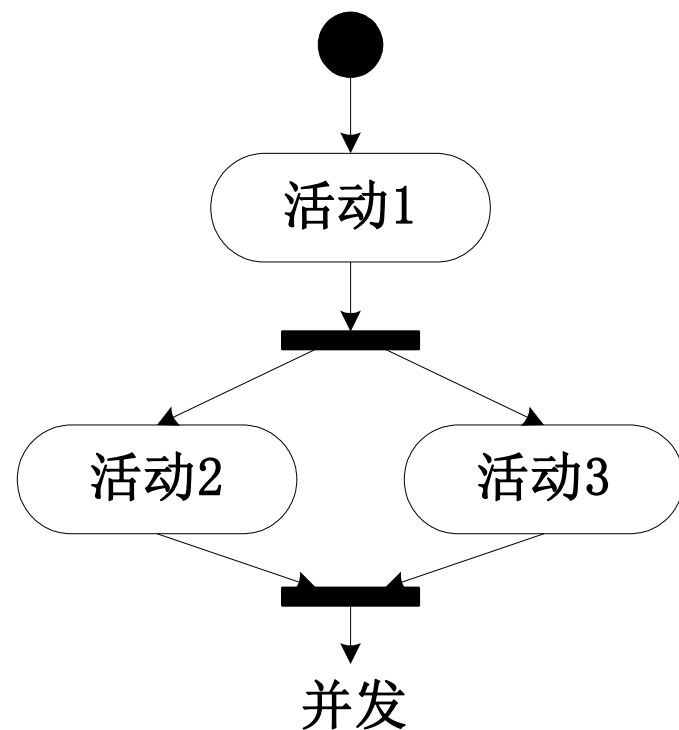
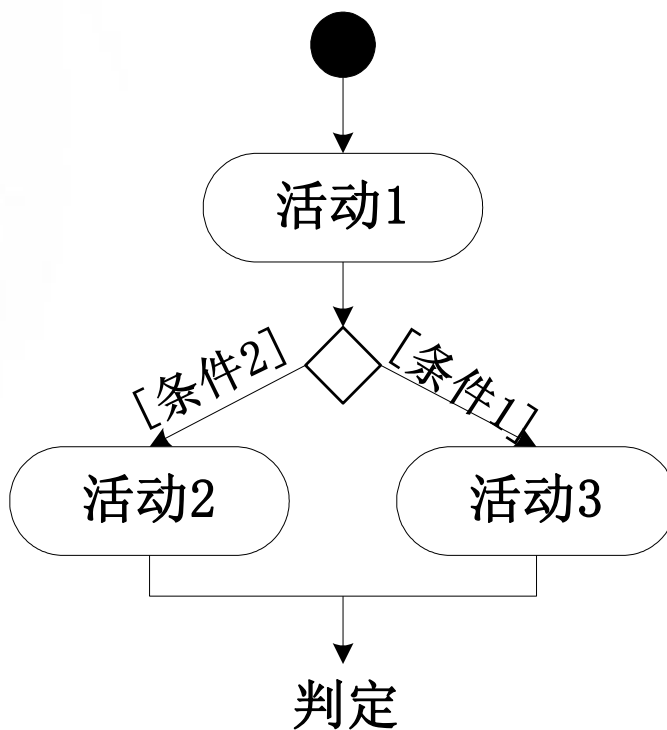


5.4.4 OOA方法—活动图 (2/7)

- 基本图形元素



起点、终点、
活动和转移

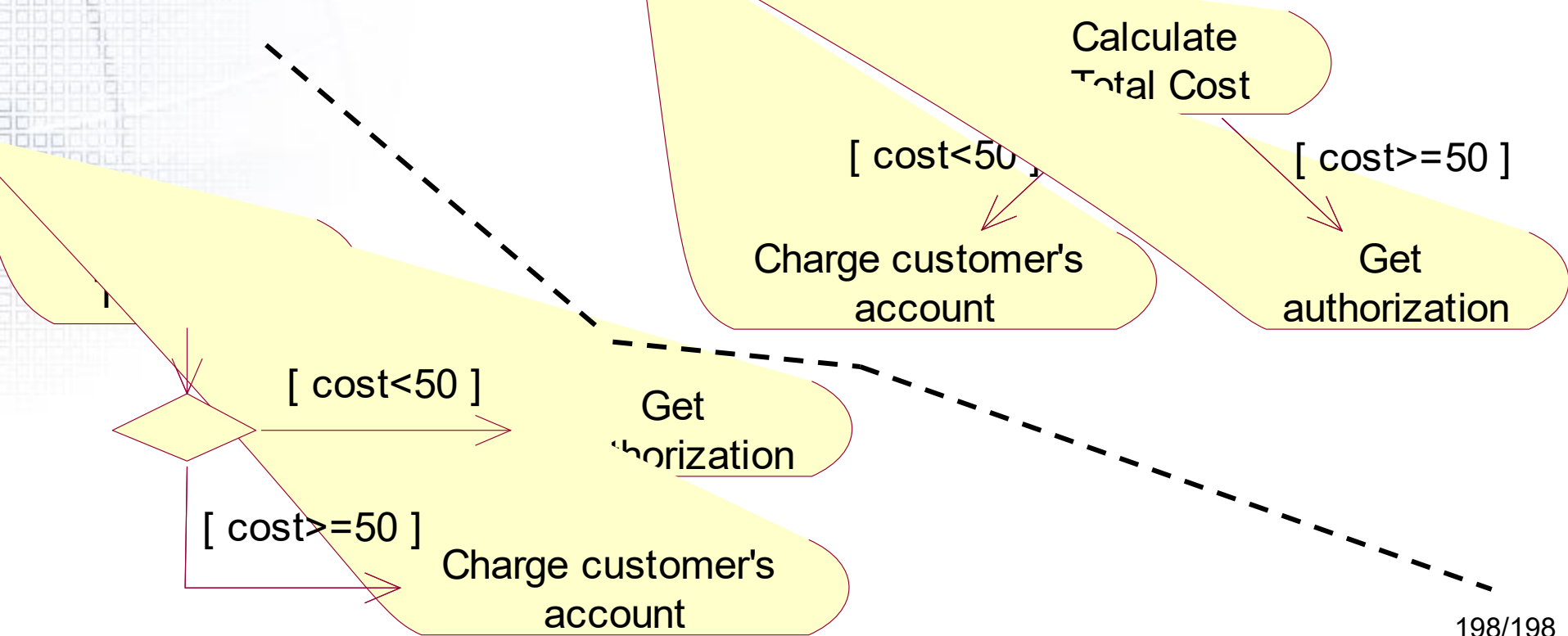




5.4.4 OOA方法—活动图 (3/7)

- 基本图形元素——分支

—对同一个触发事件，可根据不同警戒条件转向不同的活动，每个可能的转移是一个分支





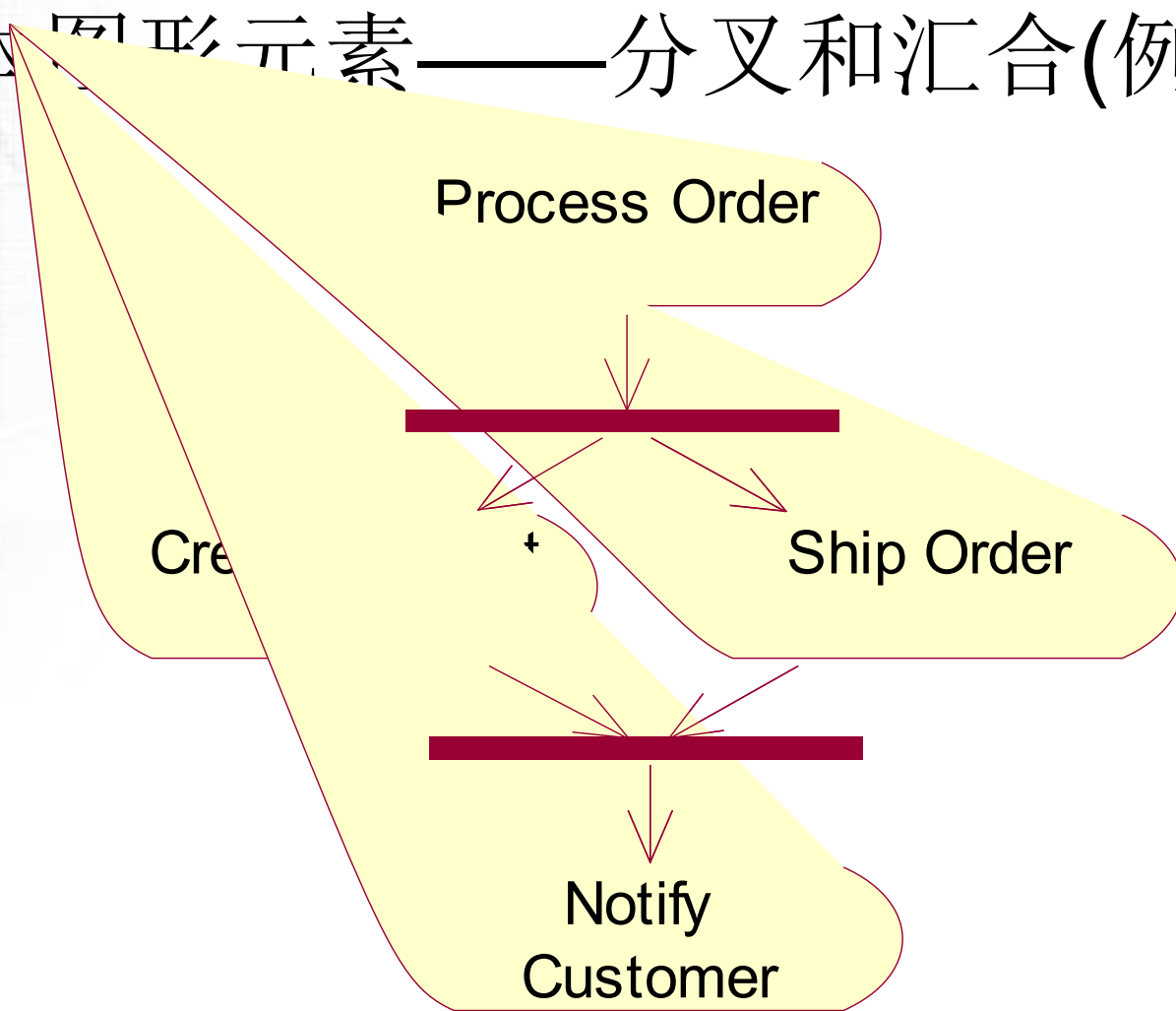
5.4.4 OOA方法—活动图 (4/7)

- 基本图形元素——分叉和汇合
 - 表示系统或对象的并发行为
 - 分叉：
 - 一个控制流被两个或多个控制流代替
 - 并发
 - 汇合：
 - 两个或多个控制流被一个控制流代替



5.4.4 OOA方法—活动图 (5/7)

- 基本图形元素——分叉和汇合(例)





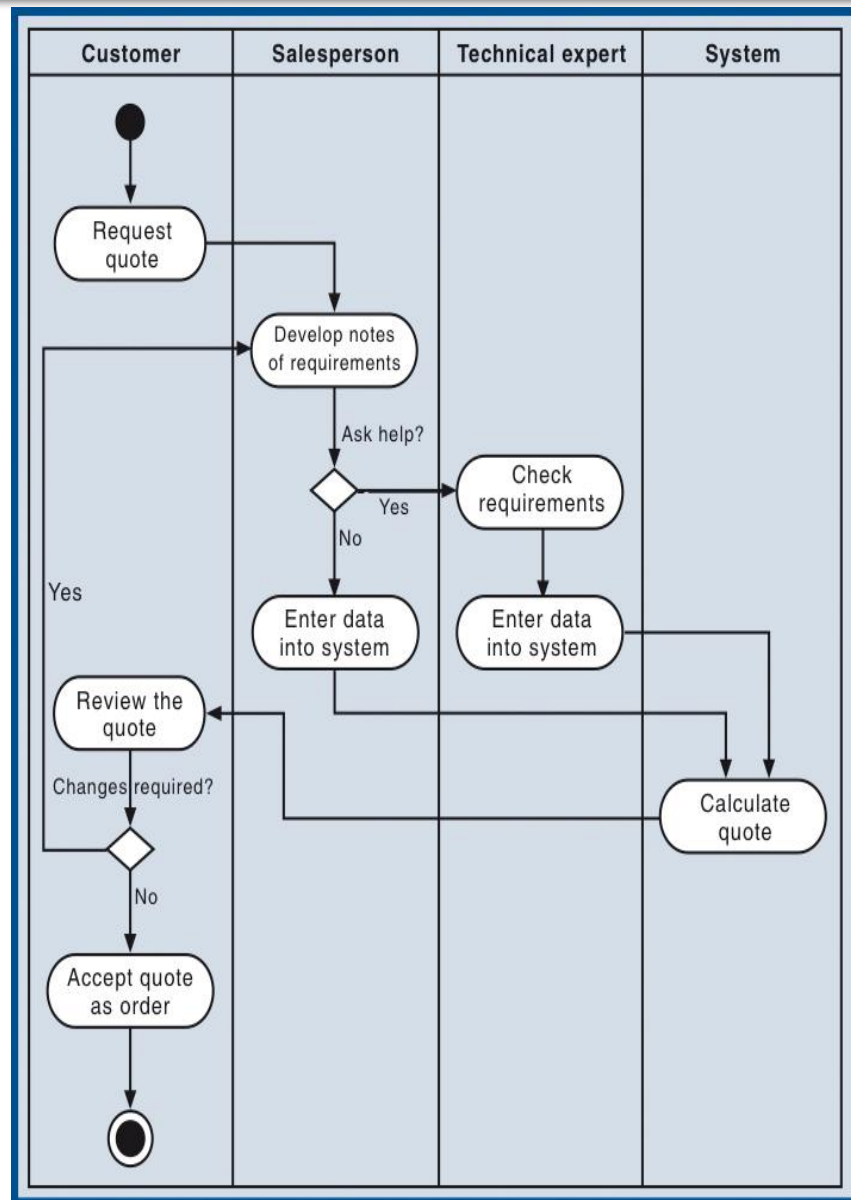
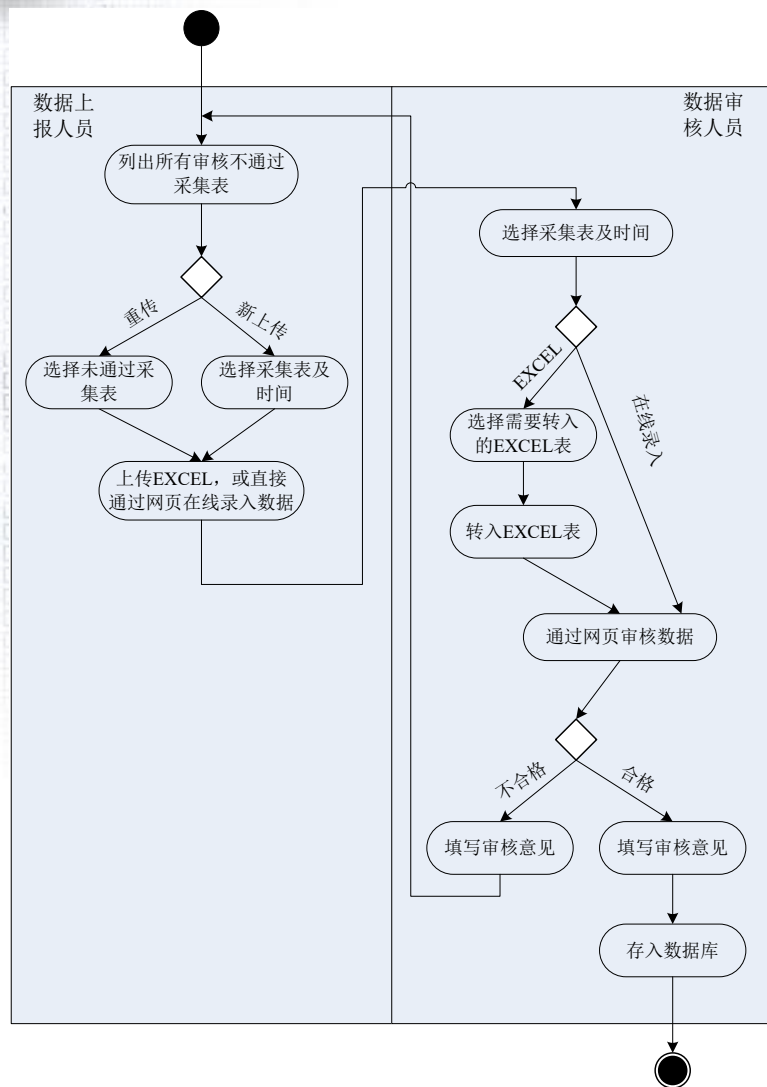
5.4.4 OOA方法—活动图 (6/7)

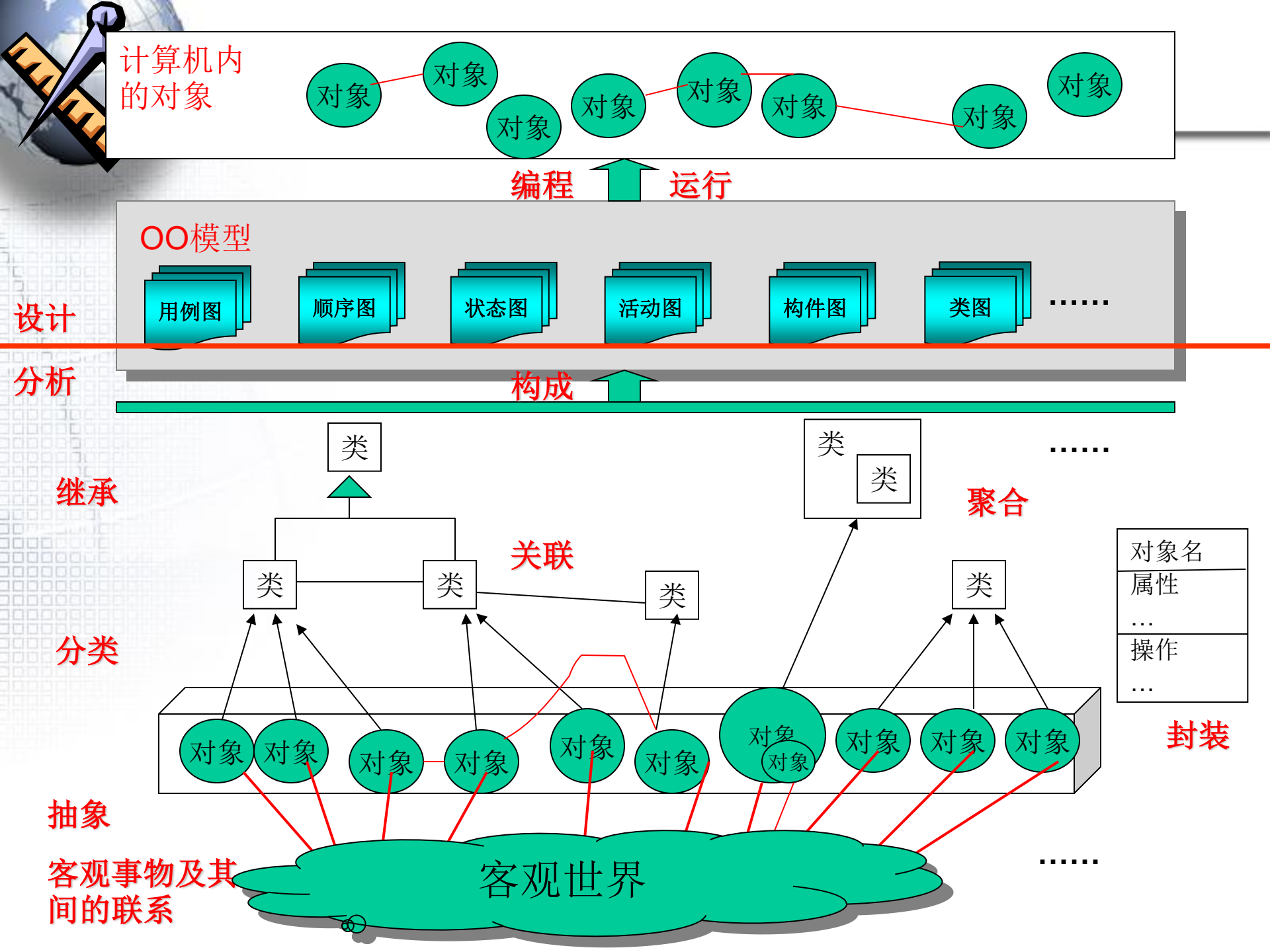
- 泳道
 - 活动图中的区域划分，根据活动职责划分
 - 每个泳道代表一个责任区
 - 泳道和类不是一一对应
 - 泳道关心的是其所代表的职责
 - 一个泳道可能由1或n个类实现



5.4.4 OOA方法—活动图 (7/7)

• 2)泳道(续)







5.5 相关文档规范(1/4)

- 相关需求文档
 - 需求卡片
 - RUP前景文档
 - 需求规约文档
 - 术语表
 - 界面原型
 - 补充需求（可用性、可靠性、性能...）



5.5 相关文档规范(2/4)

Requirement #:

Requirement Type:

Event/Use Case#:

Description:

Rationale:

Source:

Fit Criteria:

Customer Satisfaction:

Customer Dissatisfaction:

Dependencies:

Conflicts:

Supporting Materials:

History:

Volere



5.5 相关文档规范(3/4)

- 需求规范内容
 - 背景
 - 名称、单位、参考文献
 - 产品概述
 - 目的、主要指标、主要特征、软硬件环境、费用承担方
 - 外部接口
 - 哪些系统、接口方式、数据结构和格式
 - 功能要求
 - 边界、接口、格式、特殊要求



5.5 相关文档规范(4/4)

- 需求规范内容[P66] (续)
 - 性能要求
 - 吞吐量、响应时间、可靠性、安全性等
 - 系统逻辑模型
 - 数据流程图、数据字典、有限状态机、Petri网等。
 - 验收准则
 - 采用什么实验、采用什么测试数据和测试环境
 - 附录补充材料，如重要表格、关键算法。